



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΕΠΙΚΟΙΝΩΝΙΩΝ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ

Πτυχιακή Εργασία

Τίτλος Πτυχιακής Εργασίας	Ανάπτυξη ενός Συστήματος Διαχείρισης Παραγγελιών (Order Management System) βασισμένου σε Microservices με Spring Boot και Spring Cloud Development of a Microservices-Based Order Management System Using Spring Boot and Spring Cloud
Ονοματεπώνυμο Φοιτητή	Αριστείδης Κουμουτσάκος
Πατρώνυμο	Παναγιώτης
Αριθμός Μητρώου	Π22077
Επιβλέπων	Ευθύμιος Αλεπής, Καθηγητής

Ημερομηνία Παράδοσης

Σεπτέμβριος 2026

Πνευματικά δικαιώματα και δήλωση συγγραφέα

Copyright © Αριστείδης Κουμουτσάκος, 2026

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα.

Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν αποκλειστικά τον συγγραφέα και δεν αντιπροσωπεύουν τις επίσημες θέσεις του Πανεπιστημίου Πειραιώς.

Ως συγγραφέας της παρούσας εργασίας δηλώνω πως η παρούσα εργασία δεν αποτελεί προϊόν λογοκλοπής και δεν περιέχει υλικό από μη αναφερόμενες πηγές.

Ευχαριστίες

Θα ήθελα να εκφράσω τις ειλικρινείς μου ευχαριστίες προς τον επιβλέποντα καθηγητή μου, κ. Ευθύμιο Αλεπή, για την καθοδήγηση, τις ουσιαστικές παρατηρήσεις και τη συνεχή υποστήριξή του κατά την εκπόνηση της παρούσας πτυχιακής εργασίας. Η επιστημονική του συμβολή και η εμπιστοσύνη που έδειξε στην προσπάθειά μου συνέβαλαν καθοριστικά στην οργάνωση και την ολοκλήρωση του έργου.

Ευχαριστώ επίσης την οικογένειά μου για την υπομονή, την ενθάρρυνση και τη σταθερή συμπαράσταση που μου παρείχε σε όλη τη διάρκεια των σπουδών μου. Τέλος, ευχαριστώ τους φίλους και συμφοιτητές μου για τις συζητήσεις, τη συνεργασία και το θετικό κλίμα που μοιραστήκαμε κατά την ακαδημαϊκή αυτή πορεία.

Περίληψη

Η παρούσα πτυχιακή εργασία αφορά τον σχεδιασμό και την υλοποίηση ενός Συστήματος Διαχείρισης Παραγγελιών βασισμένου σε αρχιτεκτονική microservices. Στόχος είναι η υποστήριξη της πλήρους ροής ηλεκτρονικής παραγγελίας, από τον κατάλογο προϊόντων και το καλάθι έως τον έλεγχο αποθέματος, τη δημιουργία παραγγελίας, την παρακολούθηση κατάστασης και την ειδοποίηση του πελάτη. Η λύση υλοποιείται με Java, Spring Boot και Spring Cloud στο backend και με React στο frontend. Η υποδομή περιλαμβάνει Eureka για ανακάλυψη υπηρεσιών, Spring Cloud Gateway για δρομολόγηση, OpenFeign για εσωτερικές κλήσεις, Resilience4j για ανθεκτικότητα, RabbitMQ για ασύγχρονα γεγονότα, Keycloak για αυθεντικοποίηση και εξουσιοδότηση, Redis και PostgreSQL για αποθήκευση, καθώς και Micrometer/Zipkin για κατανεμημένη ιχνηλάτηση. Η αξιολόγηση βασίζεται σε τεκμηριωμένα σενάρια χρήσης και στην επιθεώρηση της υποδομής. Διακρίνει τα παρατηρούμενα αποτελέσματα από τους μηχανισμούς που αναλύονται στον κώδικα. Η κύρια συνεισφορά της εργασίας είναι η τεκμηριωμένη εφαρμογή των αρχών microservices σε ένα συνεκτικό σύστημα παραγγελιών, με ρητά όρια υπηρεσιών και μηχανισμούς διαχείρισης των κατανεμημένων αλληλεπιδράσεων.

Επιστημονική περιοχή: Συστήματα Πληροφορικής και Κατανεμημένα Συστήματα

Λέξεις κλειδιά: microservices, Spring Boot, Spring Cloud, διαχείριση παραγγελιών, κατανεμημένα συστήματα

Abstract

This thesis presents the design and implementation of an Order Management System based on a microservices architecture. The system supports the end-to-end e-commerce order flow, from product catalogue browsing and cart management to inventory validation, order creation, status tracking, and customer notification. The backend is implemented with Java, Spring Boot, and Spring Cloud, while the user interface is implemented with React. The infrastructure includes Eureka for service discovery, Spring Cloud Gateway for routing, OpenFeign for inter-service communication, Resilience4j for fault tolerance, RabbitMQ for asynchronous events, Keycloak for authentication and authorization, Redis and PostgreSQL for persistence, and Micrometer/Zipkin for distributed tracing. The evaluation uses documented user scenarios and infrastructure inspection, distinguishing observed results from mechanisms analysed in the source code. The main contribution is a documented application of microservices principles to a coherent order-management solution with explicit service boundaries and mechanisms for managing distributed interactions.

Scientific area: Information Systems and Distributed Systems

Keywords: microservices, Spring Boot, Spring Cloud, order management, distributed systems

Πίνακας Περιεχομένων

Εισαγωγή.....	1
Αντικείμενο και σημασία της εργασίας	1
Σκοπός και στόχοι	1
Ερευνητικά ερωτήματα.....	2
Κύριο ερευνητικό ερώτημα	2
Επιμέρους ερωτήματα	2
Μεθοδολογία	2
Οριοθέτηση	2
Διάρθρωση της εργασίας	2
1. Θεωρητικό Πλαίσιο και Βιβλιογραφική Επισκόπηση.....	3
1.1 Συστήματα Διαχείρισης Παραγγελιών	3
1.1.1 Βασική ροή παραγγελίας	3
1.2 Αρχιτεκτονική Microservices	3
1.2.1 Χαρακτηριστικά και οφέλη	3
1.2.2 Προκλήσεις	4
1.2.3 Επικοινωνία και συμβόλαια υπηρεσιών	4
1.2.4 Διαχείριση δεδομένων και συνέπεια	4
1.3 Spring Boot	5
1.4 Spring Cloud	6
1.4.1 Service Discovery και API Gateway	6
1.4.2 Δηλωτικές κλήσεις μεταξύ υπηρεσιών.....	6
1.4.3 Ασύγχρονη ανταλλαγή μηνυμάτων.....	6
1.4.4 Ασφάλεια, OAuth 2.0 και JWT	7
1.4.5 Ανθεκτικότητα και κατανεμημένη ιχνηλάτηση.....	7
1.5 Συγκριτική αποτίμηση της έρευνας για τις μικροϋπηρεσίες	8
1.6 Συστήματα συστάσεων και ομοιότητα προϊόντων	9
1.6.1 Αναπαράσταση περιεχομένου και κριτήρια συνάφειας	9
1.6.2 Ερμηνευσιμότητα της κατάταξης	10
2. Ανάλυση Απαιτήσεων.....	11
2.1 Επισκόπηση συστήματος	11
2.2 Ρόλοι χρηστών	11
2.2.1 Πελάτης.....	11
2.2.2 Διαχειριστής	11
2.3 Λειτουργικές απαιτήσεις.....	12
2.3.1 Διαχείριση καταλόγου προϊόντων	12
2.3.2 Καλάθι και δέσμευση αποθέματος.....	12
2.3.3 Υποβολή και παρακολούθηση παραγγελίας	12
2.3.4 Ειδοποιήσεις και διαχείριση ροής	12
2.4 Μη λειτουργικές απαιτήσεις	13
2.5 Περιπτώσεις χρήσης	13
2.5.1 UC-01: Διαχείριση καλαθιού	13
2.5.2 UC-02: Υποβολή παραγγελίας	13
2.5.3 UC-03: Παρακολούθηση και ακύρωση παραγγελίας	13

2.5.4 UC-04: Διαχείριση καταλόγου.....	14
2.6 Περιορισμοί του τρέχοντος πεδίου.....	14
2.7 Κριτήρια αποδοχής και επιχειρησιακή σημασία.....	14
3. Σχεδιασμός Συστήματος.....	16
3.1 Σχεδιαστικοί στόχοι και αρχές.....	16
3.2 Συνολική αρχιτεκτονική.....	16
3.3 Ευθύνες και όρια των υπηρεσιών.....	17
3.4 Ροή υποβολής παραγγελίας.....	18
3.5 Μοντέλο καταστάσεων παραγγελίας.....	19
3.6 Σχεδιασμός API και επικοινωνίας.....	20
3.7 Σχεδιασμός δεδομένων.....	21
3.8 Σχεδιασμός ασφάλειας και παρατηρησιμότητας.....	21
3.9 Σχεδιαστικές παραδοχές και περιορισμοί.....	21
3.10 Αρχιτεκτονικές αντισταθμίσεις.....	21
3.11 Ταυτότητα δεδομένων και κατανομή ευθύνης.....	22
3.12 Διαχωρισμός εντολών και παρατηρήσιμων αποτελεσμάτων.....	22
3.13 Αναλυτική αποτύπωση των συμβολαίων HTTP.....	23
3.13.1 Κατάλογος και κατηγορίες.....	23
3.13.2 Καλάθι και παραγγελία.....	24
3.13.3 Απόθεμα και ενέργειες δέσμευσης.....	24
3.14 Μοντέλο αποθήκευσης και σχέσεις οντοτήτων.....	25
3.14.1 Ορισμός χαρακτηριστικού και πραγματική τιμή.....	26
3.14.2 Ταυτότητα παραγγελίας και ιστορική πληροφορία.....	26
3.14.3 Χρονική διάρκεια καλαθιού και δέσμευσης.....	27
4. Υλοποίηση.....	28
4.1 Τεχνολογική στοίβα και οργάνωση αποθετηρίου.....	28
4.2 Υλοποίηση της εφαρμογής React.....	28
4.2.1 Δρομολόγηση και έλεγχος πρόσβασης.....	28
4.2.2 Κατάλογος και λεπτομέρεια προϊόντος.....	29
4.2.3 Καλάθι και checkout.....	30
4.2.4 Παρακολούθηση παραγγελιών και διαχείριση.....	31
4.3 Υλοποίηση backend υπηρεσιών.....	33
4.3.1 Product Service.....	33
4.3.2 Αλγόριθμος βαθμολόγησης σχετικών προϊόντων.....	33
4.3.3 Πρότυπα κατηγοριών και έλεγχος χαρακτηριστικών.....	35
4.3.4 Cart Service.....	36
4.3.5 Συγχρονισμός καλαθιού και προσωρινή δέσμευση αποθέματος.....	36
4.3.6 Inventory Service.....	37
4.3.7 Order Service.....	37
4.3.8 Idempotency και οριστικοποίηση checkout.....	37
4.3.9 Notification Service.....	38
4.3.10 Ροή ειδοποίησης βάσει γεγονότος.....	38
4.4 Χειρισμός σφαλμάτων και διασύνδεση υπηρεσιών.....	38
4.5 Μοντέλο μεταβάσεων και ιστορικό παραγγελίας.....	39
4.6 Διαχείριση κατάστασης και επικοινωνίας στο frontend.....	39
4.7 Συγχρονισμός κοινής κατάστασης καλαθιού.....	40

4.8 Χρηματικά ποσά και ανακατασκευή αποκρίσεων	41
4.9 Σύνοψη υλοποίησης.....	41
5. Υποδομή Spring Cloud και Κατανεμημένες Υπηρεσίες	43
5.1 Ρόλος της υποδομής Spring Cloud	43
5.2 Eureka Discovery Server	43
5.3 Spring Cloud Gateway	44
5.4 OpenFeign για επικοινωνία υπηρεσία-προς-υπηρεσία	44
5.5 Resilience4j και χειρισμός μερικών αποτυχιών	45
5.6 RabbitMQ και ασύγχρονες ειδοποιήσεις	45
5.7 Keycloak, OAuth2 και JWT	46
5.8 Distributed tracing με Micrometer και Zipkin	46
5.9 Τοπική υποδομή με Docker Compose.....	47
5.10 Συσχέτιση παραγγελίας γεγονότος και ειδοποίησης.....	48
5.11 Ταυτότητα χρήστη και κατανομή εξουσιοδότησης	48
5.12 Συμπεράσματα υποδομής	49
6. Έλεγχος και Αξιολόγηση	50
6.1 Μεθοδολογία	50
6.2 Σενάρια λειτουργικού ελέγχου.....	50
6.2.1 TC-01: Περιήγηση στον κατάλογο	50
6.2.2 TC-02: Διαχείριση καλαθιού	50
6.2.3 TC-03: Υποβολή και επιβεβαίωση παραγγελίας	51
6.2.4 TC-04: Ακύρωση παραγγελίας	52
6.2.5 TC-05: Restock αποθέματος από διαχειριστή.....	53
6.2.6 TC-06: Ασύγχρονη ειδοποίηση παραγγελίας	54
6.3 Επιβεβαίωση υποδομής και ασφάλειας.....	56
6.4 Ανάλυση τεκμηρίων και εύρος συμπερασμάτων	57
6.5 Αναπαραγωγικότητα της λειτουργικής επίδειξης	58
6.6 Πίνακας ιχνηλασιμότητας των αποτελεσμάτων	58
6.7 Συνοπτική αποτίμηση	59
7. Συμπεράσματα και Μελλοντική Εργασία.....	60
7.1 Συμπεράσματα.....	60
7.2 Απάντηση στα ερευνητικά ερωτήματα	60
7.3 Μελλοντική εργασία	60
7.4 Αποτίμηση της σχεδιαστικής συνεισφοράς.....	61
7.5 Σύνδεση της συνεισφοράς με τη βιβλιογραφία	61
Πίνακας ορολογίας	63
Πίνακας ακρωνυμίων.....	64
Βιβλιογραφία.....	65
Παράρτημα Α Αντιπροσωπευτικός πηγαίος κώδικας.....	68
A.1 Βαθμολόγηση σχετικών προϊόντων.....	68
A.2 Έλεγχος προτύπου κατηγορίας.....	68
A.3 Επιβεβαίωση δέσμευσης.....	69
A.4 Κατανάλωση γεγονότος ειδοποίησης	70
Παράρτημα Β Οδηγίες τοπικής εκτέλεσης.....	71
B.1 Προϋποθέσεις και δομή έργου	71
B.2 Υποδομές και βάσεις δεδομένων	71

B.3 Διαμόρφωση ταυτότητας	71
B.4 Εκκίνηση εφαρμογών	71
B.5 Ειδοποιήσεις και επανάληψη επίδειξης	72
Παράρτημα Γ Αποθετήριο και ψηφιακό υλικό	73

Κατάλογος Εικόνων

Εικόνα 4.1: Αρχική σελίδα και κατάλογος προϊόντων.	29
Εικόνα 4.2: Σελίδα λεπτομέρειας προϊόντος.	29
Εικόνα 4.3: Χαρακτηριστικά και σχετικές προτάσεις προϊόντος.	30
Εικόνα 4.4: Σελίδα καλαθιού.	30
Εικόνα 4.5: Σελίδα ανασκόπησης checkout.	31
Εικόνα 4.6: Επιτυχής επιβεβαίωση παραγγελίας.	31
Εικόνα 4.7: Ιστορικό παραγγελιών πελάτη.	32
Εικόνα 4.8: Διαχείριση προτύπων κατηγορίας.	32
Εικόνα 4.9: Διαχείριση καταλόγου και αποθέματος.	32
Εικόνα 4.10: Διαχειριστική ροή παραγγελιών.	33
Εικόνα 5.1: Eureka dashboard με τις εγγεγραμμένες υπηρεσίες.	44
Εικόνα 5.2: Ρόλοι ADMIN και CUSTOMER στο Keycloak.	46
Εικόνα 5.3: Trace αιτήματος από API Gateway προς Order Service στο Zipkin.	47
Εικόνα 6.1: Προβολή καταλόγου προϊόντων και διαθέσιμων χειριστηρίων.	50
Εικόνα 6.2: Τεκμήριο διαχείρισης καλαθιού και υπολογισμού συνόλου.	51
Εικόνα 6.3: Ανασκόπηση στοιχείων πριν από την υποβολή παραγγελίας.	51
Εικόνα 6.4: Επιτυχής επιβεβαίωση παραγγελίας με κατάσταση PENDING.	52
Εικόνα 6.5: Παραγγελία PENDING με διαθέσιμη ενέργεια ακύρωσης.	52
Εικόνα 6.6: Διάλογος επιβεβαίωσης ακύρωσης παραγγελίας.	52
Εικόνα 6.7: Ιστορικό παραγγελιών μετά από επιτυχή ακύρωση.	53
Εικόνα 6.8: Εισαγωγή ποσότητας restock από τον πίνακα διαχείρισης.	53
Εικόνα 6.9: Επιτυχής ενημέρωση αποθέματος μετά από restock.	54
Εικόνα 6.10: Συνολικές ενδείξεις μηνυμάτων και δραστηριότητας στο RabbitMQ.	55
Εικόνα 6.11: Email επιβεβαίωσης παραγγελίας στο Mailtrap.	56
Εικόνα 6.12: Καταχωρισμένες υπηρεσίες στο Eureka Discovery Server.	56
Εικόνα 6.13: Ρόλοι ADMIN και CUSTOMER στο Keycloak realm.	57
Εικόνα 6.14: Trace αιτήματος από API Gateway προς Order Service.	57

Κατάλογος Πινάκων

Πίνακας 2.1: Αντιστοίχιση ρόλων και βασικών αρμοδιοτήτων.	11
Πίνακας 2.2: Κριτήρια αποδοχής των βασικών επιχειρησιακών διαδρομών.	14
Πίνακας 3.1: Υπηρεσίες, ευθύνες και κύρια αποθήκευση δεδομένων.	18
Πίνακας 3.2: Συμβόλαια HTTP προϊόντων και κατηγοριών.	23
Πίνακας 3.3: Συμβόλαια HTTP καλαθιού και παραγγελίας.	24
Πίνακας 3.4: Συμβόλαια HTTP διαχείρισης αποθέματος.	25
Πίνακας 3.5: Βασικές οντότητες και σχέσεις του καταλόγου και των παραγγελιών.	25
Πίνακας 3.6: Διαφορετικοί κύκλοι ζωής δεδομένων στην υλοποίηση.	27
Πίνακας 4.1: Παράγοντες βαθμολόγησης σχετικών προϊόντων.	34
Πίνακας 4.2: Καταστάσεις του CartProvider και επίδραση στη διεπαφή.	40
Πίνακας 5.1: Κατανομή ελέγχων ταυτότητας και δικαιωμάτων.	48
Πίνακας 6.1: Αντιστοίχιση σεναρίων με τα διαθέσιμα τεκμήρια.	59

Κατάλογος Διαγραμμάτων

Διάγραμμα 3.1: Συνολική αρχιτεκτονική του συστήματος διαχείρισης παραγγελιών.	17
Διάγραμμα 3.2: Ροή υποβολής παραγγελίας και δημοσίευσης συμβάντος ειδοποίησης.	19
Διάγραμμα 3.3: Μοντέλο καταστάσεων παραγγελίας.	20

Εισαγωγή

Αντικείμενο και σημασία της εργασίας

Τα συστήματα διαχείρισης παραγγελιών συντονίζουν μια ακολουθία επιχειρησιακών λειτουργιών, από την παρουσίαση των προϊόντων και το καλάθι αγορών έως τον έλεγχο αποθέματος, τη δημιουργία παραγγελίας, την παρακολούθηση της κατάστασής της και την ενημέρωση του πελάτη. Η ανάγκη ανεξάρτητης εξέλιξης αυτών των λειτουργιών, σε συνδυασμό με την ανάγκη για διαθεσιμότητα και παρατηρησιμότητα, καθιστά τις κατανεμημένες αρχιτεκτονικές ιδιαίτερα σχετικές για τέτοια συστήματα.

Στο πλαίσιο ενός ηλεκτρονικού καταστήματος, η ορθή διαχείριση παραγγελίας δεν περιορίζεται στην αποθήκευση μιας εγγραφής αγοράς. Απαιτεί συνεπή έλεγχο διαθεσιμότητας, αποτροπή ανεπιθύμητων διπλών υποβολών, διατήρηση ιστορικού κατάστασης, ασφαλή ταυτοποίηση του χρήστη και έγκαιρη ενημέρωση μετά την ολοκλήρωση της συναλλαγής. Επομένως, η αρχιτεκτονική του συστήματος πρέπει να υποστηρίζει τόσο τις άμεσες επιχειρησιακές ανάγκες όσο και τις απαιτήσεις αξιοπιστίας μιας εφαρμογής με πολλά αλληλεπιδρώντα υποσυστήματα.

Η αρχιτεκτονική *microservices* προσεγγίζει το σύστημα ως ένα σύνολο μικρότερων, χαλαρά συνδεδεμένων υπηρεσιών που επικοινωνούν με ελαφρούς μηχανισμούς. Η βιβλιογραφία αναγνωρίζει ότι αυτή η προσέγγιση μπορεί να βελτιώσει τη δυνατότητα ανεξάρτητης ανάπτυξης και κλιμάκωσης, αλλά ταυτόχρονα εισάγει δυσκολίες επικοινωνίας, λειτουργίας και παρακολούθησης των υπηρεσιών [1, 2, 3].

Η παρούσα εργασία εξετάζει αυτές τις αντισταθμίσεις μέσα από την κατασκευή ενός συγκεκριμένου τεχνολογικού τεχνουργήματος. Αντί να παρουσιάζει τη μετάβαση από έναν υπάρχοντα μονολιθικό κώδικα, σχεδιάζει εξ αρχής υπηρεσίες με διακριτές ευθύνες για προϊόντα, καλάθι, απόθεμα, παραγγελίες και ειδοποιήσεις. Η λύση συμπληρώνεται από υποδομή που επιτρέπει ανακάλυψη υπηρεσιών, ελεγχόμενη δρομολόγηση αιτημάτων, ανθεκτικότητα και παρατηρησιμότητα.

Σκοπός και στόχοι

Σκοπός της εργασίας είναι ο σχεδιασμός, η υλοποίηση και η αξιολόγηση ενός Συστήματος Διαχείρισης Παραγγελιών βασισμένου σε *microservices*. Το σύστημα υλοποιείται με Spring Boot και Spring Cloud, προσφέρει διεπαφή χρήστη με React και ενσωματώνει λειτουργίες καταλόγου προϊόντων, καλαθιού, αποθέματος, παραγγελιών, ειδοποιήσεων και διαχειριστικού ελέγχου.

Κεντρική επιδίωξη δεν είναι απλώς η παρουσίαση μιας λειτουργικής διεπαφής ηλεκτρονικού εμπορίου, αλλά η τεκμηρίωση των αρχιτεκτονικών αποφάσεων που επιτρέπουν στις επιμέρους λειτουργίες να συνεργάζονται ως κατανεμημένο σύστημα. Για τον λόγο αυτό, η εργασία συνδέει τις απαιτήσεις του επιχειρησιακού προβλήματος με μηχανισμούς του οικοσυστήματος Spring και με μετρήσιμα σενάρια ελέγχου.

- Ανάλυση των λειτουργικών και μη λειτουργικών απαιτήσεων του συστήματος.
- Σχεδιασμός σαφών επιχειρησιακών ορίων για τις ανεξάρτητες υπηρεσίες.
- Υλοποίηση της ροής από το καλάθι μέχρι την καταχώριση της παραγγελίας και την ειδοποίηση του χρήστη.
- Αξιοποίηση μηχανισμών *service discovery*, *API gateway*, ανθεκτικότητας, μηνυμάτων, ασφάλειας και *distributed tracing*.
- Αξιολόγηση της ορθότητας των βασικών σεναρίων και τεκμηρίωση των περιορισμών της υλοποίησης.

Ερευνητικά ερωτήματα

Κύριο ερευνητικό ερώτημα

Πώς μπορεί να σχεδιαστεί και να υλοποιηθεί ένα ασφαλές, επεκτάσιμο και ανθεκτικό Σύστημα Διαχείρισης Παραγγελιών με αρχιτεκτονική microservices χρησιμοποιώντας Spring Boot και Spring Cloud;

Επιμέρους ερωτήματα

- Πώς υποστηρίζει ο διαχωρισμός σε υπηρεσίες τις βασικές επιχειρησιακές λειτουργίες ενός Order Management System;
- Πώς συνεργάζονται η ανακάλυψη υπηρεσιών, το API Gateway και οι δηλωτικές κλήσεις υπηρεσία-προς-υπηρεσία;
- Πώς ενισχύουν η ανθεκτικότητα, η ασύγχρονη ανταλλαγή μηνυμάτων και η κατανεμημένη ιχνηλάτηση την αξιοπιστία και την παρατηρησιμότητα του συστήματος;

Μεθοδολογία

Η εργασία ακολουθεί προσανατολισμένη στον σχεδιασμό και την υλοποίηση μεθοδολογία. Το πρόβλημα προσδιορίζεται, ορίζονται οι στόχοι της λύσης, σχεδιάζεται και υλοποιείται το τεχνολογικό τεχνούργημα, παρουσιάζονται σενάρια χρήσης και αξιολογούνται τα αποτελέσματα. Η ακολουθία αυτή ευθυγραμμίζεται με τη Design Science Research Methodology των Peffers et al. [4].

Η αξιολόγηση συνδέει τα στιγμιότυπα της διεπαφής με συγκεκριμένα σενάρια: παρουσίαση καλαθιού, δημιουργία παραγγελίας, ακύρωση, αναπλήρωση αποθέματος και παραλαβή ειδοποίησης στο sandbox. Η επιθεώρηση της υποδομής συμπληρώνει τα σενάρια με καταχωρίσεις στο Eureka, ρόλους στο Keycloak και ένα κατανεμημένο ίχνος ανάκτησης ιστορικού. Οι μηχανισμοί ανθεκτικότητας αναλύονται στον κώδικα, χωρίς να παρουσιάζονται μη εκτελεσμένα πειράματα αποτυχίας ως αποτελέσματα.

Οριοθέτηση

Η εργασία καλύπτει διαχείριση προϊόντων και κατηγοριών, καλάθι, έλεγχο και δέσμευση αποθέματος, καταχώριση και παρακολούθηση παραγγελιών, ειδοποιήσεις και διαχειριστικές λειτουργίες. Δεν περιλαμβάνει διασύνδεση με πραγματικό πάροχο πληρωμών, εξωτερικό σύστημα μεταφορών ή πλήρη εμπορική λειτουργία επιστροφών και επιστροφής χρημάτων.

Διάρθρωση της εργασίας

Το πρώτο κεφάλαιο παρουσιάζει το θεωρητικό υπόβαθρο. Ακολουθούν η ανάλυση απαιτήσεων, ο σχεδιασμός του συστήματος, η υλοποίηση, ο έλεγχος και η αξιολόγηση. Η εργασία ολοκληρώνεται με συμπεράσματα, μελλοντικές επεκτάσεις, τη βιβλιογραφία και τα παραρτήματα.

1. Θεωρητικό Πλαίσιο και Βιβλιογραφική Επισκόπηση

1.1 Συστήματα Διαχείρισης Παραγγελιών

Ένα Σύστημα Διαχείρισης Παραγγελιών ενοποιεί πληροφορίες προϊόντων, διαθεσιμότητας και πελατειακών παραγγελιών ώστε μια παραγγελία να ακολουθεί ελεγχόμενη ροή από την υποβολή έως την ολοκλήρωσή της. Για την παρούσα εργασία, η βασική μονάδα συντονισμού είναι η παραγγελία: συγκεντρώνει τα επιλεγμένα είδη, το δεσμευμένο απόθεμα, την κατάσταση εκτέλεσης και το ιστορικό μεταβολών.

Η επιλογή της παραγγελίας ως κεντρικής επιχειρησιακής οντότητας επιτρέπει τον σαφή διαχωρισμό των ευθυνών. Ο κατάλογος προϊόντων είναι υπεύθυνος για την πληροφορία που παρουσιάζεται στον χρήστη, το καλάθι διατηρεί τις προσωρινές επιλογές, το απόθεμα αποφασίζει για τη διαθεσιμότητα και η υπηρεσία παραγγελιών διατηρεί την τελική καταγραφή και το ιστορικό της. Η ειδοποίηση εκτελείται ως ανεξάρτητη συνέπεια της επιτυχούς τοποθέτησης παραγγελίας.

1.1.1 Βασική ροή παραγγελίας

Η λειτουργική ροή αρχίζει με την εξερεύνηση του καταλόγου και την προσθήκη ειδών στο καλάθι. Πριν από την οριστική υποβολή, το σύστημα ελέγχει τη διαθεσιμότητα και δεσμεύει ποσότητες ώστε να αποφευχθεί η υπερπώληση. Μετά την επιβεβαίωση της δέσμευσης δημιουργείται η παραγγελία, καθαρίζεται το καλάθι και δημοσιεύεται συμβάν για την αποστολή ειδοποίησης. Η διάκριση αυτή κάνει ρητό ποια υπηρεσία είναι υπεύθυνη για κάθε μεταβολή κατάστασης.

1.2 Αρχιτεκτονική Microservices

1.2.1 Χαρακτηριστικά και οφέλη

Η αρχιτεκτονική microservices οργανώνει μια εφαρμογή ως συλλογή ανεξάρτητων υπηρεσιών που αντιστοιχούν σε διακριτές επιχειρησιακές ευθύνες. Η προσέγγιση διευκολύνει την ανεξάρτητη ανάπτυξη και εξέλιξη υπηρεσιών, όμως ο ακριβής καθορισμός των ορίων και της επικοινωνίας τους αποτελεί κρίσιμη αρχιτεκτονική απόφαση [5, 2].

Ο διαχωρισμός δεν πρέπει να γίνεται με μοναδικό κριτήριο το τεχνολογικό επίπεδο, όπως για παράδειγμα ξεχωριστή υπηρεσία για κάθε βάση δεδομένων ή κάθε οθόνη. Αντίθετα, τα όρια των υπηρεσιών ορίζονται με βάση τις επιχειρησιακές ευθύνες και τις μεταβολές που πρέπει να εξελίσσονται ανεξάρτητα. Η προσέγγιση αυτή ελαχιστοποιεί την άμεση εξάρτηση μεταξύ υπηρεσιών και καθιστά σαφέστερο ποια υπηρεσία είναι ιδιοκτήτης κάθε πληροφορίας.

Η βιβλιογραφία συνδέει την επιτυχία μιας αρχιτεκτονικής microservices με συνεχή αρχιτεκτονική εξέλιξη, σαφή granularity υπηρεσιών, κατάλληλα όρια domain και ώριμες πρακτικές λειτουργίας. Οι αρχές αυτές συμπληρώνουν το θεωρητικό πλαίσιο της παρούσας λύσης [6, 7, 8].

Ως αφετηρία για τον καθορισμό των ορίων, ο Parnas προτείνει τη διάσπαση με βάση πληροφορίες και σχεδιαστικές αποφάσεις που κάθε μονάδα οφείλει να αποκρύπτει από τις υπόλοιπες [9]. Στο σύστημα παραγγελιών, το κριτήριο αυτό βοηθά να ερμηνευθεί ο χωρισμός καταλόγου, καταθέσεων, αποθέματος και παραγγελίας. Η διαχείριση χαρακτηριστικών προϊόντος ανήκει στον κατάλογο, ενώ η δέσμευση ποσότητας στο απόθεμα. Έτσι, μια αλλαγή στον εσωτερικό τρόπο αποθήκευσης μιας υπηρεσίας δεν χρειάζεται να εκτεθεί ως λεπτομέρεια στο συμβόλαιο των άλλων.

Στην ανάπτυξη microservices, κάθε υπηρεσία εκθέτει ένα συγκεκριμένο συμβόλαιο και αναλαμβάνει μια διακριτή επιχειρησιακή ευθύνη. Η ανεξάρτητη ανάπτυξη και διάθεση των υπηρεσιών συνδέεται με τον καθορισμό των ορίων τους και με τον τρόπο συνεργασίας τους [7,

8]. Για την παρούσα εργασία, η διάκριση είναι πρακτική: κάθε backend εφαρμογή εκκινεί ανεξάρτητα και διαχειρίζεται τις δικές της λειτουργίες, αλλά συνεργάζεται με τις άλλες για την ολοκλήρωση της ίδιας επιχειρησιακής ροής. Επομένως, η αυτονομία της υπηρεσίας δεν καταργεί την ανάγκη για σαφείς διεπαφές επικοινωνίας.

1.2.2 Προκλήσεις

Η κατανομή μιας εφαρμογής δημιουργεί νέες απαιτήσεις: αντιμετώπιση μερικών αποτυχιών, παρακολούθηση αιτημάτων που διασχίζουν περισσότερες υπηρεσίες, διαχείριση δεδομένων ανά υπηρεσία και συντονισμό αλλαγών στα συμβόλαια επικοινωνίας. Οι προκλήσεις αυτές τεκμηριώνονται στις συστηματικές ανασκοπήσεις της βιβλιογραφίας και καθοδηγούν τις επιλογές σχεδιασμού της παρούσας υλοποίησης [1, 3].

Ειδικά στη ροή παραγγελίας, μια αποτυχία δεν είναι αναγκαστικά καθολική. Μπορεί, για παράδειγμα, να μην είναι διαθέσιμη η υπηρεσία αποθέματος ενώ οι υπόλοιπες υπηρεσίες λειτουργούν κανονικά. Η αρχιτεκτονική οφείλει να αντιμετωπίζει αυτήν την περίπτωση με σαφή μήνυμα προς τον χρήστη, χωρίς να δημιουργεί ελλιπή παραγγελία ή να διατηρεί ασυνεπή δέσμευση αποθέματος. Παρόμοια, η αποτυχία αποστολής ειδοποίησης δεν πρέπει από μόνη της να αναιρεί την επιτυχή καταχώριση της παραγγελίας.

Η αντιμετώπιση μερικών αποτυχιών και η προστασία από cascading failures αποτελούν ουσιώδεις διαστάσεις της πρακτικής microservices. Για τον λόγο αυτό, η εργασία συνδυάζει ρητά όρια υπηρεσιών με μηχανισμούς ανθεκτικότητας και παρατηρησιμότητας [10, 11].

1.2.3 Επικοινωνία και συμβόλαια υπηρεσιών

Οι υπηρεσίες επικοινωνούν είτε συγχρονικά, όταν η άμεση απάντηση είναι απαραίτητη για τη συνέχιση μιας επιχειρησιακής ροής, είτε ασύγχρονα, όταν η ενέργεια μπορεί να εκτελεστεί ως συνέπεια ενός γεγονότος. Στην παρούσα λύση, ο έλεγχος και η επιβεβαίωση αποθέματος είναι συγχρονικά βήματα της υποβολής παραγγελίας, ενώ η ειδοποίηση υλοποιείται ως ασύγχρονη αντίδραση στο γεγονός τοποθέτησης παραγγελίας. Αυτή η διάκριση περιορίζει τη σύζευξη εκεί όπου δεν απαιτείται άμεση απάντηση.

Τα συμβόλαια επικοινωνίας αποτελούν σημαντικό στοιχείο της αρχιτεκτονικής, επειδή η αλλαγή ενός API μπορεί να επηρεάσει περισσότερους από έναν καταναλωτές. Για τον λόγο αυτό, κάθε υπηρεσία πρέπει να εκθέτει δεδομένα που αρκούν για την ολοκλήρωση της συγκεκριμένης συνεργασίας, χωρίς να μεταφέρει την εσωτερική δομή των δεδομένων της σε άλλες υπηρεσίες. Η αρχή αυτή μειώνει την εξάρτηση και επιτρέπει την εξέλιξη μιας υπηρεσίας με μικρότερο κίνδυνο ασυμβατότητας.

Η προσέγγιση REST οργανώνει την επικοινωνία γύρω από πόρους και περιορισμούς της αρχιτεκτονικής του Ιστού. Ο Fielding εξετάζει τη σχέση αυτών των περιορισμών με τη διαλειτουργικότητα και τη δυνατότητα εξέλιξης μιας δικτυακής εφαρμογής [12]. Στην εφαρμογή, οι μέθοδοι HTTP, οι διαδρομές και οι αποκρίσεις καθορίζονται από τους controllers και αποτελούν τα συμβόλαια που χρησιμοποιεί το frontend. Η περιγραφή των endpoints συμπληρώνει το αρχιτεκτονικό πλαίσιο, προσδιορίζοντας τις συγκεκριμένες ενέργειες που εκθέτει κάθε υπηρεσία.

1.2.4 Διαχείριση δεδομένων και συνέπεια

Σε ένα κατανεμημένο σύστημα, η διατήρηση μιας ενιαίας βάσης δεδομένων για όλες τις υπηρεσίες θα αναιρούσε σε μεγάλο βαθμό την ανεξαρτησία τους. Η συνήθης προσέγγιση είναι η κάθε υπηρεσία να έχει την ευθύνη των δικών της δεδομένων και να συνεργάζεται με τις υπόλοιπες μέσω API ή γεγονότων. Η επιλογή αυτή αυξάνει την αυτονομία, αλλά καθιστά δυσκολότερο τον συντονισμό μεταβολών που αφορούν περισσότερες από μία υπηρεσίες [1, 3].

Η ροή παραγγελίας αποτελεί χαρακτηριστικό παράδειγμα: η καταχώριση παραγγελίας και η οριστική μείωση αποθέματος συνδέονται επιχειρησιακά, αλλά ανήκουν σε διαφορετικές ευθύνες. Η χρήση προσωρινής δέσμευσης πριν από την τελική επιβεβαίωση μειώνει την πιθανότητα υπερπώλησης χωρίς να απαιτεί μία κοινή κατανεμημένη συναλλαγή για όλες τις υπηρεσίες. Η

τελική λύση πρέπει να τεκμηριώσει ρητά ποιες εγγυήσεις συνέπειας παρέχονται και ποιες περιπτώσεις αποτυχίας παραμένουν περιορισμοί του συστήματος.

Η επιλογή διακριτών μοντέλων δεδομένων προϋποθέτει αξιόπιστη τοπική αποθήκευση και ελεγχόμενες επιχειρησιακές μεταβάσεις. Η σχεσιακή μοντελοποίηση, οι sagas και οι αντισταθμιστικές ενέργειες αποτελούν βασικές θεωρητικές προσεγγίσεις για αυτό το πρόβλημα [13, 14, 15].

Σε επίπεδο αποθήκευσης, η συναλλαγή οριοθετεί μια μονάδα μεταβολής που πρέπει να ολοκληρωθεί ή να αναιρεθεί ως σύνολο. Η τεκμηρίωση Spring Data JPA εξηγεί τον καθορισμό συναλλακτικών ορίων στα repositories και στις υπηρεσίες, ενώ η PostgreSQL περιγράφει την απομόνωση μεταξύ συναλλαγών [16, 17]. Με χωριστές υπηρεσίες, όμως, μια τοπική συναλλαγή δεν περιλαμβάνει αυτομάτως τις απομακρυσμένες ενέργειες του checkout. Η σειρά ανάκτησης καλαθιού, επιβεβαίωσης δέσμευσης και αποθήκευσης παραγγελίας περιγράφεται επομένως ως επιχειρησιακή συνεργασία, με σαφή διάκριση από τις εγγυήσεις μιας μεμονωμένης βάσης δεδομένων.

Αντίστοιχα, ο έλεγχος ταυτόχρονων προσβάσεων αφορά τον συντονισμό αναγνώσεων και εγγραφών ώστε οι μεταβολές διαφορετικών ενεργειών να μην αλληλεπιδρούν ανεξέλεγκτα. Η απομόνωση συναλλαγών προσδιορίζει ποιες αλληλεπιδράσεις είναι επιτρεπτές κατά την πρόσβαση στα δεδομένα [17]. Στο πεδίο της εργασίας, το επιχειρησιακό ενδιαφέρον εντοπίζεται στην ποσότητα αποθέματος και στην ενεργή δέσμευσή της. Η περιγραφή των σχετικών μεθόδων εξηγεί ποιους κανόνες εφαρμόζεται και σε ποια υπηρεσία. Δεν αποτελεί απόδειξη ορθότητας υπό οποιοδήποτε σενάριο ταυτόχρονου φορτίου.

Για την κατανομημένη αποθήκευση, η σημασία της συνέπειας πρέπει να συνδέεται με την επιχειρησιακή ενέργεια που εξυπηρετεί και με τον τρόπο επικοινωνίας των υπηρεσιών [11, 18]. Η δυνατότητα δημιουργίας παραγγελίας απαιτεί διαφορετική απόφαση από την εμφάνιση μιας μεταγενέστερης ειδοποίησης. Στην εφαρμογή δεν παρουσιάζεται μηχανισμός αναπαραγωγής δεδομένων ως υλοποίηση eventual consistency. Η εννοιολογική διάκριση εξηγεί γιατί η ενημέρωση με email ακολουθεί διαφορετική ροή από τη σύγχρονη επιβεβαίωση checkout, χωρίς η ολοκλήρωση των δύο ενεργειών να θεωρείται ένα ενιαίο χρονικό γεγονός.

Υπό συνθήκες διαμερισμού του δικτύου, οι Gilbert και Lynch αποδεικνύουν ότι δεν μπορούν να διατηρούνται ταυτόχρονα οι συγκεκριμένες εγγυήσεις συνέπειας και διαθεσιμότητας του μοντέλου τους [19]. Το αποτέλεσμα θέτει όρια στις γενικές υποσχέσεις για κατανομημένα συστήματα. Δεν επιτρέπει, μόνο από την παρουσία Redis ή PostgreSQL, να χαρακτηριστεί ολόκληρη η εφαρμογή ως CP ή AP. Στην εργασία χρησιμοποιείται ως θεωρητικό πλαίσιο και όχι ως ταξινόμηση ή πειραματικό αποτέλεσμα της υλοποίησης.

Πέρα από τον μηχανισμό αποθήκευσης, χρειάζεται να προσδιορίζονται οι επιχειρησιακές αμετάβλητες συνθήκες. Οι Bailis et al. εξετάζουν τη σχέση τέτοιων συνθηκών με την ανάγκη συντονισμού [20]. Στο έργο, αντίστοιχα παραδείγματα είναι η συμβατότητα χαρακτηριστικών με το πρότυπο κατηγορίας και η επιτρεπόμενη ποσότητα δέσμευσης. Η αντιστοίχιση βοηθά να αιτιολογηθεί η ύπαρξη ελέγχων στις αρμόδιες υπηρεσίες, χωρίς να δηλώνεται εφαρμογή της τυπικής μεθόδου invariant confluence της μελέτης.

1.3 Spring Boot

Στην παρούσα υλοποίηση, το Spring Boot αποτελεί το πλαίσιο των ανεξάρτητων backend εφαρμογών. Τα αρχεία pom.xml και οι κλάσεις διαμόρφωσης προσδιορίζουν τις εξαρτήσεις για web endpoints, αποθήκευση, ασφάλεια και ανταλλαγή μηνυμάτων. Η παρουσίαση των συγκεκριμένων δυνατοτήτων προκύπτει από τον πηγαίο κώδικα του έργου, ενώ οι αρχές διαχωρισμού και συνεργασίας των υπηρεσιών εξετάζονται στο προηγούμενο θεωρητικό πλαίσιο. Η επίσημη τεκμηρίωση Spring Boot περιγράφει τις δυνατότητες και τη διαμόρφωση του πλαισίου που αξιοποιούνται σε αυτό το επίπεδο [21].

Η χρήση ενός κοινού πλαισίου στις υπηρεσίες δεν αναιρεί την αυτονομία τους. Κάθε υπηρεσία παραμένει ξεχωριστή εφαρμογή Spring Boot με δική της ρύθμιση, δικό της κύκλο εκτέλεσης και σαφές API. Με αυτόν τον τρόπο, η ομοιογένεια της τεχνολογικής στοίβας μειώνει την περιττή πολυπλοκότητα, ενώ τα επιχειρησιακά όρια διατηρούνται ανεξάρτητα.

1.4 Spring Cloud

Στο επίπεδο της υλοποίησης, το Spring Cloud συνδέει τις επιχειρησιακές υπηρεσίες με κοινή υποδομή ανακάλυψης, δρομολόγησης και δηλωτικών κλήσεων. Η επίσημη τεκμηρίωση παρουσιάζει αυτές τις δυνατότητες ως εργαλεία για επαναλαμβανόμενα πρότυπα καταναμημένων συστημάτων και συνδέει την επιλογή release train με την αντίστοιχη γενιά Spring Boot [22]. Η εργασία εφαρμόζει τις συγκεκριμένες αρχιτεκτονικές ανάγκες μέσω Eureka, Spring Cloud Gateway και OpenFeign, όπως αποτυπώνεται στις ρυθμίσεις και στις διεπαφές των επιμέρους εφαρμογών.

Η επιλογή του Spring Cloud συνδέεται άμεσα με τις απαιτήσεις που προκύπτουν όταν μια ροή παραγγελίας διασχίζει περισσότερες υπηρεσίες. Οι μηχανισμοί του επιτρέπουν να αποφεύγεται η ενσωμάτωση σταθερών φυσικών διευθύνσεων στον επιχειρησιακό κώδικα, να συγκεντρώνεται η δρομολόγηση και να εφαρμόζεται χειρισμός αποτυχιών στις απομακρυσμένες κλήσεις.

1.4.1 Service Discovery και API Gateway

Για τη συνεργασία ανεξάρτητων υπηρεσιών χρειάζεται να προσδιορίζονται οι διαθέσιμοι δικτυακοί προορισμοί και το σημείο εισόδου των εξωτερικών αιτημάτων. Η τεκμηρίωση Spring Cloud Netflix περιγράφει την υποδομή Eureka, ενώ το Spring Cloud Gateway παρέχει το πλαίσιο ορισμού διαδρομών και φίλτρων [23, 24]. Στο έργο, η εγγραφή στο Eureka και οι διαδρομές του Gateway εκφράζουν τις συγκεκριμένες επιλογές ανακάλυψης και πρόσβασης, χωρίς να μεταφέρουν επιχειρησιακές ευθύνες στην υποδομή.

Στην αρχιτεκτονική που μελετάται, ο χρήστης και το frontend επικοινωνούν με ένα ενιαίο σημείο εισόδου αντί να γνωρίζουν τη θύρα ή τη διεύθυνση κάθε backend υπηρεσίας. Οι εσωτερικές υπηρεσίες, αντίστοιχα, εντοπίζουν τις απαραίτητες εξαρτήσεις μέσω λογικών ονομάτων. Η επιλογή αυτή διευκολύνει την αλλαγή ή την κλιμάκωση μιας υπηρεσίας χωρίς αλλαγές στον client κώδικα.

1.4.2 Δηλωτικές κλήσεις μεταξύ υπηρεσιών

Στις απομακρυσμένες κλήσεις, η διεπαφή που χρησιμοποιεί ο καλών διαχωρίζεται από τη μεταφορά των δεδομένων στο δίκτυο. Το Spring Cloud OpenFeign επιτρέπει την περιγραφή HTTP clients μέσω διεπαφών Java και υποστηρίζει τη σύνδεσή τους με τη διαμόρφωση της εφαρμογής [25]. Στην υλοποίηση, οι δηλώσεις των clients προσδιορίζουν την υπηρεσία προορισμού, τη διαδρομή και τα δεδομένα κάθε κλήσης. Η δηλωτική μορφή διευκολύνει την ανάγνωση του συμβολαίου επικοινωνίας, χωρίς να μετατρέπει την απομακρυσμένη εκτέλεση σε τοπική συναλλαγή.

Η ευκολία της δηλωτικής επικοινωνίας δεν εξαλείφει τον κίνδυνο των δικτυακών αποτυχιών. Κάθε απομακρυσμένη κλήση μπορεί να καθυστερήσει, να αποτύχει ή να επιστρέψει απρόβλεπτο σφάλμα. Επομένως, η χρήση HTTP client πρέπει να συνδυάζεται με χρονικά όρια, κανόνες χειρισμού σφαλμάτων και μηχανισμούς ανθεκτικότητας, ιδίως όταν η απάντηση είναι προϋπόθεση για τη συνέχιση μιας επιχειρησιακής ροής.

1.4.3 Ασύγχρονη ανταλλαγή μηνυμάτων

Με ασύγχρονη ανταλλαγή μηνυμάτων, ο παραγωγός δηλώνει ένα γεγονός χωρίς να καλεί απευθείας κάθε ενδιαφερόμενο καταναλωτή. Οι Eugster et al. διακρίνουν την αποσύζευξη ως προς τη γνώση των συμμετεχόντων, τον χρόνο και τον συγχρονισμό της επικοινωνίας [18]. Η ειδοποίηση παραγγελίας αξιοποιεί αυτόν τον εννοιολογικό διαχωρισμό: το Order Service δημοσιεύει το γεγονός στο RabbitMQ και το Notification Service αναλαμβάνει χωριστά την αποστολή email. Η συγκεκριμένη διαδρομή περιγράφεται από τις δηλώσεις exchange, ουράς και listener του έργου.

Η υιοθέτηση μηνυμάτων δεν σημαίνει ότι κάθε λειτουργία πρέπει να γίνεται ασύγχρονα. Για τη δημιουργία μιας παραγγελίας, η επιβεβαίωση αποθέματος είναι επιχειρησιακή προϋπόθεση και χρειάζεται άμεση απάντηση. Αντίθετα, η αποστολή email μπορεί να εκτελείται μετά την

Ανάπτυξη ενός Συστήματος Διαχείρισης Παραγγελιών (Order Management System)
Βασισμένου σε Microservices με Spring Boot και Spring Cloud

καταχώριση της παραγγελίας, χωρίς να καθυστερεί τον πελάτη ή να ακυρώνει την παραγγελία σε περίπτωση προσωρινού προβλήματος αποστολής.

Στην υποδομή RabbitMQ, η δημοσίευση ενός μηνύματος και η επιλογή της ουράς παραλαβής συνδέονται μέσω exchange και binding. Το routing key συμμετέχει στη δρομολόγηση σύμφωνα με τον τύπο του exchange και τις δηλωμένες συνδέσεις [26]. Στην παρούσα εφαρμογή, αυτή η διαδρομή συνδέει το γεγονός παραγγελίας με τον καταναλωτή ειδοποιήσεων. Το Spring AMQP παρέχει τα στοιχεία διαμόρφωσης και παραλαβής μηνυμάτων που χρησιμοποιούν οι εφαρμογές Spring [27]. Η συγκεκριμένη λειτουργία προκύπτει από τις δηλώσεις του έργου και δεν αποδίδει στην εφαρμογή πρόσθετες εγγυήσεις παράδοσης που δεν έχουν αξιολογηθεί.

1.4.4 Ασφάλεια, OAuth 2.0 και JWT

Σε ένα κατανεμημένο σύστημα, η ταυτοποίηση του χρήστη και η άδεια πρόσβασης σε συγκεκριμένο πόρο πρέπει να διακρίνονται. Η ανάλυση OAuth 2.0 των Fett, Küsters και Schmitz εξετάζει τους ρόλους του πρωτοκόλλου και τη σχέση εξουσιοδότησης και αυθεντικοποίησης [28]. Στην εφαρμογή, το access token προέρχεται από τον Keycloak και οι υπηρεσίες αξιοποιούν την πληροφορία ταυτότητας και ρόλων που περιέχει. Η αναφορά στην ερευνητική εργασία θεμελιώνει τον εννοιολογικό διαχωρισμό και δεν συνιστά τυπική επαλήθευση της ασφάλειας του συγκεκριμένου συστήματος.

Πέρα από την έκδοση token, απαιτούνται κανόνες για τις επιτρεπόμενες ενέργειες. Στο μοντέλο ελέγχου πρόσβασης βάσει ρόλων, οι άδειες συνδέονται με ρόλους και οι χρήστες αποκτούν πρόσβαση μέσω της ένταξής τους σε αυτούς [29]. Η εφαρμογή διακρίνει CUSTOMER και ADMIN και εφαρμόζει ελέγχους στο backend. Επομένως, η εμφάνιση ή απόκρυψη ενός κουμπιού στη διεπαφή λειτουργεί συμπληρωματικά προς την εξουσιοδότηση των API.

Στη διαμόρφωση του έργου, ο Keycloak αναλαμβάνει τη σύνδεση του χρήστη και οι εφαρμογές Spring επαληθεύουν τα bearer tokens ως resource servers. Οι αντίστοιχες ρυθμίσεις issuer, realm και μετατροπής ρόλων τεκμηριώνουν τη συγκεκριμένη σύνδεση υποδομής και εφαρμογής. Οι όροι OAuth 2.0, OpenID Connect και JWT περιγράφουν διαφορετικά στοιχεία της ροής και δεν χρησιμοποιούνται ως συνώνυμα. Η διαχείριση realm και ρόλων και η επαλήθευση JWT τεκμηριώνονται χωριστά από τον Keycloak και το Spring Security [30, 31].

Στη διαχείριση ταυτότητας του Keycloak, το realm οργανώνει χρήστες και ρόλους, ενώ η αντιστοίχιση ρόλου συνδέει τον χρήστη με τα δικαιώματα που εξετάζει η εφαρμογή [30]. Η διάκριση διευκολύνει την ακριβή οριοθέτηση του έργου: υλοποιούνται οι δύο επιχειρησιακοί ρόλοι CUSTOMER και ADMIN και οι αντίστοιχοι κανόνες πρόσβασης. Η παρουσία αυτών των ρόλων δεν σημαίνει ότι χρησιμοποιούνται όλες οι δυνατότητες διαχείρισης δικαιωμάτων του Keycloak. Η εφαρμογή αξιοποιεί τις πληροφορίες ταυτότητας που περιέχονται στο token για τις συγκεκριμένες λειτουργίες των υπηρεσιών της.

Ως γενικές αρχές προστασίας, οι Saltzer και Schroeder προτείνουν την ελάχιστη αναγκαία εξουσιοδότηση και τον έλεγχο κάθε πρόσβασης σε προστατευμένο αντικείμενο [32]. Η εφαρμογή συνδέει αυτές τις αρχές με τον έλεγχο των API και με την επιλογή των δεδομένων του συνδεδεμένου πελάτη. Η ύπαρξη ελέγχου στο backend είναι διακριτή από τον τρόπο παρουσίασης της διεπαφής και τεκμηριώνεται στις σχετικές κλάσεις ασφάλειας και υπηρεσιών.

1.4.5 Ανθεκτικότητα και κατανεμημένη ιχνηλάτηση

Η προστασία των απομακρυσμένων κλήσεων και η συσχέτιση των επιμέρους βημάτων ενός αιτήματος αντιμετωπίζουν διαφορετικές ανάγκες. Τα πρότυπα ανθεκτικότητας εξετάζουν τον περιορισμό της επίδρασης μερικών αποτυχιών [10], ενώ η τεκμηρίωση Resilience4j περιγράφει τις καταστάσεις και τη λειτουργία του circuit breaker [33]. Αντίστοιχα, το Micrometer Tracing και το Zipkin οργανώνουν την παραγωγή και συγκέντρωση στοιχείων ιχνηλάτησης [34, 35]. Στην εργασία, ο circuit breaker και η καταγραφή traces αποτελούν συγκεκριμένες υλοποιήσεις αυτών των αναγκών και δεν ταυτίζονται μεταξύ τους ως μηχανισμοί.

Η ιχνηλάτηση είναι ιδιαίτερα χρήσιμη όταν ένα αίτημα δημιουργίας παραγγελίας ξεκινά από το frontend, διέρχεται από το API Gateway, επικοινωνεί με υπηρεσίες καλαθιού και αποθέματος και καταλήγει στην υπηρεσία παραγγελιών. Η δυνατότητα συσχέτισης αυτών των επιμέρους

βημάτων επιτρέπει την εντόπιση καθυστερήσεων ή αποτυχιών χωρίς να απαιτείται χειροκίνητη συσχέτιση ανεξάρτητων αρχείων καταγραφής.

Η λειτουργία *microservices* απαιτεί πέρα από κώδικα και μια υποδομή παρακολούθησης. Η συσχέτιση *trace* και *span* επιτρέπει την ανάγνωση δραστηριότητας που διασχίζει περισσότερες εφαρμογές, ενώ το *Zipkin* συγκεντρώνει τα στοιχεία και τα παρουσιάζει στη διεπαφή του [34, 35]. Στην παρούσα εργασία, το διαθέσιμο στιγμιότυπο αφορά την ανάκτηση παραγγελιών μέσω *Gateway* και *Order Service*. Η εικόνα λειτουργεί ως τεκμήριο της συγκεκριμένης διαδρομής και δεν παρουσιάζεται ως επαλήθευση ολόκληρου του *checkout* ή ως πείραμα απόδοσης.

1.5 Συγκριτική αποτίμηση της έρευνας για τις μικροϋπηρεσίες

Πέρα από τον ορισμό των *microservices*, η βιβλιογραφική επισκόπηση χρειάζεται να εξετάζει πώς παράγονται τα συμπεράσματα για την αρχιτεκτονική. Μια συστηματική χαρτογράφηση ταξινομεί τις ερευνητικές εργασίες και αναδεικνύει τις περιοχές στις οποίες επικεντρώνεται η έρευνα. Μια ανασκόπηση βιομηχανικών πηγών αποτυπώνει τις εμπειρίες από την ανάπτυξη και τη λειτουργία εφαρμογών. Οι δύο κατηγορίες τεκμηρίων είναι συμπληρωματικές, αλλά δεν είναι μεθοδολογικά ταυτόσημες. Η διάκριση βοηθά να συνδεθεί η παρούσα υλοποίηση με προηγούμενες εργασίες χωρίς να αποδοθούν σε μια λειτουργική επίδειξη γενικεύσεις που προϋποθέτουν διαφορετικό ερευνητικό σχεδιασμό [1, 3].

Στη μελέτη των Di Francesco, Lago και Malavolta, το ερώτημα αφορά την κατάσταση της έρευνας για τον σχεδιασμό αρχιτεκτονικών *microservices* και τη δυνατότητα μεταφοράς των αποτελεσμάτων στη βιομηχανία. Με συστηματική επιλογή 103 πρωτογενών εργασιών και ταξινόμηση των ευρημάτων, οι συγγραφείς αναδεικνύουν, μεταξύ άλλων, την έμφαση στην απόδοση, τη συντηρησιμότητα και τη λειτουργική καταλληλότητα. Το χρονικό εύρος της αναζήτησης φθάνει στις αρχές Μαΐου 2017, παρότι το άρθρο δημοσιεύθηκε το 2019. Επομένως, η μελέτη αξιοποιείται εδώ ως τεκμήριο για τη διαμόρφωση του πεδίου και όχι ως πλήρης απογραφή της έρευνας έως το 2026 [1].

Αντίστοιχα, οι Soldani, Tamburri και van den Heuvel εξετάζουν τα τεχνικά και λειτουργικά οφέλη και τις δυσκολίες που αναφέρονται στην πράξη. Η δημοσιευμένη σε επιστημονικό περιοδικό ανασκόπηση τους αναλύει 51 επιλεγμένες βιομηχανικές πηγές της περιόδου 2014-2017 και οργανώνει τα ευρήματα γύρω από τον σχεδιασμό, την ανάπτυξη και τη λειτουργία. Το ενδιαφέρον της για την παρούσα εργασία βρίσκεται στη συνύπαρξη της αυτονομίας των υπηρεσιών με την πολυπλοκότητα της κατανεμημένης συνεργασίας. Επειδή το πρωτογενές υλικό είναι γκρίζα βιβλιογραφία, τα συμπεράσματα αποτυπώνουν συστηματικά οργανωμένη βιομηχανική εμπειρία και δεν αποτελούν ελεγχόμενο πείραμα σύγκρισης όλων των αρχιτεκτονικών επιλογών [3].

Για το *Order Management System*, η σύνθεση των παραπάνω εργασιών μεταφράζεται σε συγκεκριμένα σχεδιαστικά ερωτήματα: πού ανήκει ο κανόνας διαθεσιμότητας, ποια υπηρεσία καταχωρίζει την παραγγελία και ποια συνεργασία απαιτεί άμεση απάντηση. Το *Inventory Service* διατηρεί την ευθύνη του αποθέματος, το *Order Service* την τελική καταγραφή και το *Notification Service* την ειδοποίηση. Με αυτόν τον τρόπο, η βιβλιογραφία χρησιμοποιείται για να αιτιολογήσει τα επιχειρησιακά όρια και την επιλογή σύγχρονης ή ασύγχρονης επικοινωνίας, όχι για να θεωρηθεί αυτονόητη η υπεροχή μιας εφαρμογής επειδή διαθέτει περισσότερες υπηρεσίες.

Για την ερμηνεία της εφαρμογής χρειάζεται διάκριση μεταξύ αρχιτεκτονικής έννοιας και τεχνολογικής επιλογής. Τα άρθρα και τα βιβλία τεκμηριώνουν τις ευθύνες των υπηρεσιών, τα συμβόλαια συνεργασίας και τις αντισταθμίσεις του σχεδιασμού [7, 8]. Η επίσημη τεκμηρίωση συμπληρώνει το πλαίσιο με την περιγραφή των εργαλείων *Spring* και των δυνατοτήτων ενσωμάτωσής τους [21, 22]. Τα συγκεκριμένα ονόματα εφαρμογών, οι θύρες και οι ρυθμίσεις προκύπτουν από το αποθετήριο. Έτσι, η βιβλιογραφία αιτιολογεί τις επιλογές και ο κώδικας προσδιορίζει ακριβώς όσα υλοποιήθηκαν.

1.6 Συστήματα συστάσεων και ομοιότητα προϊόντων

Με βάση τις πληροφορίες που αξιοποιούνται, οι Adomavicius και Tuzhilin οργανώνουν τα συστήματα συστάσεων σε προσεγγίσεις περιεχομένου, συνεργατικού φιλτράρισματος και υβριδικού συνδυασμού [36]. Η ταξινόμηση επιτρέπει να τοποθετηθεί με ακρίβεια ο μηχανισμός του έργου. Τα σχετικά προϊόντα προκύπτουν από χαρακτηριστικά των ίδιων των αντικειμένων και από σταθερά βάρη συνάφειας. Δεν αξιοποιούνται αξιολογήσεις χρηστών για συνεργατικό φιλτράρισμα ούτε εκπαιδεύεται μοντέλο προσωπικών προτιμήσεων.

Στην ανάλυση των Jannach et al., η σύσταση μπορεί να εξεταστεί σε σχέση με το είδος της πληροφορίας που είναι διαθέσιμη και τον βαθμό εξατομίκευσης [37]. Για την παρούσα εφαρμογή, σημείο αναφοράς είναι το προϊόν που προβάλλεται και όχι το προσωπικό ιστορικό ενός πελάτη. Η ίδια περιγραφή προϊόντος οδηγεί επομένως σε κατάταξη με τα ίδια κριτήρια, ανεξάρτητα από την ταυτότητα του χρήστη.

Στο ηλεκτρονικό εμπόριο, οι Schafer, Konstan και Riedl εξετάζουν διαφορετικούς τρόπους με τους οποίους η γνώση προϊόντων και χρηστών τροφοδοτεί προτάσεις κατά την περιήγηση [38]. Η εφαρμογή αξιοποιεί μόνο την πλευρά της γνώσης προϊόντων: κατηγορία, χαρακτηριστικά, όνομα και τιμή. Η σύνδεση με τη μελέτη αφορά τον ρόλο της υποβοήθησης ανακάλυψης συναφών αντικειμένων και δεν αποδίδει στην υλοποίηση αύξηση πωλήσεων ή μετρημένη βελτίωση εμπειρίας χρήστη.

1.6.1 Αναπαράσταση περιεχομένου και κριτήρια συνάφειας

Στο πεδίο των συστημάτων συστάσεων, η αναπαράσταση των αντικειμένων καθορίζει ποια γνωρίσματα μπορούν να συγκριθούν και με ποιον τρόπο παράγεται μια κατάταξη. Το Recommender Systems Handbook διακρίνει ένα ευρύ σύνολο προσεγγίσεων, όπως συνεργατικό φιλτράρισμα, μεθόδους περιεχομένου, αξιοποίηση έμμεσης ανατροφοδότησης και μεθόδους που λαμβάνουν υπόψη το πλαίσιο χρήσης. Η παρουσίαση της τρίτης έκδοσης διαχωρίζει επίσης τις τεχνικές σύστασης από ζητήματα αξιολόγησης και αλληλεπίδρασης με τον χρήστη. Το εύρος αυτό επιτρέπει να τοποθετηθεί ο μηχανισμός της εφαρμογής σε συγκεκριμένη κατηγορία, αντί να χρησιμοποιηθεί αδιακρίτως ο όρος τεχνητή νοημοσύνη [39].

Στο κεφάλαιο των Lops, de Gemmis και Semeraro, οι μέθοδοι περιεχομένου περιγράφονται μέσα από την αναπαράσταση των αντικειμένων, τη διαμόρφωση προφίλ ενδιαφερόντων και τη σύγκριση των αντίστοιχων γνωρισμάτων. Η διάκριση από το συνεργατικό φιλτράρισμα αφορά κυρίως το είδος της πληροφορίας που αξιοποιείται: χαρακτηριστικά αντικειμένων και προτιμήσεις στο πρώτο παράδειγμα, σχέσεις αξιολογήσεων μεταξύ χρηστών στο δεύτερο. Για την παρούσα εργασία, το συναφές θεωρητικό στοιχείο είναι η χρήση χαρακτηριστικών περιεχομένου ως βάσης σύγκρισης. Δεν υιοθετείται το σύνολο του μοντέλου εξατομίκευσης που παρουσιάζει το κεφάλαιο [40].

Στην υλοποίηση του Product Service, σημείο αναφοράς είναι το προϊόν που εξετάζει ο χρήστης και όχι ένα εκπαιδευμένο προφίλ του χρήστη. Η κατηγορία, τα ζεύγη ονόματος και τιμής χαρακτηριστικού, οι λέξεις της ονομασίας και η σχετική απόσταση τιμής σχηματίζουν τις εισόδους της σύγκρισης. Επομένως, ο ακριβέστερος χαρακτηρισμός είναι κατάταξη σχετικών προϊόντων βάσει περιεχομένου με προκαθορισμένους κανόνες. Η ίδια πληροφορία προϊόντος οδηγεί στους ίδιους κανόνες υπολογισμού, χωρίς να απαιτείται ιστορικό αγορών, πίνακας αξιολογήσεων ή διαδικασία εκπαίδευσης.

Με τη συγκεκριμένη αναπαράσταση, το πρότυπο κατηγορίας αποκτά σημασία και πέρα από την επικύρωση της φόρμας. Επιτρέπει την καταχώριση ελεγχόμενων ονομάτων χαρακτηριστικών, τα οποία στη συνέχεια μπορούν να συμμετέχουν στη σύγκριση προϊόντων. Η εφαρμογή συγκρίνει κανονικοποιημένα ζεύγη ονόματος και τιμής, όχι μόνο την παρουσία ενός γενικού όρου στην περιγραφή. Έτσι, η ανάλυση του αλγορίθμου συνδέεται με τον τρόπο οργάνωσης του καταλόγου. Πρόκειται για σύνδεση δύο υλοποιημένων λειτουργιών του Product Service και όχι για ισχυρισμό ότι έχει μετρηθεί η εμπορική αποτελεσματικότητα των προτάσεων.

Στην προσέγγιση των Pazzani και Billsus, η αναπαράσταση του περιεχομένου καθορίζει ποια στοιχεία μπορούν να αξιοποιηθούν κατά την παραγωγή σύστασης [41]. Στην εργασία, τα χαρακτηριστικά οργανώνονται μέσω προτύπου κατηγορίας, οπότε η σύγκριση βασίζεται σε αναγνωρίσιμα πεδία και τιμές. Η σύνδεση αυτή αφορά την περιγραφή των αντικειμένων· ο

υλοποιημένος μηχανισμός παραμένει σύγκριση προϊόντος με προϊόν και δεν κατασκευάζει μαθημένο προφίλ χρήστη.

1.6.2 Ερμηνευσιμότητα της κατάταξης

Στην ανασκόπηση των Zhang και Chen, η εξηγήσιμη σύσταση εξετάζεται ως πρόβλημα που συνδυάζει την παραγωγή προτάσεων με την παρουσίαση των λόγων για τους οποίους αυτές παράγονται. Η μελέτη οργανώνει τις σχετικές τεχνικές και διακρίνει την αξιολόγηση της ποιότητας των συστάσεων από την αξιολόγηση των εξηγήσεων. Αυτό το τελευταίο σημείο είναι ουσιώδες: η δυνατότητα εμφάνισης ενός λόγου αντιστοίχισης δεν αποδεικνύει από μόνη της ότι η σύσταση είναι χρήσιμη ή ότι ενισχύει την εμπιστοσύνη κάθε χρήστη. Τέτοια συμπεράσματα απαιτούν κατάλληλα δεδομένα ή μελέτη χρηστών [42].

Στο συγκεκριμένο σύστημα, οι λεκτικές αιτιολογήσεις παράγονται από τους ίδιους ελέγχους που προσθέτουν μονάδες στη βαθμολογία. Όταν συμπίπτει η κατηγορία, καταγράφεται ο αντίστοιχος λόγος: όταν υπάρχουν κοινά χαρακτηριστικά, εμφανίζεται το πλήθος τους: όταν ικανοποιείται ένα όριο εγγύτητας τιμής, προστίθεται η σχετική ένδειξη. Δεν παράγεται μια γενική περιγραφή ανεξάρτητα από τον υπολογισμό. Η React διεπαφή μεταφέρει αυτές τις πληροφορίες στις κάρτες σχετικών προϊόντων, ώστε ο αναγνώστης της πρότασης να μπορεί να συνδέσει την εμφάνισή της με συγκεκριμένα δεδομένα του καταλόγου.

Κατά συνέπεια, ο αριθμητικός δείκτης πρέπει να ερμηνεύεται με βάση τον ορισμό του. Οι εκατό μονάδες αποτελούν το θεωρητικό άθροισμα των ανώτατων συνεισφορών των κριτηρίων. Δεν εκφράζουν πιθανότητα αγοράς, ποσοστό ικανοποίησης ή ακρίβεια πρόβλεψης. Ακόμη και αν η διεπαφή παρουσιάζει τη συνάφεια με μορφή ποσοστού, η σημασία της παραμένει εκείνη του προσθετικού κανόνα του κώδικα. Η ερμηνεία αυτή διατηρεί σαφή τη διαφορά ανάμεσα σε μια διαφανή κατάταξη και σε ένα στατιστικά αξιολογημένο μοντέλο πρόβλεψης.

Για τις επεξηγήσεις συστάσεων, οι Tintarev και Masthoff διακρίνουν διαφορετικούς στόχους, όπως τη διαφάνεια του μηχανισμού και την υποστήριξη της απόφασης του χρήστη [43]. Στη διεπαφή του έργου, οι λόγοι συνάφειας εκφράζουν επιμέρους στοιχεία που συμμετείχαν στη βαθμολόγηση. Η εμφάνισή τους εξηγεί την προέλευση της πρότασης και δεν αποτελεί μέτρηση της πειστικότητας ή της εμπιστοσύνης που προκαλεί.

Στο ίδιο κεφάλαιο, οι συγγραφείς συνδέουν τον σχεδιασμό μιας επεξήγησης με τον αλγόριθμο και τον τρόπο παρουσίασης των προτάσεων [43]. Για αυτόν τον λόγο, οι ενδείξεις κοινής κατηγορίας, χαρακτηριστικών ή εύρους τιμής πρέπει να διαβάζονται μαζί με τη διαδικασία υπολογισμού του score. Το εμφανιζόμενο ποσοστό αντιστοιχεί στη βαθμολογία του συγκεκριμένου μηχανισμού και δεν ερμηνεύεται ως πιθανότητα αγοράς ή ως αξιολογημένη ακρίβεια σύστασης.

2. Ανάλυση Απαιτήσεων

2.1 Επισκόπηση συστήματος

Το υπό ανάπτυξη σύστημα αποτελεί εφαρμογή ηλεκτρονικής διαχείρισης παραγγελιών. Επιτρέπει σε ταυτοποιημένους χρήστες να εξερευνούν τον κατάλογο προϊόντων, να διατηρούν καλάθι αγορών, να δεσμεύουν διαθέσιμο απόθεμα κατά τη μεταβολή του καλαθιού, να υποβάλλουν παραγγελία και να παρακολουθούν την εξέλιξή της. Παράλληλα, διαθέτει διαχειριστικό περιβάλλον για την οργάνωση του καταλόγου και την ενημέρωση της ροής των παραγγελιών.

Η ανάλυση που ακολουθεί βασίζεται στις λειτουργίες που υλοποιούνται στον κώδικα της εφαρμογής. Οι απαιτήσεις διατυπώνονται με τρόπο που να μπορεί να ελεγχθεί αργότερα με συγκεκριμένα σενάρια δοκιμών.

2.2 Ρόλοι χρηστών

2.2.1 Πελάτης

- Ταυτοποιείται μέσω Keycloak πριν από την πρόσβαση στη διεπαφή της εφαρμογής.
- Προβάλλει τον κατάλογο και τη λεπτομερή σελίδα προϊόντος, μαζί με εικόνες, χαρακτηριστικά και σχετικά προϊόντα.
- Προσθέτει προϊόντα στο καλάθι, μεταβάλλει ποσότητες και αφαιρεί είδη.
- Υποβάλλει παραγγελία, προβάλλει το προσωπικό ιστορικό παραγγελιών και παρακολουθεί την κατάστασή τους.
- Ακυρώνει δική του παραγγελία πριν από τη συσκευασία ή, όταν έχει παραδοθεί, εντός του προβλεπόμενου χρονικού παραθύρου ακύρωσης.

2.2.2 Διαχειριστής

- Διαθέτει τον ρόλο ADMIN και αποκτά πρόσβαση στο διαχειριστικό περιβάλλον.
- Διαχειρίζεται προϊόντα, κατηγορίες, χαρακτηριστικά κατηγοριών και εικόνες προϊόντων.
- Ελέγχει τις παραγγελίες και ενημερώνει την κατάστασή τους ακολουθώντας τους κανόνες της ροής εργασίας.
- Έχει πρόσβαση σε λειτουργίες που δεν είναι διαθέσιμες στον απλό πελάτη.

Η διάκριση ρόλων αποτυπώνει τον διαχωρισμό μεταξύ της εξυπηρέτησης παραγγελίας και της διοικητικής ευθύνης για τον κατάλογο και τη ροή εκτέλεσης. Δεν πρόκειται απλώς για διαφορετική εμφάνιση της διεπαφής. Οι αρμοδιότητες μεταφράζονται σε ελέγχους εξουσιοδότησης στο Gateway και στις υπηρεσίες, ώστε ένα αίτημα να αξιολογείται με βάση το επαληθευμένο token και όχι με βάση επιλογές που εμφανίζονται στον client.

Πίνακας 2.1: Αντιστοίχιση ρόλων και βασικών αρμοδιοτήτων.

Ρόλος	Βασικές αρμοδιότητες	Πεδίο πρόσβασης
CUSTOMER	Περιήγηση, καλάθι, checkout, προσωπικό ιστορικό και επιτρεπτή ακύρωση.	Δικά του δεδομένα και δημόσιος κατάλογος.
ADMIN	Διαχείριση catalog, κατηγοριών, αποθέματος και ροής παραγγελιών.	Διοικητικά endpoints με έλεγχο ρόλου.

2.3 Λειτουργικές απαιτήσεις

2.3.1 Διαχείριση καταλόγου προϊόντων

- FR-01: Το σύστημα πρέπει να εμφανίζει τον διαθέσιμο κατάλογο προϊόντων στον χρήστη.
- FR-02: Το σύστημα πρέπει να εμφανίζει λεπτομερή στοιχεία προϊόντος, όπως κωδικό SKU, όνομα, περιγραφή, τιμή, κατηγορία, χαρακτηριστικά και εικόνες.
- FR-03: Το σύστημα πρέπει να προτείνει σχετικά προϊόντα με βάση κατηγορία, κοινά χαρακτηριστικά, ομοιότητα ονόματος και εγγύτητα τιμής.
- FR-04: Ο διαχειριστής πρέπει να μπορεί να δημιουργεί, ενημερώνει και διαγράφει προϊόντα.
- FR-05: Ο διαχειριστής πρέπει να μπορεί να δημιουργεί και να διαχειρίζεται κατηγορίες και τους επιτρεπόμενους τύπους χαρακτηριστικών τους.

2.3.2 Καλάθι και δέσμευση αποθέματος

- FR-06: Κάθε ταυτοποιημένος χρήστης πρέπει να διαθέτει ανεξάρτητο καλάθι αγορών.
- FR-07: Ο χρήστης πρέπει να μπορεί να προσθέτει είδη, να μεταβάλλει την ποσότητα και να αφαιρεί είδη από το καλάθι.
- FR-08: Μετά από κάθε μεταβολή του καλαθιού, το σύστημα πρέπει να συγχρονίζει προσωρινή δέσμευση αποθέματος για τα τρέχοντα είδη του καλαθιού.
- FR-09: Το σύστημα πρέπει να απορρίπτει μεταβολή καλαθιού όταν η διαθέσιμη ποσότητα δεν επαρκεί.
- FR-10: Η προσωρινή δέσμευση αποθέματος πρέπει να λήγει μετά από περιορισμένο χρονικό διάστημα, ώστε ποσότητες που δεν οδήγησαν σε παραγγελία να γίνονται ξανά διαθέσιμες.

2.3.3 Υποβολή και παρακολούθηση παραγγελίας

- FR-11: Η υποβολή παραγγελίας πρέπει να αποτυγχάνει όταν το καλάθι είναι κενό.
- FR-12: Πριν από τη δημιουργία παραγγελίας, το σύστημα πρέπει να επιβεβαιώνει τη δέσμευση αποθέματος και να αφαιρεί οριστικά τις αντίστοιχες ποσότητες από το συνολικό απόθεμα.
- FR-13: Το σύστημα πρέπει να δημιουργεί μοναδικό αριθμό παραγγελίας, να αποθηκεύει τα είδη, τις ποσότητες, τις τιμές και την αρχική κατάσταση PENDING.
- FR-14: Η υποβολή παραγγελίας πρέπει να υποστηρίζει κλειδί idempotency, ώστε επανάληψη του ίδιου αιτήματος να μην δημιουργεί δεύτερη παραγγελία.
- FR-15: Μετά την επιτυχή δημιουργία παραγγελίας, το σύστημα πρέπει να εκκαθαρίζει το καλάθι του πελάτη.
- FR-16: Ο πελάτης πρέπει να μπορεί να προβάλλει μόνο τις δικές του παραγγελίες και το ιστορικό κατάστασής τους.
- FR-17: Το σύστημα πρέπει να καταγράφει κάθε αλλαγή κατάστασης με προηγούμενη κατάσταση, νέα κατάσταση, χρόνο, υπεύθυνο και σημείωση.
- FR-18: Το σύστημα πρέπει να επιτρέπει ακύρωση πριν από τη συσκευασία και, για παραδομένες παραγγελίες, μόνο εντός δεκατεσσάρων ημερών από την παράδοση.

2.3.4 Ειδοποιήσεις και διαχείριση ροής

- FR-19: Μετά την επιτυχή υποβολή παραγγελίας, το σύστημα πρέπει να δημοσιεύει συμβάν τοποθέτησης παραγγελίας.
- FR-20: Η υπηρεσία ειδοποιήσεων πρέπει να καταναλώνει το συμβάν και να αποστέλλει ενημερωτικό μήνυμα ηλεκτρονικού ταχυδρομείου όταν η λειτουργία email είναι ενεργοποιημένη.
- FR-21: Ο διαχειριστής πρέπει να μπορεί να ενημερώνει την κατάσταση παραγγελίας μόνο σε επιτρεπτές επόμενες καταστάσεις.

2.4 Μη λειτουργικές απαιτήσεις

- NFR-01 - Ασφάλεια: Η πρόσβαση στις εφαρμογές και στα API πρέπει να βασίζεται σε ταυτοποίηση μέσω Keycloak και επικύρωση JWT από τις υπηρεσίες πόρων.
- NFR-02 - Εξουσιοδότηση: Οι διαχειριστικές λειτουργίες πρέπει να είναι προσβάσιμες μόνο σε χρήστες με ρόλο ADMIN. Ο πελάτης δεν πρέπει να έχει πρόσβαση στα δεδομένα άλλου πελάτη.
- NFR-03 - Συνέπεια αποθέματος: Οι προσωρινές και οριστικές μεταβολές αποθέματος πρέπει να αποτρέπουν την υπερπώληση διαθέσιμων ειδών.
- NFR-04 - Ανθεκτικότητα: Αποτυχία κρίσιμης απομακρυσμένης κλήσης, όπως της υπηρεσίας αποθέματος, πρέπει να επιστρέφει ελεγχόμενο μήνυμα και να μην δημιουργεί ελλιπή παραγγελία.
- NFR-05 - Παρατηρησιμότητα: Τα αιτήματα μεταξύ υπηρεσιών πρέπει να φέρουν στοιχεία ιχνηλάτησης ώστε η πορεία μιας παραγγελίας να είναι ορατή στο Zipkin.
- NFR-06 - Επεκτασιμότητα: Οι κύριες επιχειρησιακές λειτουργίες πρέπει να υλοποιούνται ως ανεξάρτητες υπηρεσίες ώστε να μπορούν να κλιμακωθούν ή να αναπτυχθούν χωριστά.
- NFR-07 - Συντηρησιμότητα: Η ευθύνη κάθε υπηρεσίας, τα API και οι μεταβολές βάσης δεδομένων πρέπει να τεκμηριώνονται σαφώς.
- NFR-08 - Χρηστικότητα: Το frontend πρέπει να εμφανίζει κατανοητά μηνύματα επιτυχίας ή σφάλματος κατά τη διαχείριση καλαθιού και την υποβολή παραγγελίας.

2.5 Περιπτώσεις χρήσης

2.5.1 UC-01: Διαχείριση καλαθιού

- Πρωτεύων δρων: Πελάτης.
- Προϋπόθεση: Ο πελάτης έχει ταυτοποιηθεί και το προϊόν υπάρχει στον κατάλογο.
- Κύρια ροή: Ο πελάτης προσθέτει προϊόν ή αλλάζει ποσότητα. Η υπηρεσία καλαθιού ενημερώνει το καλάθι και ζητά από την υπηρεσία αποθέματος τη δέσμευση των απαιτούμενων ποσοτήτων.
- Εναλλακτική ροή: Αν το απόθεμα δεν επαρκεί, η μεταβολή απορρίπτεται και ο χρήστης ενημερώνεται για την έλλειψη διαθεσιμότητας.
- Αποτέλεσμα: Το καλάθι περιέχει μόνο ποσότητες που μπορούν να δεσμευθούν προσωρινά.

2.5.2 UC-02: Υποβολή παραγγελίας

- Πρωτεύων δρων: Πελάτης.
- Προϋπόθεση: Το καλάθι δεν είναι κενό και διαθέτει ενεργή δέσμευση αποθέματος.
- Κύρια ροή: Ο πελάτης ελέγχει τη σύνοψη, επιλέγει υποβολή και αποστέλλεται μοναδικό κλειδί idempotency. Η υπηρεσία παραγγελιών επιβεβαιώνει τη δέσμευση αποθέματος, δημιουργεί την παραγγελία, καταγράφει την αρχική κατάσταση, εκκαθαρίζει το καλάθι και δημοσιεύει συμβάν ειδοποίησης.
- Εναλλακτική ροή: Αν το καλάθι είναι κενό ή η υπηρεσία αποθέματος δεν είναι διαθέσιμη, η παραγγελία δεν ολοκληρώνεται και επιστρέφεται κατανοητό μήνυμα σφάλματος.
- Αποτέλεσμα: Δημιουργείται μία μόνο παραγγελία, μειώνεται οριστικά το απόθεμα και ενεργοποιείται η διαδικασία ειδοποίησης.

2.5.3 UC-03: Παρακολούθηση και ακύρωση παραγγελίας

- Πρωτεύων δρων: Πελάτης ή διαχειριστής.
- Κύρια ροή: Ο πελάτης προβάλλει το προσωπικό ιστορικό παραγγελιών και τις επιτρεπτές επόμενες ενέργειες. Ο διαχειριστής ενημερώνει την κατάσταση σύμφωνα με τη ροή εργασίας.

- Ακύρωση: Ο πελάτης ακυρώνει μόνο δική του παραγγελία, πριν από τη συσκευασία ή εντός δεκατεσσάρων ημερών από την παράδοση. Ο διαχειριστής μπορεί να ενεργεί στο πλαίσιο των δικαιωμάτων του.
- Αποτέλεσμα: Η αλλαγή κατάστασης καταγράφεται στο ιστορικό της παραγγελίας.

2.5.4 UC-04: Διαχείριση καταλόγου

- Πρωτεύων δρων: Διαχειριστής.
- Κύρια ροή: Ο διαχειριστής δημιουργεί ή ενημερώνει κατηγορία και ορίζει τα χαρακτηριστικά της. Στη συνέχεια δημιουργεί ή ενημερώνει προϊόν με έγκυρη κατηγορία, χαρακτηριστικά και εικόνες.
- Εναλλακτική ροή: Το σύστημα απορρίπτει προϊόν όταν λείπει υποχρεωτικό χαρακτηριστικό ή όταν υποβάλλεται μη επιτρεπτή τιμή χαρακτηριστικού.
- Αποτέλεσμα: Ο κατάλογος διατηρεί συνεπή δεδομένα προϊόντων και χαρακτηριστικών.

2.6 Περιορισμοί του τρέχοντος πεδίου

- Δεν έχει ενσωματωθεί πραγματικός πάροχος ηλεκτρονικών πληρωμών.
- Δεν περιλαμβάνεται διασύνδεση με εξωτερικό σύστημα αποστολών ή μεταφορών.
- Η υπηρεσία ειδοποίησης υποστηρίζει ρυθμιζόμενη αποστολή email, αλλά η τελική παράδοση εξαρτάται από τη ρύθμιση του παρόχου SMTP.
- Οι μετρήσεις απόδοσης και το τελικό επίπεδο κλιμάκωσης θα αξιολογηθούν με ελεγχόμενα σενάρια και όχι με παραγωγικό φορτίο πραγματικού καταστήματος.

2.7 Κριτήρια αποδοχής και επιχειρησιακή σημασία

Η διατύπωση μιας απαίτησης χρειάζεται να συνδέεται με αποτέλεσμα που μπορεί να αναγνωρίσει ο χρήστης ή να ελέγξει ο αναγνώστης της τεκμηρίωσης. Η γενική απαίτηση «διαχείριση παραγγελιών» δεν αρκεί για να διακρίνει την απλή εμφάνιση μιας φόρμας από την ολοκλήρωση της υποβολής. Στην παρούσα εφαρμογή, η ολοκλήρωση αναγνωρίζεται από την απόκριση της υπηρεσίας, τον αριθμό παραγγελίας και την αρχική κατάσταση. Με ανάλογο τρόπο, η αναπλήρωση αποθέματος αναγνωρίζεται από την αριθμητική μεταβολή της ποσότητας και η ακύρωση από τη μεταβολή της ίδιας παραγγελίας.

Τα κριτήρια του Πίνακα 2.2 διατυπώνουν το αναμενόμενο αποτέλεσμα κάθε διαδρομής. Δεν αποτελούν πρόσθετα αποτελέσματα πειραμάτων. Χρησιμοποιούνται ως γέφυρα ανάμεσα στις απαιτήσεις του παρόντος κεφαλαίου και στα τεκμηριωμένα σενάρια του Κεφαλαίου 6. Η αντιστοίχιση αυτή αποτρέπει τη σύγχυση ανάμεσα σε μια επιθυμητή λειτουργία και στην έκταση στην οποία έχει καταγραφεί η εκτέλεσή της.

Πίνακας 2.2: Κριτήρια αποδοχής των βασικών επιχειρησιακών διαδρομών.

Διαδρομή	Αρχική συνθήκη	Αναμενόμενο αποτέλεσμα
Κατάλογος	Υπάρχουν καταχωρισμένα προϊόντα.	Ο πελάτης βλέπει ονομασία, περιγραφή, τιμή και ένδειξη διαθεσιμότητας για τα εμφανιζόμενα είδη.
Λεπτομέρεια προϊόντος	Έχει επιλεγεί προϊόν του καταλόγου.	Παρουσιάζονται τα χαρακτηριστικά του συγκεκριμένου είδους και οι επιστρεφόμενες προτάσεις με τις αιτιολογήσεις τους.
Καλάθι	Συνδεδεμένος χρήστης επιλέγει είδος και ποσότητα.	Η απόκριση καλαθιού περιέχει τη γραμμή με το αντίστοιχο SKU. Η διεπαφή παρουσιάζει ποσότητες και σύνολα.
Υποβολή	Το ανακτημένο καλάθι δεν είναι κενό.	Επιστρέφεται αριθμός παραγγελίας και αρχική κατάσταση PENDING. Η επιβεβαίωση αναφέρεται στα είδη της συγκεκριμένης υποβολής.

Διαδρομή	Αρχική συνθήκη	Αναμενόμενο αποτέλεσμα
Ακύρωση	Η επιλεγμένη παραγγελία επιτρέπει ακύρωση στον τρέχοντα χρήστη.	Μετά την επιβεβαίωση της ενέργειας, το ίδιο αναγνωριστικό παραγγελίας εμφανίζεται με κατάσταση CANCELLED.
Αναπλήρωση	Ο διαχειριστής έχει επιλέξει SKU και ποσότητα αύξησης.	Το νέο συνολικό απόθεμα ισούται με το προηγούμενο συν την καταχωρισμένη αύξηση.
Email	Έχει δημιουργηθεί παραγγελία και έχει διαμορφωθεί το δοκιμαστικό SMTP.	Το sandbox περιέχει μήνυμα που αναφέρεται στον ίδιο αριθμό παραγγελίας και στον αντίστοιχο παραλήπτη.

Σημαντική είναι επίσης η διάκριση ανάμεσα στη δυνατότητα πρόσβασης σε μια οθόνη και στη δυνατότητα εκτέλεσης συγκεκριμένης ενέργειας. Ο πελάτης χρησιμοποιεί το καλάθι και το προσωπικό του ιστορικό, ενώ ο διαχειριστής διαθέτει λειτουργίες καταλόγου, αναπλήρωσης και εξέλιξης παραγγελιών. Κάθε ενέργεια εξαρτάται από πρόσθετη κατάσταση του αντικειμένου: η ακύρωση αφορά επιλέξιμη παραγγελία και η υποβολή αφορά μη κενό καλάθι. Επομένως, το ζεύγος ρόλου και ενέργειας δεν περιγράφει μόνο του ολόκληρο τον επιχειρησιακό κανόνα.

Από την πλευρά του χρήστη, τα χρηματικά σύνολα έχουν διαφορετικό ρόλο από τις ποσότητες αποθέματος. Η ποσότητα στη γραμμή του καλαθιού εκφράζει πόσα τεμάχια επιθυμεί να αγοράσει. Η διαθέσιμη ποσότητα καταλόγου εκφράζει πόσα μπορούν να δεσμευθούν σύμφωνα με τον υπολογισμό της υπηρεσίας αποθέματος. Η τιμή μονάδας και το γινόμενο τιμής επί ποσότητας συνθέτουν το ποσό της γραμμής. Η τεκμηρίωση διατηρεί διακριτές αυτές τις έννοιες, ώστε η επιτυχής παρουσίαση ενός ποσού να μην ερμηνεύεται ως ανεξάρτητη επιβεβαίωση αποθέματος.

Η συνάφεια προϊόντων αποτελεί συμπληρωματική απαίτηση του καταλόγου. Το αποτέλεσμα πρέπει να αφορά άλλα ενεργά προϊόντα, να παρουσιάζει συγκεκριμένη βαθμολογία και να συνοδεύεται από αιτιολόγηση. Το περιεχόμενο της απαίτησης δεν προϋποθέτει προσωπικό ιστορικό αγορών ή εκπαίδευση μοντέλου. Η επιλογή είναι συμβατή με την υλοποίηση βάσει περιεχομένου: η κατάταξη προκύπτει από κατηγορία, χαρακτηριστικά, ονομασία και τιμή. Κατά συνέπεια, η αξιολόγηση της λειτουργίας χρειάζεται να ξετάζει τη συμφωνία αυτών των κριτηρίων με το εμφανιζόμενο αποτέλεσμα, όχι την ακρίβεια ενός ανύπαρκτου μοντέλου πρόβλεψης.

Για τις μη λειτουργικές απαιτήσεις διατηρείται αντίστοιχη ακρίβεια. Η δυνατότητα κατανεμημένης ιχνηλάτησης τεκμηριώνεται με συσχετισμένο ίχνος, ενώ η ύπαρξη μηχανισμού circuit breaker τεκμηριώνεται από τη διαμόρφωση και τη χρήση του στον κώδικα. Αριθμητική διαθεσιμότητα ή απόδοση υπό φορτίο θα απαιτούσε χωριστή πειραματική διαδικασία. Η απαίτηση περιγράφει την επιδιωκόμενη ιδιότητα, ο σχεδιασμός τον μηχανισμό υποστήριξής της και η αξιολόγηση το αποτέλεσμα που πράγματι παρατηρήθηκε.

3. Σχεδιασμός Συστήματος

3.1 Σχεδιαστικοί στόχοι και αρχές

Ο σχεδιασμός μεταφράζει τις απαιτήσεις του προηγούμενου κεφαλαίου σε σαφή αρχιτεκτονικά όρια. Η βασική επιλογή είναι η διάσπαση του Order Management System σε υπηρεσίες με συγκεκριμένη επιχειρησιακή ευθύνη: κατάλογος προϊόντων, καλάθι, απόθεμα, παραγγελία και ειδοποίηση. Έτσι, η λειτουργία της παραγγελίας δεν συγχέεται με τη διαχείριση προϊόντων ή με την αποστολή email και κάθε τμήμα μπορεί να εξελίσσεται με μικρότερη επίδραση στα υπόλοιπα.

Η διάσπαση αυτή δεν αντιμετωπίζεται ως αυτοσκοπός. Η αρχιτεκτονική *microservices* συνδέεται με ανεξάρτητη ανάπτυξη και κλιμάκωση, αλλά ταυτόχρονα απαιτεί ρητές αποφάσεις για επικοινωνία, συνέπεια δεδομένων και παρατηρησιμότητα [1, 3]. Για τον λόγο αυτό, ο σχεδιασμός συνδυάζει συγχρονισμένες κλήσεις όπου χρειάζεται άμεση απόφαση, όπως η επιβεβαίωση αποθέματος, με ασύγχρονα συμβάντα για την ειδοποίηση του πελάτη.

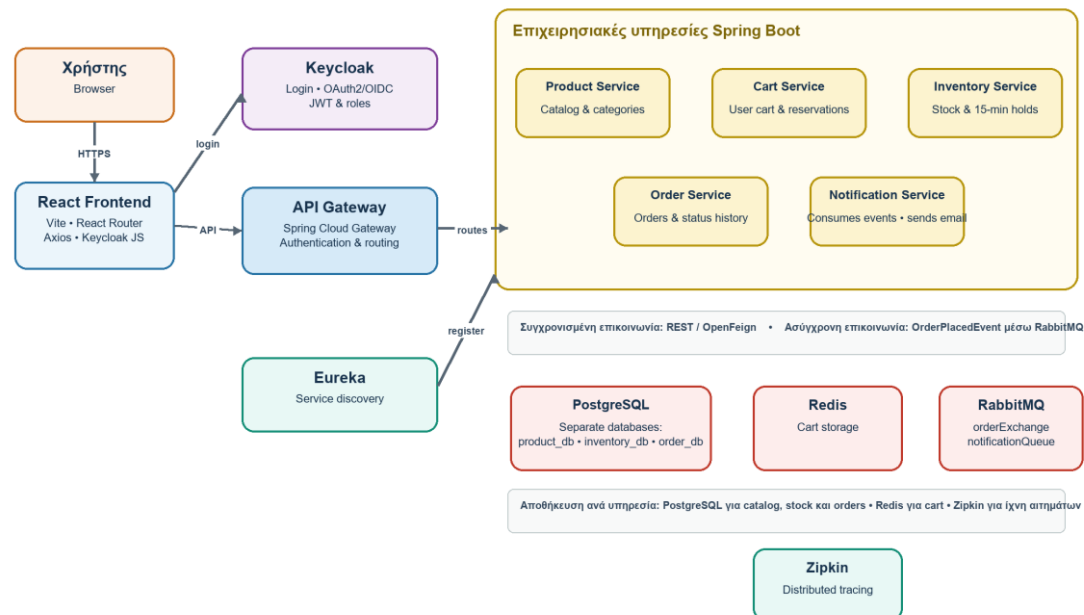
- Διαχωρισμός επιχειρησιακών ορίων: κάθε υπηρεσία διαθέτει σαφή ευθύνη και δεν εκθέτει εσωτερικά μοντέλα δεδομένων σε άλλες υπηρεσίες.
- Ενιαίο σημείο εισόδου: το frontend επικοινωνεί με το API Gateway αντί να γνωρίζει διευθύνσεις των επιμέρους υπηρεσιών.
- Ανεξάρτητη αποθήκευση: τα δεδομένα *catalog*, *inventory* και *order* αποθηκεύονται σε διαφορετικές βάσεις PostgreSQL, ενώ το καλάθι διατηρείται στο Redis.
- Ασφάλεια από άκρο σε άκρο: η ταυτοποίηση γίνεται μέσω Keycloak και το JWT μεταφέρεται στα προστατευμένα API.
- Ανθεκτικότητα και ιχνηλάτηση: η κρίσιμη κλήση προς *inventory* προστατεύεται με *circuit breaker* και τα αιτήματα μπορούν να παρακολουθηθούν μέσω Zipkin.

3.2 Συνολική αρχιτεκτονική

Το Διάγραμμα 3.1 παρουσιάζει τη λογική τοπολογία της εφαρμογής. Ο χρήστης αλληλεπιδρά με τη React εφαρμογή. Η εφαρμογή αναλαμβάνει το login μέσω Keycloak και αποστέλλει τα API αιτήματα προς το Spring Cloud Gateway. Το Gateway εφαρμόζει δρομολόγηση με βάση τα *prefixes* των διαδρομών και μεταφέρει το token στις υπηρεσίες. Η χρήση *gateway* και *service discovery* αποτρέπει τη σύνδεση του frontend με σταθερές διευθύνσεις υπηρεσιών και επιτρέπει τη δρομολόγηση μέσω λογικών ονομάτων υπηρεσίας.

Αρχιτεκτονική συστήματος διαχείρισης παραγγελιών

React frontend, Spring Cloud υποδομή και ανεξάρτητες υπηρεσίες



Διάγραμμα 3.1: Συνολική αρχιτεκτονική του συστήματος διαχείρισης παραγγελιών.

Οι Product, Cart, Inventory, Order και Notification Service είναι ανεξάρτητες Spring Boot εφαρμογές. Όλες εγγράφονται στον Eureka Discovery Server. Το Product, το Inventory και το Order Service διαθέτουν τη δική τους λογική αποθήκευσης PostgreSQL, ενώ το Cart Service χρησιμοποιεί Redis για την προσωρινή, ανά χρήστη κατάσταση καλαθιού. Η επικοινωνία του Order Service με Cart και Inventory είναι συγχρονισμένη, επειδή η δημιουργία παραγγελίας χρειάζεται άμεση απάντηση. Αντίθετα, η αποστολή ειδοποίησης αποσυνδέεται από την κρίσιμη διαδρομή με RabbitMQ, ώστε μια προσωρινή αδυναμία SMTP να μη διακόπτει τη δημιουργία παραγγελίας.

3.3 Ευθύνες και όρια των υπηρεσιών

Η επιλογή των υπηρεσιών ακολουθεί τη διάκριση των επιχειρησιακών ικανοτήτων του συστήματος. Παρακάτω συνοψίζονται οι ευθύνες όπως υλοποιούνται στον κώδικα:

- **Product Service:** διαχειρίζεται προϊόντα, κατηγορίες, χαρακτηριστικά και εικόνες. Ελέγχει τα υποχρεωτικά χαρακτηριστικά μιας κατηγορίας πριν αποθηκεύσει προϊόν και παράγει σχετικές προτάσεις με κριτήρια κατηγορίας, χαρακτηριστικών, ονόματος και τιμής.
- **Cart Service:** διατηρεί ανά χρήστη καλάθι στο Redis. Κάθε μεταβολή γραμμής καλαθιού συγχρονίζεται με το Inventory Service ώστε να αντιστοιχεί σε ενεργή δέσμευση αποθέματος.
- **Inventory Service:** διαχειρίζεται συνολική ποσότητα και ενεργές δεσμεύσεις. Η διαθέσιμη ποσότητα υπολογίζεται ως συνολικό απόθεμα μείον δεσμευμένες μονάδες. Οι δεσμεύσεις έχουν διάρκεια δεκαπέντε λεπτών και μπορούν να επιβεβαιωθούν ή να αποδεσμευθούν.
- **Order Service:** ενορχηστρώνει την υποβολή. Ελέγχει idempotency key, ανακτά καλάθι, επιβεβαιώνει απόθεμα, δημιουργεί παραγγελία με μοναδικό αριθμό, καταγράφει ιστορικό κατάστασης και δημοσιεύει OrderPlacedEvent.
- **Notification Service:** καταναλώνει το συμβάν παραγγελίας από RabbitMQ και αποστέλλει email επιβεβαίωσης, με συμπεριφορά που ελέγχεται από ρυθμίσεις SMTP και fail-on-error.
- **Υποδομή Spring Cloud:** το Eureka παρέχει ανακάλυψη υπηρεσιών και το API Gateway παρέχει την εξωτερική επιφάνεια δρομολόγησης. Η υποδομή αυτή αναλύεται λεπτομερέστερα σε επόμενο κεφάλαιο.

Η αποθήκευση δεν μοιράζεται ως κοινό σχήμα μεταξύ υπηρεσιών. Αυτή η επιλογή μειώνει τη σύζευξη στο επίπεδο της βάσης και επιτρέπει κάθε υπηρεσία να ελέγχει το δικό της μοντέλο. Σημαίνει όμως ότι μια επιχειρησιακή ενέργεια, όπως η παραγγελία, περνά από περισσότερα του ενός όρια και απαιτεί προσεκτικό χειρισμό σφαλμάτων και αντιστάθμισης.

Πίνακας 3.1: Υπηρεσίες, ευθύνες και κύρια αποθήκευση δεδομένων.

Υπηρεσία	Επιχειρησιακή ευθύνη	Κύρια αποθήκευση
Product	Κατάλογος, κατηγορίες, χαρακτηριστικά και εικόνες.	PostgreSQL (product_db)
Cart	Καλάθι ανά χρήστη και συσχέτιση με reservation.	Redis
Inventory	Ποσότητες, restock και δεσμεύσεις αποθέματος.	PostgreSQL (inventory_db)
Order	Παραγγελίες, γραμμές, idempotency και ιστορικό κατάστασης.	PostgreSQL (order_db)
Notification	Κατανάλωση OrderPlacedEvent και ειδοποίηση email.	RabbitMQ / ρυθμίσεις SMTP

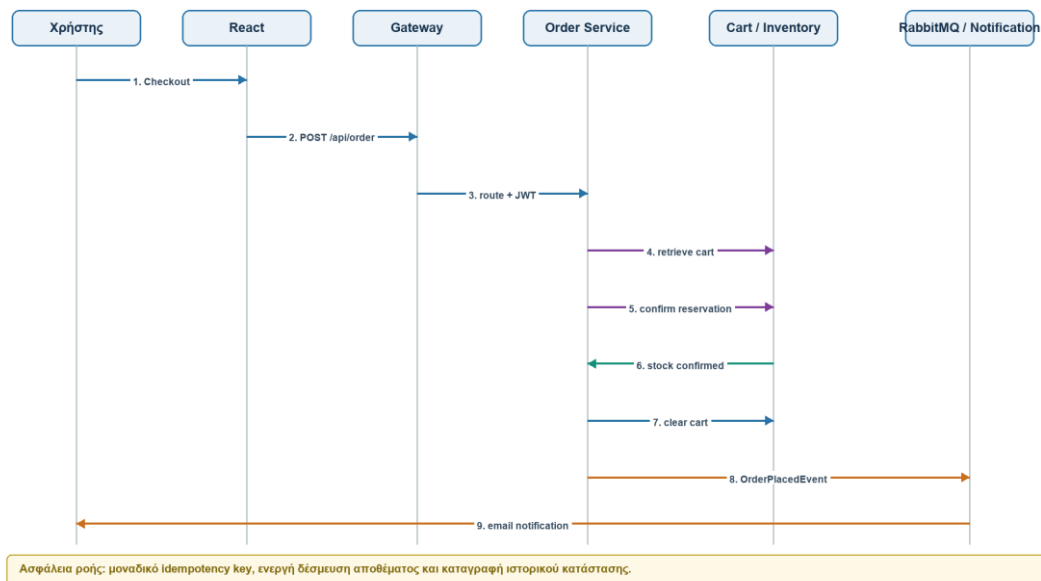
Για τη συστηματική επιλογή ορίων υπηρεσιών, το Service Cutter εξετάζει κριτήρια σύζευξης και οργανώνει υποψήφιους διαχωρισμούς με βάση το μοντέλο του συστήματος [44]. Στην παρούσα εργασία δεν εκτελέστηκε το εργαλείο. Η σχετική μελέτη χρησιμοποιείται ως συγκριτικό πλαίσιο για την ερμηνεία της διάσπασης: ο κατάλογος συγκεντρώνει περιγραφή και κατηγορίες προϊόντων, το απόθεμα τις ποσότητες και τις δεσμεύσεις, ενώ η παραγγελία την ιστορική επιχειρησιακή εγγραφή. Η ανεξαρτησία τους ορίζεται από τα συμβόλαια συνεργασίας και όχι μόνο από την ύπαρξη διαφορετικών φακέλων.

3.4 Ροή υποβολής παραγγελίας

Η υποβολή παραγγελίας είναι η κεντρική ροή του συστήματος, επειδή συνδυάζει ταυτότητα χρήστη, καλάθι, διαθεσιμότητα, μόνιμη αποθήκευση και ειδοποίηση. Το Διάγραμμα 3.2 αποτυπώνει τα κύρια βήματα. Ο έλεγχος αποθέματος είναι συγχρονισμένος: χωρίς επιβεβαίωση ενεργής δέσμευσης δεν δημιουργείται παραγγελία. Αντίθετα, η ειδοποίηση ακολουθεί ασύγχρονα μετά την επιτυχή καταχώριση.

Ροή υποβολής παραγγελίας

Συγχρονισμένες κλήσεις για δέσμευση αποθέματος και ασύγχρονη ειδοποίηση



Διάγραμμα 3.2: Ροή υποβολής παραγγελίας και δημοσίευσης συμβάντος ειδοποίησης.

- Ο πελάτης επιλέγει checkout στη React εφαρμογή. Δημιουργείται UUID idempotency key ώστε επαναλαμβανόμενο αίτημα από refresh ή δίκτυο να μην παράγει δεύτερη παραγγελία.
- Το αίτημα POST /api/order περνά από το API Gateway, το οποίο ελέγχει το JWT και το προωθεί στο Order Service.
- Το Order Service ανακτά το καλάθι του ταυτοποιημένου χρήστη. Κενό καλάθι οδηγεί σε απόρριψη του αιτήματος.
- Με OpenFeign επιβεβαιώνει τη δέσμευση inventory που αντιστοιχεί στο σταθερό reservation identifier του καλαθιού. Με την επιβεβαίωση αφαιρείται η ποσότητα από το πραγματικό απόθεμα.
- Δημιουργείται παραγγελία σε κατάσταση PENDING μαζί με τις γραμμές της και μια πρώτη εγγραφή ιστορικού. Στη συνέχεια το καλάθι εκκαθαρίζεται.
- Δημοσιεύεται OrderPlacedEvent στο orderExchange με routing key order.placed. Το Notification Service το καταναλώνει από notificationQueue και επιχειρεί αποστολή email.

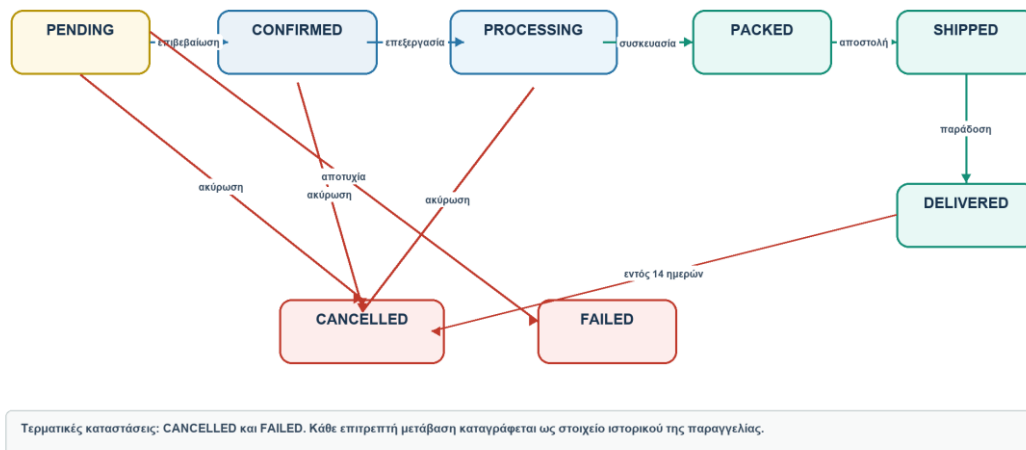
Η χρήση idempotency key είναι ουσιώδης στην πρακτική λειτουργία checkout. Αν ο πελάτης επαναλάβει το ίδιο αίτημα, το Order Service επιστρέφει την ήδη συνδεδεμένη παραγγελία αντί να επαναλάβει τη δέσμευση και την αποθήκευση. Για προσωρινή μη διαθεσιμότητα του Inventory Service, ο circuit breaker επιστρέφει ελεγχόμενο σφάλμα υπηρεσίας αντί να αφήσει την κλήση σε αναμονή.

3.5 Μοντέλο καταστάσεων παραγγελίας

Η κατάσταση παραγγελίας δεν είναι απλό πεδίο εμφάνισης. Χρησιμοποιείται ως κανόνας επιχειρησιακής ροής, ώστε κάθε αλλαγή να είναι επιτρεπτή μόνο από συγκεκριμένη προηγούμενη κατάσταση. Το Order Service διατηρεί επιπλέον ιστορικό αλλαγών, επομένως η τρέχουσα κατάσταση μπορεί να ερμηνευθεί μαζί με τη χρονική ακολουθία που την παρήγαγε.

Μοντέλο καταστάσεων παραγγελίας

Οι μεταβάσεις ελέγχονται από το Order Service και αποτυπώνονται στο ιστορικό κατάστασης

**Διάγραμμα 3.3: Μοντέλο καταστάσεων παραγγελίας.**

- PENDING → CONFIRMED, CANCELLED ή FAILED.
- CONFIRMED → PROCESSING ή CANCELLED.
- PROCESSING → PACKED ή CANCELLED.
- PACKED → SHIPPED και SHIPPED → DELIVERED.
- DELIVERED → CANCELLED μόνο εντός δεκατεσσάρων ημερών από την παράδοση, σύμφωνα με τον κανόνα που εφαρμόζει ο κώδικας.
- CANCELLED και FAILED αποτελούν τερματικές καταστάσεις.

Ο πελάτης μπορεί να ακυρώσει μόνο παραγγελία που του ανήκει και μόνο στα επιτρεπτά στάδια, ενώ ο διαχειριστής διαθέτει δικαιώματα διαχείρισης της ροής. Έτσι, η επιφάνεια του frontend δεν αποτελεί τη μόνη προστασία: ο έλεγχος εφαρμόζεται ξανά στο backend.

3.6 Σχεδιασμός API και επικοινωνίας

Το API Gateway εκθέτει ένα ενοποιημένο namespace προς τον client. Οι βασικές διαδρομές αντιστοιχούν σε υπηρεσία και δρομολογούνται με load-balanced URI, όπως lb://order-service. Η χρήση κοινών prefixes μειώνει την πολυπλοκότητα της React εφαρμογής και επιτρέπει την αλλαγή της τοποθεσίας μιας υπηρεσίας χωρίς αλλαγή στον client.

- GET /api/product/** και GET /api/categories/**: δημόσια ανάγνωση καταλόγου προϊόντων και κατηγοριών. Οι μεταβολές προϊόντων και κατηγοριών απαιτούν ρόλο ADMIN.
- /api/cart/**: ενέργειες καλαθιού για τον ταυτοποιημένο χρήστη. Η ταυτότητα προέρχεται από το subject του JWT και όχι από πεδίο που στέλνει ο client.
- /api/inventory/**: έλεγχος διαθεσιμότητας, δεσμεύσεις, επιβεβαίωση και αποδέσμευση. Οι διαχειριστικές ενέργειες όπως restock απαιτούν ADMIN.
- /api/order/**: δημιουργία, ανάκτηση, λίστα και ακύρωση παραγγελιών. Η αλλαγή κατάστασης και οι λίστες διαχείρισης ελέγχονται για ADMIN.
- /api/notification/**: endpoint της υπηρεσίας ειδοποιήσεων, ενώ η κύρια λειτουργία της ενεργοποιείται από μήνυμα RabbitMQ και όχι από άμεση κλήση του Order Service.

Για εσωτερικές κλήσεις, το Order Service χρησιμοποιεί declarative OpenFeign clients προς Cart και Inventory. Η επιλογή αυτή εκφράζει τα συμβόλαια επικοινωνίας ως διεπαφές Java και συνδυάζεται με discovery, ώστε οι κλήσεις να αναφέρονται στο όνομα της υπηρεσίας αντί για hard-coded host.

3.7 Σχεδιασμός δεδομένων

Το μοντέλο δεδομένων είναι κατανοημένο. Το Product Service διατηρεί οντότητες προϊόντος, κατηγορίας, χαρακτηριστικών και εικόνων. Το Inventory Service διατηρεί στοιχεία αποθέματος και ενεργές δεσμεύσεις. Το Order Service αποθηκεύει την παραγγελία, τις γραμμές της, το μοναδικό idempotency key και το ιστορικό κατάστασης. Το Cart Service επιλέγει Redis επειδή το καλάθι είναι βραχύβιο, συνδέεται με χρήστη και μεταβάλλεται συχνά.

Δεν χρησιμοποιείται κατανοημένη SQL συναλλαγή για όλη τη ροή. Αντί αυτής, το σύστημα εξασφαλίζει ότι η επιβεβαίωση αποθέματος προηγείται της αποθήκευσης παραγγελίας και αξιοποιεί δέσμευση με χρόνο λήξης στο στάδιο του καλαθιού. Η απόφαση αυτή είναι κατάλληλη για την παρούσα εκπαιδευτική υλοποίηση, ενώ σε μεγαλύτερη παραγωγική ανάπτυξη θα μπορούσε να εξεταστεί πληρέστερο saga ή μηχανισμός αντιστάθμισης.

3.8 Σχεδιασμός ασφάλειας και παρατηρησιμότητας

Ο Keycloak λειτουργεί ως πάροχος ταυτότητας. Η React εφαρμογή αρχικοποιεί τον client με έλεγχο check-sso και οδηγεί μη ταυτοποιημένο χρήστη στη σελίδα login. Μετά την αυθεντικοποίηση, το access token χρησιμοποιείται στις κλήσεις προς το Gateway. Το Gateway και οι υπηρεσίες Spring Security επαληθεύουν τον issuer και τους ρόλους του token, εφαρμόζοντας εξουσιοδότηση σε επίπεδο API. Η χρήση OAuth 2.0 και JWT επιτρέπει να μεταφέρεται επιβεβαιώσιμη πληροφορία ταυτότητας χωρίς το backend να διατηρεί δική του κατάσταση σύνδεσης.

- Οι αναγνώσεις καταλόγου είναι ελεύθερες, ώστε ο επισκέπτης να μπορεί να δει προϊόντα πριν συνδεθεί.
- Οι ενέργειες καλαθιού και παραγγελίας απαιτούν έγκυρο token και συνδέονται με τον subject του χρήστη.
- Οι μεταβολές catalog, restock και διαχειριστικές μεταβολές παραγγελιών προστατεύονται με ρόλο ADMIN.
- Το Micrometer tracing καταγράφει το πλήρες δείγμα αιτημάτων προς Zipkin, ώστε μία ροή να συσχετίζεται μεταξύ Gateway και υπηρεσιών. Η ανάγκη αυτή είναι σημαντική σε κατανοημένη αρχιτεκτονική, όπου ένα μόνο log δεν περιγράφει όλο το αίτημα.

3.9 Σχεδιαστικές παραδοχές και περιορισμοί

Ο σχεδιασμός δίνει προτεραιότητα στη σαφήνεια των ορίων και στην αξιοπιστία της βασικής ροής παραγγελίας. Δεν ενσωματώνει ακόμη πραγματική χρέωση, εξωτερικό φορέα αποστολής ή μηχανισμό πλήρους αντιστάθμισης για κάθε πιθανή αποτυχία μετά τη δέσμευση. Η αποστολή email εξαρτάται από σωστή ρύθμιση SMTP. Οι περιορισμοί αυτοί ορίζουν με ακρίβεια το τρέχον πεδίο της εφαρμογής και αποτελούν βάση για τη μελλοντική εξέλιξη που θα παρουσιαστεί στα συμπεράσματα.

3.10 Αρχιτεκτονικές αντισταθμίσεις

Η κατανοημένη δομή προσφέρει σαφή επιχειρησιακά όρια, όμως μεταφέρει μέρος της πολυπλοκότητας από τις εσωτερικές κλήσεις μίας εφαρμογής στο δίκτυο και στα συμβόλαια μεταξύ υπηρεσιών. Σε ένα μονολιθικό σύστημα, η πρόσβαση στο καλάθι ή στο απόθεμα θα μπορούσε να γίνει ως τοπική κλήση και η συνέπεια να εξαρτάται από μία σχεσιακή συναλλαγή. Στην παρούσα λύση, η ίδια επιχειρησιακή ενέργεια περιλαμβάνει αυθεντικοποίηση, δρομολόγηση, discovery, απομακρυσμένες κλήσεις και ανεξάρτητα data stores. Η χρήση Spring Cloud δεν αναιρεί αυτή την πολυπλοκότητα· την οργανώνει με ρητούς μηχανισμούς discovery, gateway, πελατών επικοινωνίας και ανθεκτικότητας.

Η επιλογή συγχρονισμένης επιβεβαίωσης reservation στο checkout έγινε επειδή η δημιουργία παραγγελίας δεν πρέπει να παρουσιάζεται ως επιτυχής χωρίς άμεση επιβεβαίωση διαθεσιμότητας. Αντίθετα, η ειδοποίηση δεν αλλάζει την εγκυρότητα της παραγγελίας και επομένως δημοσιεύεται ασύγχρονα. Ο διαχωρισμός αυτός περιορίζει τον χρόνο και τον αριθμό

των εξαρτήσεων της κρίσιμης διαδρομής, ενώ ταυτόχρονα διατηρεί την επιχειρησιακή συνέπεια εκεί όπου είναι απαραίτητη. Η απόφαση αποτελεί συνειδητή εφαρμογή του διαχωρισμού μεταξύ ισχυρής συνέπειας στη δημιουργία order και τελικής συνέπειας στην παράδοση ειδοποίησης [8, 14, 15].

3.11 Ταυτότητα δεδομένων και κατανομή ευθύνης

Σε ένα σύστημα παραγγελιών, η ταυτότητα ενός αντικειμένου δεν ταυτίζεται με την υπηρεσία που το εμφανίζει. Το προϊόν μπορεί να προβάλλεται στο καλάθι, στην παραγγελία και στη διαχείριση αποθέματος, αλλά η περιγραφή του ανήκει στο Product Service. Αντίστοιχα, η ποσότητα που είναι διαθέσιμη προς αγορά υπολογίζεται από το Inventory Service. Η διάκριση αυτή αποτρέπει την ερμηνεία κάθε αντιγράφου δεδομένων ως ανεξάρτητης πηγής αλήθειας και εξηγεί γιατί οι υπηρεσίες συνδέονται μέσω συγκεκριμένων αναγνωριστικών αντί μέσω κοινών οντοτήτων Java. Η αρχή της ιδιοκτησίας δεδομένων ανά υπηρεσία αποτελεί κεντρικό στοιχείο του σχεδιασμού microservices [7, 8, 11].

Στην υλοποίηση συνυπάρχουν τρεις διαφορετικές μορφές ταυτότητας. Το UUID προϊόντος χρησιμοποιείται για την αναφορά στην οντότητα του καταλόγου και για την αναζήτηση σχετικών προϊόντων. Ο κωδικός SKU συνδέει το προϊόν με τις εγγραφές αποθέματος και τις γραμμές του καλαθιού. Ο αριθμός παραγγελίας είναι νέο UUID, το οποίο δημιουργείται κατά την υποβολή και χρησιμοποιείται στην παρακολούθηση και στην ειδοποίηση. Η διαφορετική χρήση των αναγνωριστικών είναι ουσιώδης: ένας αριθμός παραγγελίας περιγράφει συγκεκριμένη συναλλαγή, ενώ το SKU εξακολουθεί να περιγράφει το ίδιο εμπορεύσιμο είδος σε πολλές παραγγελίες.

Ξεχωριστό ρόλο έχει το αναγνωριστικό προσωρινής δέσμευσης. Το Cart Service το παράγει με UUID.nameUUIDFromBytes από την ταυτότητα του χρήστη και το Order Service το αναπαράγει από την ταυτότητα του καλαθιού. Επομένως, η κράτηση που επιβεβαιώνεται στο checkout δεν αναζητείται με τον τελικό αριθμό παραγγελίας, ο οποίος δεν έχει ακόμη δημιουργηθεί. Αυτή η ακολουθία διαχωρίζει τη φάση επιλογής προϊόντων από τη φάση καταγραφής της αγοράς. Μετά την επιβεβαίωση, η παραγγελία αποκτά δικό της αναγνωριστικό και ανεξάρτητο κύκλο ζωής.

Κατά τη δημιουργία της παραγγελίας αντιγράφονται στις γραμμές της το SKU, η ονομασία, η τιμή και η ποσότητα του καλαθιού. Το αποθηκευμένο αυτό σύνολο συγκροτεί την αναπαράσταση της συγκεκριμένης αγοράς. Η εμφάνιση του ιστορικού δεν απαιτεί να επανασυναρμολογηθεί κάθε γραμμή από την τρέχουσα σελίδα του προϊόντος. Αντίθετα, οι ενέργειες καταλόγου και οι ενέργειες παραγγελίας διατηρούν διαφορετική σημασία: η πρώτη ομάδα αφορά το τι προσφέρεται στο κατάστημα, ενώ η δεύτερη αφορά το τι έχει ήδη καταχωριστεί για τον πελάτη.

3.12 Διαχωρισμός εντολών και παρατηρήσιμων αποτελεσμάτων

Κάθε βασική διαδρομή του συστήματος μπορεί να περιγραφεί ως ακολουθία εντολής, μεταβολής κατάστασης και παρουσίας αποτελέσματος. Για παράδειγμα, το πάτημα του πλήκτρου προσθήκης στο καλάθι αποτελεί πρόθεση του χρήστη. Η υπηρεσία καλαθιού επιχειρεί τον συγχρονισμό με το απόθεμα και αποθηκεύει το καλάθι. Η τελική ανανέωση της διεπαφής παρουσιάζει την απόκριση της υπηρεσίας. Η διάκριση αυτή βοηθά να περιγραφεί η αρχιτεκτονική χωρίς να αποδίδονται στη διεπαφή αποφάσεις που λαμβάνονται στο backend.

Στη ροή αναπλήρωσης αποθέματος, ο διαχειριστής εισάγει ποσότητα αύξησης και όχι τη νέα συνολική ποσότητα. Το Inventory Service προσθέτει την τιμή αυτή στο αποθηκευμένο σύνολο. Για υφιστάμενο απόθεμα δέκα τεμαχίων και αίτημα εννέα τεμαχίων, το αποτέλεσμα είναι δεκαεννέα. Η ορθή ερμηνεία της εντολής είναι απαραίτητη τόσο για τη φόρμα διαχείρισης όσο και για την αξιολόγηση: η σύγκριση πριν και μετά ελέγχει ότι εφαρμόστηκε η ζητούμενη μεταβολή, όχι απλώς ότι εμφανίστηκε ένα μήνυμα επιτυχίας.

Στην υποβολή παραγγελίας, το αποτέλεσμα της σύγχρονης κλήσης είναι η απόκριση με τον αριθμό και την κατάσταση της παραγγελίας. Η παραλαβή email αποτελεί διαφορετικό παρατηρήσιμο αποτέλεσμα της αλυσίδας ειδοποίησης. Δεν χρειάζεται να συγχωνευθούν σε ένα

ενιαίο χρονικό γεγονός. Η αναγνώριση του ίδιου αριθμού παραγγελίας στη σελίδα επιβεβαίωσης και στο Mailtrap επιτρέπει τη συσχέτιση των δύο αποτελεσμάτων χωρίς να απαιτείται υπόθεση για τον ακριβή χρόνο μεσολάβησης. Αυτός ο διαχωρισμός αντανάκλα τη χρήση σύγχρονης επικοινωνίας για το checkout και μηνυμάτων για την ενημέρωση του πελάτη [8].

3.13 Αναλυτική αποτύπωση των συμβολαίων HTTP

Τα συμβόλαια των υπηρεσιών προσδιορίζονται από τον συνδυασμό μεθόδου HTTP, διαδρομής, παραμέτρων, σώματος και απόκρισης. Η διαδρομή από μόνη της δεν αρκεί: το GET /api/product και το POST /api/product αναφέρονται στην ίδια συλλογή, αλλά εκφράζουν ανάκτηση και δημιουργία αντίστοιχα. Οι πίνακες της ενότητας αποτυπώνουν τις δηλώσεις των controllers του έργου. Η πολιτική πρόσβασης εξετάζεται χωριστά, ώστε η απαρίθμηση διαδρομών να μην εκλαμβάνεται ως δήλωση ότι όλες είναι δημόσιες [12].

3.13.1 Κατάλογος και κατηγορίες

Στο ProductController συνυπάρχουν ανάκτηση με UUID και ανάκτηση με SKU. Το UUID προσδιορίζει την οντότητα σε ενέργειες ενημέρωσης και σε αιτήματα σχετικών προϊόντων, ενώ το SKU εξυπηρετεί τη συσχέτιση με απόθεμα και γραμμές αγοράς. Η CategoryController διαχειρίζεται το πρότυπο που χρησιμοποιείται πριν αποθηκευθούν τα χαρακτηριστικά ενός προϊόντος. Με αυτή τη διάκριση, η αλλαγή του προτύπου αποτελεί διαχειριστική ενέργεια διαφορετική από την αλλαγή της τιμής ενός προϊόντος.

Πίνακας 3.2: Συμβόλαια HTTP προϊόντων και κατηγοριών.

Μέθοδος	Διαδρομή	Σημασία και δεδομένα
GET	/api/product	Επιστρέφει λίστα ProductResponse για τον κατάλογο.
POST	/api/product	Δέχεται ProductRequest και δημιουργεί προϊόν μετά την επικύρωση της κατηγορίας και των χαρακτηριστικών.
PUT	/api/product/{id}	Ενημερώνει το προϊόν με το συγκεκριμένο UUID από τα πεδία του ProductRequest.
DELETE	/api/product/{id}	Διαγράφει την οντότητα προϊόντος που προσδιορίζεται από το UUID.
GET	/api/product/sku/{skuCode}	Ανακτά προϊόν με τον εμπορικό κωδικό SKU αντί με το εσωτερικό UUID.
GET	/api/product/{id}/related	Επιστρέφει βαθμολογημένες προτάσεις. Η παράμετρος limit καθορίζει το ζητούμενο πλήθος, το οποίο περιορίζει η υπηρεσία.
GET	/api/categories	Επιστρέφει τις κατηγορίες μαζί με τη δομή των προτύπων τους.
POST	/api/categories	Δέχεται CategoryRequest για δημιουργία κατηγορίας και ορισμών χαρακτηριστικών.
PUT	/api/categories/{id}	Ενημερώνει την κατηγορία με το συγκεκριμένο UUID.
DELETE	/api/categories/{id}	Ζητά διαγραφή της επιλεγμένης κατηγορίας από την υπηρεσία κατηγοριών.

Η απόκριση προϊόντος μεταφέρει όσα χρειάζεται η διεπαφή: αναγνωριστικά, περιγραφή, τιμή, κατηγορία, κατάσταση, χαρακτηριστικά και διευθύνσεις εικόνων. Η απόκριση σχετικών προϊόντων περιβάλλει αυτή την πληροφορία με βαθμολογία και αιτιολογήσεις. Επομένως, η ίδια αναπαράσταση προϊόντος μπορεί να χρησιμοποιείται στον κατάλογο και στις προτάσεις, χωρίς να μετατρέπεται η βαθμολογία σε μόνιμο γνώρισμα του προϊόντος. Η βαθμολογία είναι αποτέλεσμα της σύγκρισης με συγκεκριμένο προϊόν αναφοράς.

3.13.2 Καλάθι και παραγγελία

Οι διαδρομές καλαθιού χρησιμοποιούν το subject του JWT για την επιλογή της εγγραφής του χρήστη. Η αλλαγή ποσότητας λαμβάνει το SKU από τη διαδρομή και την ποσότητα από παράμετρο αιτήματος. Η διαγραφή ενός είδους και η εκκαθάριση ολόκληρου του καλαθιού διατηρούν διακριτές διευθύνσεις, επειδή έχουν διαφορετικό εύρος μεταβολής. Στις παραγγελίες, η προσωπική προβολή my-orders επιλέγει την ταυτότητα από το token, ενώ οι υπόλοιπες ανακτήσεις δέχονται συγκεκριμένο αναγνωριστικό ή email σύμφωνα με την αντίστοιχη διαδρομή.

Πίνακας 3.3: Συμβόλαια HTTP καλαθιού και παραγγελίας.

Μέθοδος	Διαδρομή	Σημασία και δεδομένα
GET	/api/cart	Ανακτά το καλάθι του subject που περιέχεται στο JWT του αιτήματος.
POST	/api/cart/add	Δέχεται γραμμή CartItem και επιστρέφει το ενημερωμένο καλάθι μετά τον συγχρονισμό αποθέματος.
PUT	/api/cart/item/{skuCode}	Χρησιμοποιεί την παράμετρο quantity για την αλλαγή ποσότητας του είδους.
DELETE	/api/cart/item/{skuCode}	Αφαιρεί το συγκεκριμένο είδος και επιστρέφει το ενημερωμένο καλάθι.
DELETE	/api/cart/clear	Απελευθερώνει τη δέσμευση και διαγράφει το καλάθι. Ο controller δηλώνει απόκριση 204.
POST	/api/order	Δέχεται OrderRequest, Authorization και προαιρετική κεφαλίδα Idempotency-Key. Ο controller δηλώνει απόκριση 201.
GET	/api/order	Επιστρέφει τη λίστα παραγγελιών για τη διαχειριστική λειτουργία.
GET	/api/order/my-orders	Επιλέγει την ταυτότητα πελάτη από το JWT και ανακτά το αντίστοιχο ιστορικό.
GET	/api/order/{orderNumber}	Ανακτά απόκριση παραγγελίας με βάση τον δημόσιο αριθμό UUID.
GET	/api/order/customer/{email}	Αναζητεί παραγγελίες με βάση τη διεύθυνση που παρέχεται στη διαδρομή.
PATCH	/api/order/{orderNumber}/status	Μεταβάλλει την κατάσταση σύμφωνα με την παράμετρο status και τους κανόνες μετάβασης.
POST	/api/order/{orderNumber}/cancel	Καλεί την επιχειρησιακή διαδικασία ακύρωσης με έλεγχο ταυτότητας και επιλεξιμότητας.

Η κεφαλίδα Idempotency-Key μεταφέρει μεταδεδομένο της προσπάθειας υποβολής και όχι χαρακτηριστικό ενός προϊόντος. Η τοποθέτησή της στην κεφαλίδα διαχωρίζει το αίτημα ταυτοποίησης της προσπάθειας από την επιχειρησιακή περιγραφή της αγοράς. Ο controller τη δέχεται ως προαιρετική και την προωθεί στην υπηρεσία. Η συμπεριφορά αναγνώρισης επανάληψης υλοποιείται στο service και αναλύεται στο Κεφάλαιο 4, επομένως δεν συνάγεται μόνο από την ύπαρξη της κεφαλίδας.

3.13.3 Απόθεμα και ενέργειες δέσμευσης

Στην υπηρεσία αποθέματος, η απλή ανάκτηση επιστρέφει συνολική και διαθέσιμη ποσότητα. Ο έλεγχος διαθεσιμότητας δέχεται πολλαπλές γραμμές, χωρίς να δημιουργεί από μόνος του κράτηση. Η δημιουργία κράτησης, η επιβεβαίωσή της και η απελευθέρωση εκφράζονται με διαφορετικές εντολές. Παρότι οι μέθοδοι confirm και release δέχονται παράμετρο με όνομα

orderNumber, στη ροή καλαθιού η τιμή είναι το αναγνωριστικό της προσωρινής κράτησης που περιγράφηκε στην Ενότητα 3.11.

Πίνακας 3.4: Συμβόλαιο HTTP διαχείρισης αποθέματος.

Μέθοδος	Διαδρομή	Σημασία και δεδομένα
GET	/api/inventory/sku/{skuCode}	Επιστρέφει SKU, συνολική ποσότητα και υπολογισμένη διαθέσιμη ποσότητα.
POST	/api/inventory/check	Δέχεται λίστα ειδών και ποσοτήτων και επιστρέφει αποτέλεσμα διαθεσιμότητας ανά SKU.
POST	/api/inventory/reserve	Δέχεται αναγνωριστικό δέσμευσης και γραμμές. Επιστρέφει το αναγνωριστικό και ένδειξη reserved.
POST	/api/inventory/confirm	Χρησιμοποιεί παράμετρο orderNumber για την επιβεβαίωση των συσχετισμένων δεσμεύσεων.
POST	/api/inventory/release	Χρησιμοποιεί παράμετρο orderNumber για διαγραφή των αντίστοιχων δεσμεύσεων.
POST	/api/inventory/restock	Δέχεται SKU και ποσότητα αύξησης μέσω InventoryRestockRequest.
POST	/api/inventory/create	Δέχεται στοιχεία αρχικής εγγραφής αποθέματος και επιστρέφει InventoryResponse.

Ο διαχωρισμός check και reserve έχει σημασία για την κατανόηση της διεπαφής. Η εμφάνιση διαθέσιμης ποσότητας είναι παρατήρηση κατά την ανάκτηση. Η μεταβολή καλαθιού ενεργοποιεί ξεχωριστή διαδικασία συγχρονισμού και απόκριση δέσμευσης. Αντίστοιχα, το restock προσθέτει ποσότητα, ενώ το create εκφράζει αρχική εγγραφή αποθέματος. Τα ονόματα των ενεργειών επιτρέπουν να περιγραφεί η επιχειρησιακή σημασία χωρίς να συγχέονται όλες οι μεταβολές σε έναν γενικό όρο ενημέρωσης.

3.14 Μοντέλο αποθήκευσης και σχέσεις οντοτήτων

Το μοντέλο δεδομένων ακολουθεί τις ευθύνες των υπηρεσιών. Στον κατάλογο, η οντότητα Product συγκεντρώνει τα βασικά γνωρίσματα και διατηρεί συλλογές ProductImage και ProductAttribute. Στις παραγγελίες, η Order συγκεντρώνει τις γραμμές αγοράς και τις εγγραφές ιστορικού. Οι σχέσεις αυτές βρίσκονται εντός των αντίστοιχων υπηρεσιών. Η σύνδεση προϊόντος με γραμμή παραγγελίας ή εγγραφή αποθέματος εκφράζεται με SKU και μεταφερόμενα δεδομένα, όχι με κοινή οντότητα JPA που διατρέχει διαφορετικά έργα.

Πίνακας 3.5: Βασικές οντότητες και σχέσεις του καταλόγου και των παραγγελιών.

Οντότητα	Βασικά γνωρίσματα	Σχέση και επιχειρησιακή σημασία
Product	id, skuCode, name, price, category, status	Κεντρική οντότητα καταλόγου. Διαθέτει συλλογές εικόνων και πραγματικών χαρακτηριστικών προϊόντος.
ProductImage	URL εικόνας και αναφορά προϊόντος	Παιδί του προϊόντος. Η σειρά της συλλογής διατηρείται με display_order.
ProductAttribute	attributeKey, attributeValue και αναφορά προϊόντος	Αποθηκεύει συγκεκριμένη τιμή χαρακτηριστικού για ένα προϊόν, όχι τον ορισμό του προτύπου.
Category	id, name, description	Συγκεντρώνει τους ορισμούς χαρακτηριστικών που χρησιμοποιούνται στην επικύρωση προϊόντων.

Οντότητα	Βασικά γνωρίσματα	Σχέση και επιχειρησιακή σημασία
CategoryAttribute Definition	attributeName, inputType, required	Συνδέεται με μία κατηγορία και καθορίζει το όνομα, τον τύπο και την υποχρεωτικότητα πεδίου.
Επιλογές προτύπου	allowedValues, display_order	Συλλογή τιμών του ορισμού. Υποστηρίζει ελεγχόμενες επιλογές για τύπο SELECT.
Order	id, orderNumber, customerEmail, status	Ρίζα παραγγελίας με γραμμές ειδών και ιστορικό καταστάσεων. Το orderNumber είναι διαφορετικό από το αριθμητικό id.
OrderLineItems	SKU, ονομασία, τιμή, ποσότητα	Περιγράφει μία γραμμή της αποθηκευμένης αγοράς και συνδέεται με την παραγγελία.
OrderStatusHistory	previousStatus, status, changedAt, changedBy, note	Καταγράφει μεταβολή κατάστασης και συνδέεται με την παραγγελία μέσω order_id.

3.14.1 Ορισμός χαρακτηριστικού και πραγματική τιμή

Η διάκριση CategoryAttributeDefinition και ProductAttribute αποτελεί ουσιαστικό σημείο του σχεδιασμού. Ο ορισμός περιγράφει τι μπορεί να δηλωθεί, με ποιον τύπο και με ποια υποχρεωτικότητα. Η πραγματική εγγραφή προϊόντος περιγράφει τι δηλώθηκε για ένα συγκεκριμένο είδος. Για παράδειγμα, το πρότυπο μπορεί να απαιτεί αριθμητικό χαρακτηριστικό RAM, ενώ δύο προϊόντα της κατηγορίας μπορούν να έχουν διαφορετικές τιμές. Το πρότυπο παραμένει κοινό, αλλά οι τιμές ανήκουν στα επιμέρους προϊόντα.

Οι επιτρεπόμενες τιμές του τύπου SELECT αποθηκεύονται ως συλλογή συνδεδεμένη με τον ορισμό. Η σειρά τους διατηρείται με display_order. Αντίστοιχη έννοια σειράς χρησιμοποιείται για τις εικόνες προϊόντος και για τους ορισμούς κατηγορίας. Η αποθήκευση της σειράς εξυπηρετεί την προβλέψιμη παρουσίαση στη διεπαφή: το σύνολο των δεδομένων δεν είναι μόνο ασύνδετα πεδία, αλλά περιλαμβάνει και τη διάταξη με την οποία προβάλλονται οι επιλογές και οι εικόνες.

Στις σχέσεις προϊόντος με εικόνες και χαρακτηριστικά χρησιμοποιούνται cascade και orphanRemoval. Κατά την ενημέρωση, η υπηρεσία καθαρίζει τις αντίστοιχες συλλογές και προσθέτει τα δεδομένα του νέου αιτήματος. Η λογική αυτή αντιμετωπίζει τις συλλογές ως τμήματα της οντότητας προϊόντος. Η εξήγηση αφορά τον τρόπο διαχείρισης των παιδιών της οντότητας και δεν σημαίνει ότι τα ίδια δεδομένα κατέχονται από πολλές ανεξάρτητες υπηρεσίες.

3.14.2 Ταυτότητα παραγγελίας και ιστορική πληροφορία

Στην Order υπάρχουν αριθμητικό πρωτεύον κλειδί id και μοναδικός αριθμός orderNumber τύπου UUID. Το πρώτο εξυπηρετεί τη σχεσιακή οργάνωση, ενώ το δεύτερο χρησιμοποιείται στις διαδρομές και στις οθόνες. Η στήλη idempotency_key διατηρεί το κλειδί της προσπάθειας. Τα πεδία createdAt, updatedAt και deliveredAt περιγράφουν διαφορετικά χρονικά γεγονότα. Ο χρόνος τελευταίας ενημέρωσης δεν πρέπει να ταυτίζεται εννοιολογικά με τον χρόνο παράδοσης, επειδή η παραγγελία μπορεί να ενημερώνεται για περισσότερους λόγους.

Το ιστορικό διατηρεί την προηγούμενη και τη νέα κατάσταση μαζί με τον υπεύθυνο μεταβολής και σημείωση. Η συλλογή ταξινομείται κατά changedAt και id, παρέχοντας ορισμένη σειρά ακόμη όταν οι χρόνοι συμπίπτουν. Αυτή η αναπαράσταση επιτρέπει να παρουσιαστεί η τρέχουσα εικόνα της παραγγελίας και να εξεταστούν οι μεταβολές που έχουν καταγραφεί. Η σχέση με την Order διατηρεί κάθε εγγραφή ιστορικού εντός του πλαισίου της παραγγελίας στην οποία ανήκει.

3.14.3 Χρονική διάρκεια καλαθιού και δέσμευσης

Το καλάθι αποθηκεύεται ως RedisHash με αναγνωριστικό χρήστη και συλλογή CartItem. Η δηλωμένη διάρκεια ζωής είναι 604800 δευτερόλεπτα. Η προσωρινή κράτηση αποθέματος έχει διαφορετικό χρόνο λήξης, ο οποίος ορίζεται κατά τη δημιουργία σε δεκαπέντε λεπτά. Πρόκειται για δύο διακριτές έννοιες: το καλάθι αποτυπώνει την επιλογή προϊόντων, ενώ η κράτηση επηρεάζει τον υπολογισμό διαθέσιμου αποθέματος. Η διάρκεια ζωής της πρώτης αναπαράστασης δεν χρησιμοποιείται ως συνώνυμο της δεύτερης.

Πίνακας 3.6: Διαφορετικοί κύκλοι ζωής δεδομένων στην υλοποίηση.

Δεδομένο	Ταυτότητα και αποθήκευση	Διάρκεια και χρήση
Καλάθι	subject χρήστη, RedisHash Cart	Η οντότητα δηλώνει TTL 604800 δευτερολέπτων, δηλαδή επτά ημερών. Περιέχει τις τρέχουσες επιλογές ειδών.
Δέσμευση	UUID εγγραφής, SKU και κοινό αναγνωριστικό κράτησης	Κατά τη δημιουργία ορίζεται expiresAt σε δεκαπέντε λεπτά. Η επιβεβαίωση ή απελευθέρωση διαγράφει τις αντίστοιχες εγγραφές.
Προϊόν	UUID και μοναδικό SKU στη βάση καταλόγου	Διατηρείται ως εγγραφή καταλόγου με χρόνους δημιουργίας και τελευταίας ενημέρωσης.
Παραγγελία	Αριθμητικό id και μοναδικό orderNumber	Αποθηκεύεται με τον χρόνο υποβολής, την κατάσταση και τις γραμμές της αγοράς.
Μεταβολή κατάστασης	Αριθμητικό id και αναφορά order_id	Προστίθεται στο ιστορικό και ταξινομείται με changedAt και id, ώστε να διατηρείται η ακολουθία μεταβολών.
Προσπάθεια checkout	fingerprint και key στο attemptRef	Αφορά την κατάσταση του React component κατά τη συγκεκριμένη προσπάθεια υποβολής.

Οι διαφορετικοί κύκλοι ζωής διαχωρίζουν την προσπάθεια στη διεπαφή, τις επιλογές καλαθιού, τις κρατήσεις αποθέματος και την αποθηκευμένη αγορά. Το κοινό SKU συνδέει τις αναπαραστάσεις χωρίς να εξισώνει τη σημασία ή τη διάρκειά τους.

4. Υλοποίηση

4.1 Τεχνολογική στοίβα και οργάνωση αποθετηρίου

Η υλοποίηση οργανώνεται ως πολυ-υπηρεσιακό backend με Java 21 και Spring Boot, καθώς και ως ανεξάρτητο frontend με React και Vite. Κάθε backend υπηρεσία διαθέτει δικό της Maven project, configuration και θύρα εκτέλεσης. Η διάταξη αυτή επιτρέπει να εκτελείται ή να εξελίσσεται μία υπηρεσία χωρίς να απαιτείται ανακατασκευή μιας ενιαίας μονολιθικής εφαρμογής.

- Frontend: React 19, Vite, React Router, Axios, Bootstrap και keycloak-js.
- Backend: Java 21, Spring Boot, Spring Web, Spring Data JPA, Spring Security και Maven.
- Υποδομή Spring Cloud: Eureka Discovery Server, Spring Cloud Gateway, OpenFeign και Resilience4j.
- Αποθήκευση και messaging: PostgreSQL για Product, Inventory και Order, Redis για Cart, RabbitMQ για γεγονότα ειδοποίησης.
- Διαλειτουργικότητα και παρατήρηση: Keycloak για identity and access management, Micrometer Tracing και Zipkin για distributed tracing.
- Τοπική εκτέλεση: Docker Compose για RabbitMQ, Redis, Zipkin και Keycloak, ενώ οι υπηρεσίες Spring Boot εκτελούνται με τις αντίστοιχες ρυθμίσεις τους.

Στο backend ακολουθείται η συνήθης διάταξη controller, service, repository και domain model. Οι controllers ορίζουν HTTP endpoints και μετατρέπουν αιτήματα/απαντήσεις σε DTOs, τα services περιέχουν τους επιχειρησιακούς κανόνες και τα repositories χειρίζονται την πρόσβαση σε μόνιμη αποθήκευση. Η διάκριση αυτή περιορίζει τη λογική των HTTP endpoints και διευκολύνει τον έλεγχο των υπηρεσιών ανεξάρτητα από το UI.

Στο αποθετήριο, η React εφαρμογή οργανώνει τη διεπαφή σε components, σύμφωνα με το προγραμματιστικό μοντέλο που περιγράφεται στην επίσημη τεκμηρίωση [45]. Το Vite χρησιμοποιείται για τη διαδικασία ανάπτυξης και παραγωγής του frontend [46], το Axios συγκεντρώνει την επικοινωνία HTTP και το Maven διαχειρίζεται τις εξαρτήσεις των Java εφαρμογών. Τα στοιχεία αυτά καταγράφονται στα αρχεία διαμόρφωσης και στις εισαγωγές του κώδικα. Ο διαχωρισμός components αφορά την οργάνωση της παρουσίασης, ενώ οι επιχειρησιακοί κανόνες εξακολουθούν να εφαρμόζονται στις υπηρεσίες backend.

4.2 Υλοποίηση της εφαρμογής React

Η React εφαρμογή αποτελεί την ορατή πλευρά του συστήματος και παρέχει τις οθόνες καταλόγου, λεπτομέρειας προϊόντος, καλαθιού, checkout, ιστορικού παραγγελιών, προφίλ και διαχείρισης. Το σημείο εισόδου αρχικοποιεί τον Keycloak client με check-ssso. Όταν δεν υπάρχει ενεργή συνεδρία, ο χρήστης ανακατευθύνεται στη διαδικασία σύνδεσης. Μετά την επιτυχή αυθεντικοποίηση, τα tokens διατηρούνται τοπικά και το Axios χρησιμοποιείται για τις κλήσεις προς τα endpoints του Gateway.

4.2.1 Δρομολόγηση και έλεγχος πρόσβασης

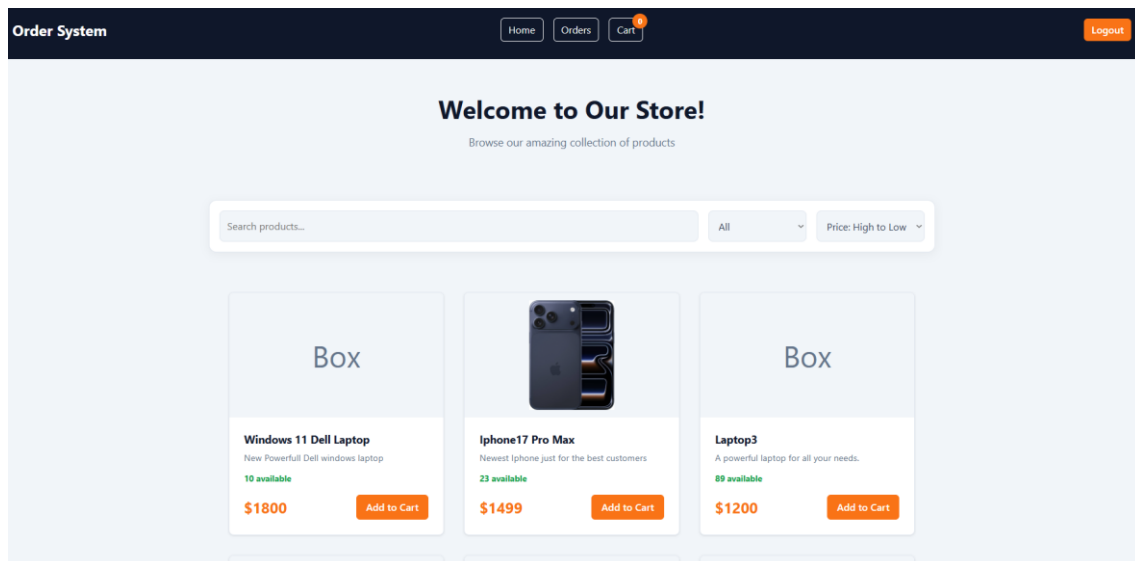
Το React Router ορίζει τις διαδρομές της εφαρμογής. Η κοινή διάταξη περιλαμβάνει πλοήγηση προς προϊόντα, παραγγελίες και καλάθι. Οι σελίδες checkout, orders και profile είναι διαθέσιμες σε ταυτοποιημένο χρήστη, ενώ η διαδρομή διαχείρισης εμφανίζεται μόνο όταν ο Keycloak client αναγνωρίζει τον ρόλο ADMIN. Ο έλεγχος στο frontend βελτιώνει την εμπειρία χρήστη, όμως δεν θεωρείται οριστική δικλείδα ασφαλείας: η εξουσιοδότηση εφαρμόζεται ξανά στους Spring Security κανόνες του backend.

- Η αρχική σελίδα και η σελίδα λεπτομέρειας προϊόντος χρησιμοποιούν τα API του Product Service.
- Το καλάθι διαβάζει και ενημερώνει τις γραμμές του μέσω Cart Service, πάντοτε για τον χρήστη που προκύπτει από το JWT.

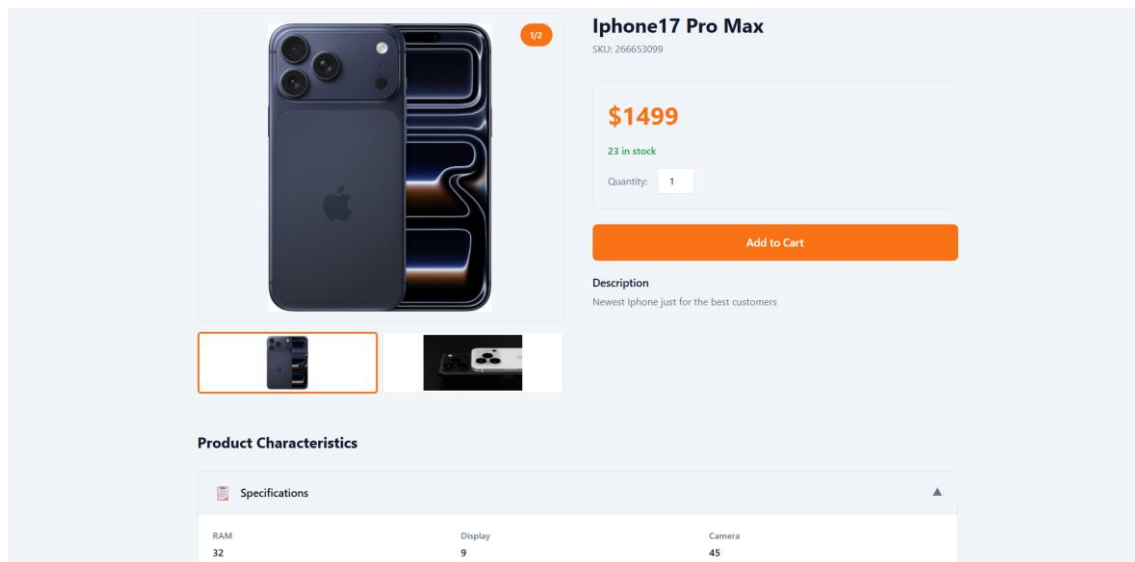
- Η σελίδα checkout ελέγχει ότι το καλάθι δεν είναι κενό, δημιουργεί UUID idempotency key και καλεί το API δημιουργίας παραγγελίας.
- Η σελίδα orders επαναφορτώνει περιοδικά τις παραγγελίες ανά δέκα δευτερόλεπτα και εμφανίζει τις επιτρεπτές ενέργειες ακύρωσης.
- Η σελίδα admin οργανώνει τις λειτουργίες catalog και διαχείρισης για τον ρόλο ADMIN.

4.2.2 Κατάλογος και λεπτομέρεια προϊόντος

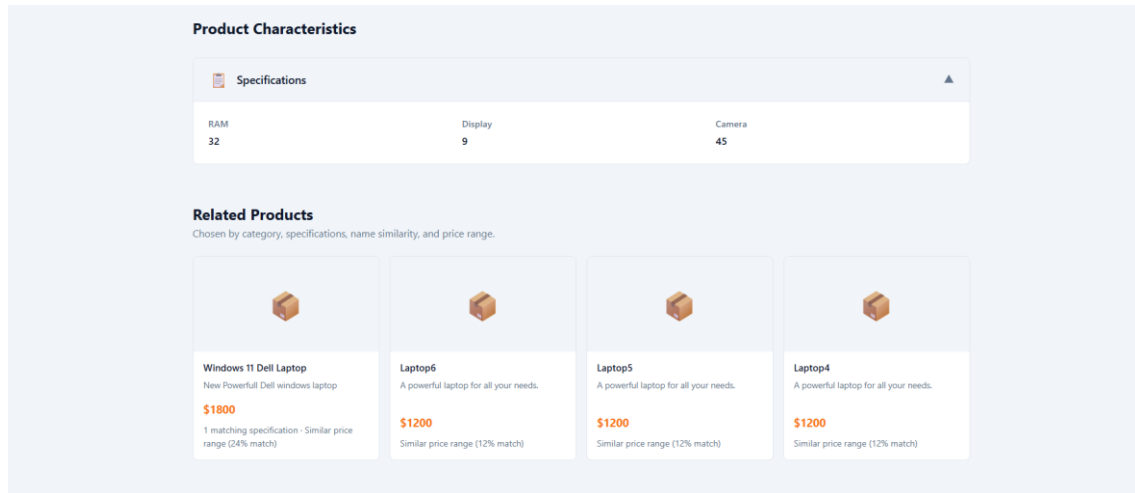
Η διεπαφή καταλόγου παρουσιάζει προϊόντα και επιτρέπει μετάβαση στη λεπτομέρειά τους. Η σελίδα λεπτομέρειας εμφανίζει τις εικόνες, τα δυναμικά χαρακτηριστικά κατηγορίας, την τιμή και προϊόντα που θεωρούνται σχετικά. Η εφαρμογή δεν υπολογίζει τη συνάφεια τοπικά: ζητά το αποτέλεσμα από το Product Service, ώστε ο κανόνας προτάσεων να παραμένει συγκεντρωμένος στο domain του catalog.



Εικόνα 4.1: Αρχική σελίδα και κατάλογος προϊόντων.



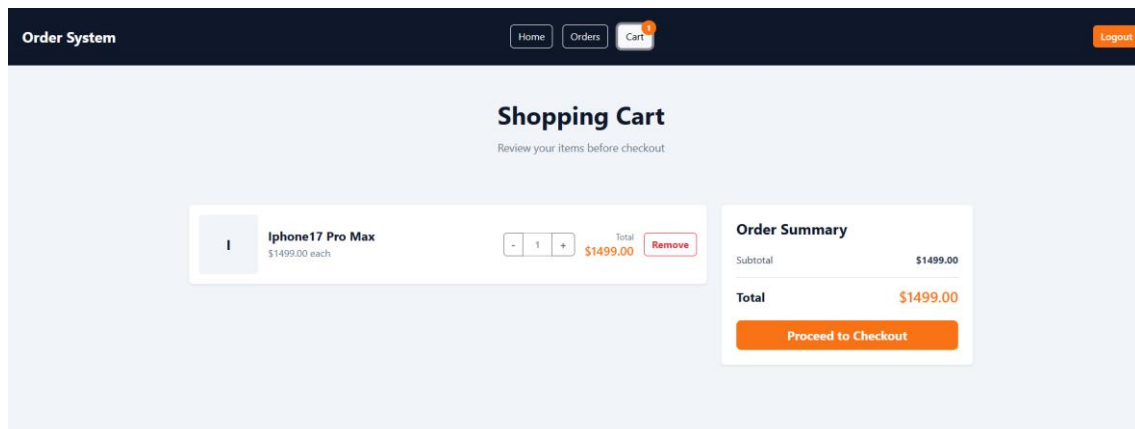
Εικόνα 4.2: Σελίδα λεπτομέρειας προϊόντος.



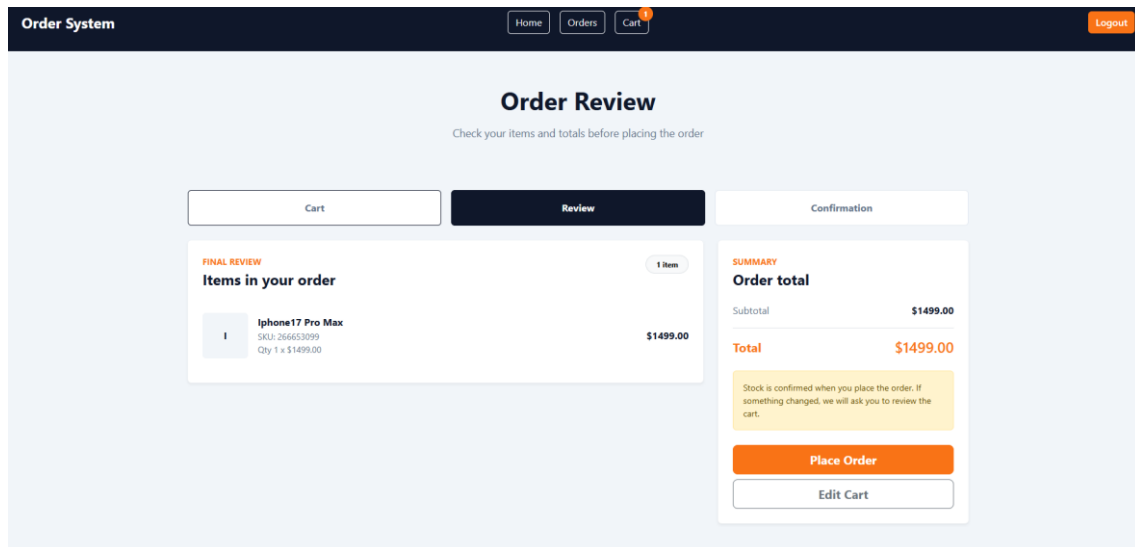
Εικόνα 4.3: Χαρακτηριστικά και σχετικές προτάσεις προϊόντος.

4.2.3 Καλάθι και checkout

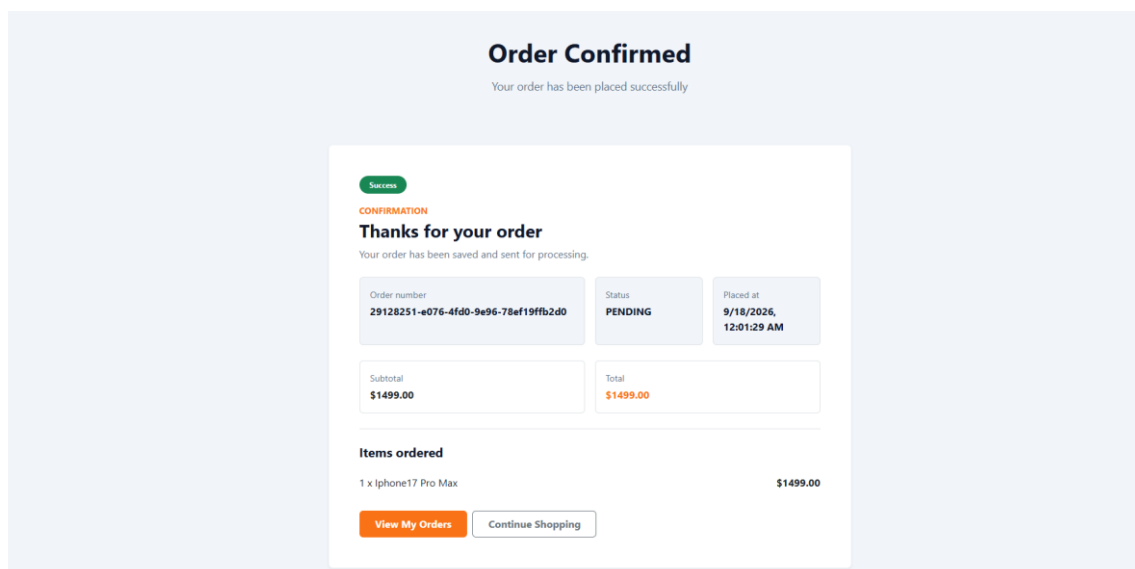
Η οθόνη καλαθιού επιτρέπει μεταβολή ποσότητας και αφαίρεση προϊόντων. Κάθε αλλαγή δεν είναι μόνο οπτική: προωθείται στο Cart Service, το οποίο συγχρονίζει τη δέσμευση αποθέματος. Κατά το checkout, η React εφαρμογή επαναφορτώνει το τρέχον καλάθι, αποτρέπει client-side την υποβολή κενού καλαθιού και υποβάλλει την παραγγελία. Μετά από επιτυχία ο χρήστης οδηγείται σε σελίδα επιβεβαίωσης, ενώ σε σφάλμα εμφανίζεται μήνυμα που διατηρεί το πλαίσιο της ενέργειας.



Εικόνα 4.4: Σελίδα καλαθιού.



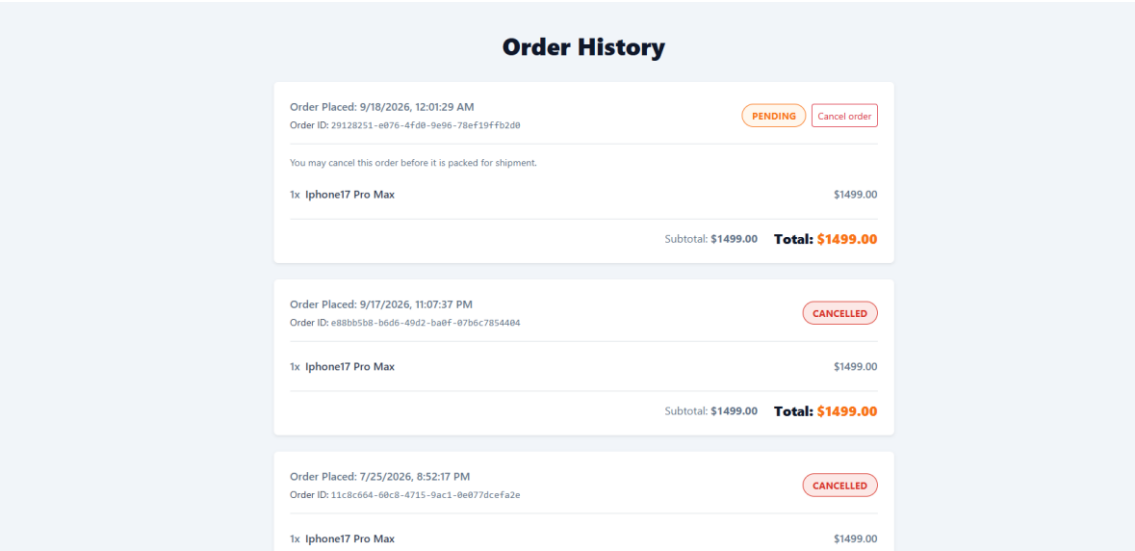
Εικόνα 4.5: Σελίδα ανασκόπησης checkout.



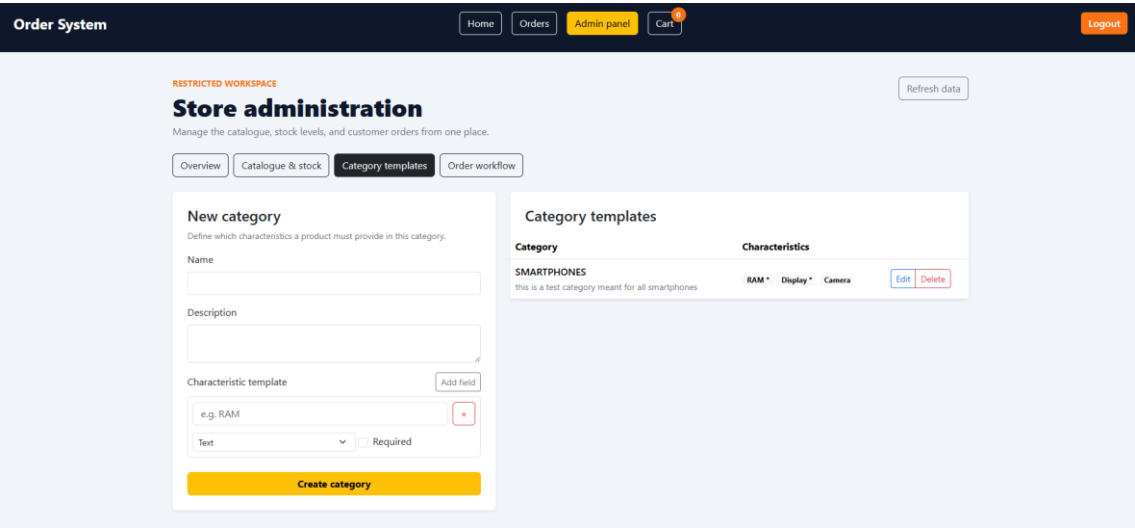
Εικόνα 4.6: Επιτυχής επιβεβαίωση παραγγελίας.

4.2.4 Παρακολούθηση παραγγελιών και διαχείριση

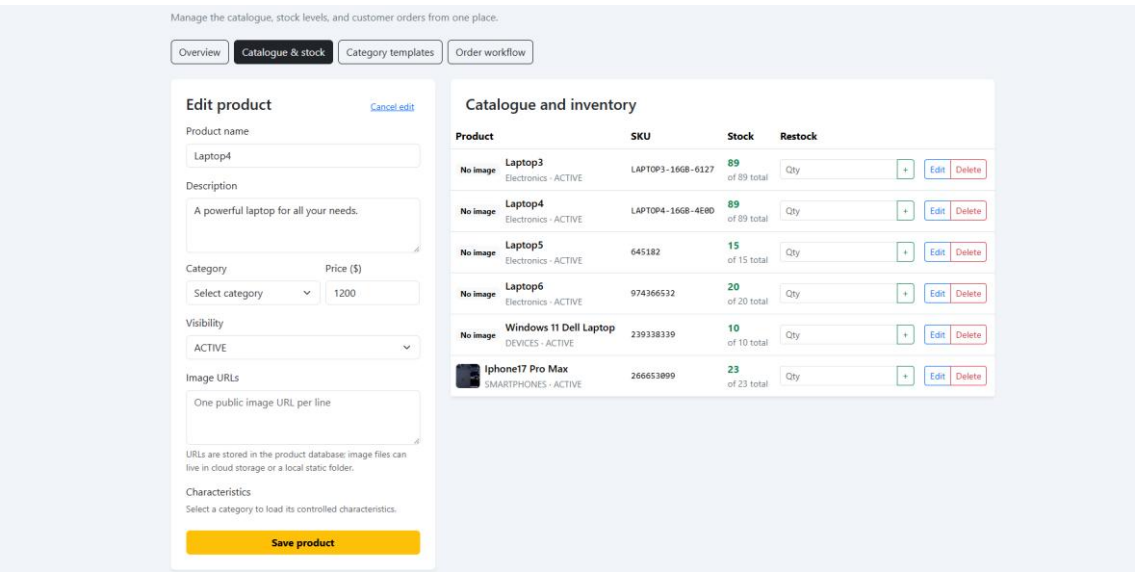
Η σελίδα παραγγελιών απευθύνεται στον πελάτη και εμφανίζει το προσωπικό ιστορικό, την τρέχουσα κατάσταση και τη δυνατότητα ακύρωσης όταν ο κανόνας του backend το επιτρέπει. Η περιοδική ανανέωση ανά δέκα δευτερόλεπτα προσφέρει ενημέρωση χωρίς να απαιτείται WebSocket υποδομή στην παρούσα έκδοση. Η σελίδα admin συγκεντρώνει τις λειτουργίες δημιουργίας και επεξεργασίας catalog, καθώς και τις διαχειριστικές ενέργειες επί των παραγγελιών.



Εικόνα 4.7: Ιστορικό παραγγελιών πελάτη.



Εικόνα 4.8: Διαχείριση προτύπων κατηγορίας.



Εικόνα 4.9: Διαχείριση καταλόγου και αποθέματος.

Ανάπτυξη ενός Συστήματος Διαχείρισης Παραγγελιών (Order Management System) βασισμένου σε Microservices με Spring Boot και Spring Cloud

Manage the catalogue, stock levels, and customer orders from one place.

Overview Catalogue & stock Category templates **Order workflow**

Order workflow
Choose a status, then save it. Cancellations and all other changes are added to the audit history.

Order	Customer	Items	Total	Status	Created	
e88b5b8...	test@test.com	1	\$1499.00	CANCELLED	9/17/2026, 11:07:37 PM	Details
11c8c664...	test@test.com	1	\$1499.00	CANCELLED	7/25/2026, 8:52:17 PM	Details
30f936c9...	test@test.com	2	\$2400.00	DELIVERED	6/17/2026, 2:29:51 PM	Details
50aa231f...	test@test.com	2	\$2400.00	DELIVERED	5/23/2026, 3:37:04 PM	Details
24e8801a...	test@test.com	3	\$3600.00	DELIVERED	5/15/2026, 6:18:08 PM	Details
523d3ad1...	test@test.com	3	\$3600.00	DELIVERED	5/10/2026, 4:11:57 PM	Details
b3ae7e52...	test@test.com	1	\$1200.00	DELIVERED	5/10/2026, 4:06:32 PM	Details
e3ae5187e...	test@test.com	1	\$1200.00	DELIVERED	5/10/2026, 3:54:02 PM	Details
6f081f23...	test@test.com	1	\$1200.00	DELIVERED	5/10/2026, 3:51:25 PM	Details
b61ce49c...	test@test.com	1	\$1200.00	DELIVERED	5/10/2026, 3:41:44 PM	Details
3aa23164...	test@test.com	1	\$1200.00	DELIVERED	4/29/2026, 6:10:32 PM	Details
eeaa4f029...	test@test.com	3	\$3600.00	DELIVERED	4/26/2026, 5:05:38 PM	Details
02e4fcbl...	test@test.com	3	\$3600.00	CANCELLED	4/26/2026, 4:00:05 PM	Details

Εικόνα 4.10: Διαχειριστική ροή παραγγελιών.

4.3 Υλοποίηση backend υπηρεσιών

Οι υπηρεσίες υλοποιούνται ως ανεξάρτητες Spring Boot εφαρμογές. Κάθε μία εκθέτει REST API και υλοποιεί τους κανόνες που αντιστοιχούν στο domain της. Η ακόλουθη παρουσίαση περιγράφει τα ουσιώδη σημεία που προκύπτουν από την υλοποίηση και όχι γενικά χαρακτηριστικά του Spring.

4.3.1 Product Service

Το Product Service εξυπηρετεί τον κατάλογο με REST endpoints για ανάκτηση, δημιουργία, ενημέρωση και διαγραφή προϊόντων, καθώς και για κατηγορίες. Η ανάγνωση προϊόντος γίνεται με αναγνωριστικό ή SKU. Οι ενέργειες μεταβολής προστατεύονται στο backend για ADMIN. Στο επίπεδο service ελέγχονται τα templates χαρακτηριστικών κατηγορίας: υποχρεωτικό χαρακτηριστικό δεν μπορεί να παραλειφθεί και οι αριθμητικές ή επιλογικές τιμές ελέγχονται ως προς τον τύπο και τις επιτρεπτές τιμές τους.

4.3.2 Αλγόριθμος βαθμολόγησης σχετικών προϊόντων

Για την επιλογή σχετικών προϊόντων δεν χρησιμοποιείται τυχαία δειγματοληψία του καταλόγου. Σε κάθε σελίδα λεπτομέρειας, το Product Service εξετάζει τα υπόλοιπα ενεργά προϊόντα και υπολογίζει μία βαθμολογία συνάφειας. Πρόκειται για content-based και ερμηνεύσιμη προσέγγιση: η τελική κατάταξη προκύπτει από γνωστά χαρακτηριστικά του ίδιου του καταλόγου, χωρίς χρήση εξωτερικού μοντέλου μηχανικής μάθησης ή ιστορικού αγορών. Με τον τρόπο αυτό, το σύστημα μπορεί να αιτιολογεί γιατί ένα προϊόν προτείνεται στον χρήστη.

Στη διαδικασία κατάταξης, η κοινή κατηγορία αποτελεί το ισχυρότερο κριτήριο, καθώς εκφράζει την ευρύτερη επιχειρησιακή συγγένεια δύο προϊόντων. Η βαθμολογία συμπληρώνεται από την ταύτιση χαρακτηριστικών, όπως τεχνικές προδιαγραφές ή ιδιότητες που έχουν οριστεί από το πρότυπο της κατηγορίας, από κοινές ουσιαστικές λέξεις της ονομασίας και από τη σχετική εγγύτητα τιμής. Πριν από τη σύγκριση, τα κείμενα κανονικοποιούνται σε πεζά γράμματα, ενώ οι λέξεις διαχωρίζονται και απορρίπτονται πολύ σύντομα ή συνηθισμένα αγγλικά stop words. Έτσι, η σύγκριση ονόματος αποφεύγει να δίνει τεχνητή βαρύτητα σε γενικούς συνδέσμους ή σε επιφανειακές διαφορές μορφοποίησης.

Πίνακας 4.1: Παράγοντες βαθμολόγησης σχετικών προϊόντων.

Κριτήριο	Κανόνας υπολογισμού	Μέγιστη συνεισφορά
Κατηγορία	Ίδια κατηγορία με το προϊόν αναφοράς.	40 μονάδες
Χαρακτηριστικά	12 μονάδες για κάθε ταυτόσημο χαρακτηριστικό, με ανώτατο όριο.	25 μονάδες
Λέξεις ονομασίας	7 μονάδες για κάθε κοινή ουσιαστική λέξη, με ανώτατο όριο.	15 μονάδες
Εγγύτητα τιμής	Διαφορά έως 10%, 25% ή 50% αποδίδει αντίστοιχα 20, 12 ή 5 μονάδες.	20 μονάδες

Με ανώτατο θεωρητικό όριο τις εκατό μονάδες, τα αποτελέσματα ταξινομούνται κατά φθίνουσα βαθμολογία. Σε ισοβαθμία, προηγείται το προϊόν με την πιο πρόσφατη ενημέρωση και στη συνέχεια εφαρμόζεται αλφαβητική ταξινόμηση ονόματος, ώστε η συμπεριφορά του endpoint να παραμένει σταθερή και αναπαραγώγιμη. Από το αποτέλεσμα αποκλείονται το ίδιο το προϊόν αναφοράς και προϊόντα που δεν είναι ενεργά. Η υπηρεσία επιτρέπει έως δώδεκα αποτελέσματα, ενώ η React διεπαφή ζητά κατά κανόνα τέσσερις προτάσεις και εμφανίζει μαζί με κάθε μία τους λόγους αντιστοίχισης και τη βαθμολογία συνάφειας.

Η βαθμολόγηση περιγράφεται συνοπτικά από τη σχέση $S = C + \min(12m, 25) + \min(7w, 15) + P$. Στη σχέση αυτή, C είναι 40 όταν συμπίπτει η κατηγορία και μηδέν διαφορετικά, m είναι ο αριθμός κοινών ζευγών ονόματος και τιμής χαρακτηριστικού, w είναι ο αριθμός κοινών λέξεων των ονομασιών και P είναι η συνιστώσα εγγύτητας τιμής. Οι ανώτατες τιμές των τεσσάρων συνιστωσών αθροίζονται σε 100. Πρόκειται για προσθετικό δείκτη ομοιότητας που ορίζει ο κώδικας και όχι για πιθανότητα αγοράς ή στατιστικά εκτιμημένη βεβαιότητα.

Για τον υπολογισμό της συνιστώσας τιμής χρησιμοποιείται η σχετική απόσταση $d = |p_0 - p_1| / p_0$, όπου p_0 η τιμή του προϊόντος αναφοράς και p_1 η τιμή του υποψήφιου προϊόντος. Η διαίρεση γίνεται με τέσσερα δεκαδικά ψηφία και στρογγυλοποίηση HALF_UP. Απόσταση έως 0,10 δίνει 20 μονάδες, έως 0,25 δίνει 12 και έως 0,50 δίνει 5. Σε μεγαλύτερη απόσταση η συνιστώσα μηδενίζεται. Όταν η τιμή αναφοράς είναι μη θετική ή κάποια τιμή απουσιάζει, η μέθοδος επιστρέφει μηδέν για το κριτήριο αυτό. Τα όρια εφαρμόζονται διαδοχικά, συνεπώς κάθε προϊόν λαμβάνει μία μόνο βαθμολογία τιμής.

Ως αριθμητικό παράδειγμα ερμηνείας του κώδικα, έστω προϊόν αναφοράς αξίας 1.000 μονάδων και υποψήφιο αξίας 1.200, στην ίδια κατηγορία, με δύο κοινά χαρακτηριστικά και μία κοινή λέξη ονομασίας. Η σχετική απόσταση είναι 0,20, επομένως $P = 12$. Οι επιμέρους συνεισφορές είναι 40, 24, 7 και 12 και η συνολική βαθμολογία είναι 83. Το παράδειγμα είναι αναλυτικός υπολογισμός με υποθετικές εισόδους και δεν αποτελεί μέτρηση των δεδομένων του καταστήματος. Αναδεικνύει όμως τον ακριβή τρόπο με τον οποίο μια αλλαγή χαρακτηριστικού επηρεάζει το αποτέλεσμα.

Πριν συγκριθούν τα χαρακτηριστικά, τα ονόματα και οι τιμές τους μετατρέπονται σε πεζά με Locale.ROOT και αφαιρούνται τα αρχικά και τελικά κενά. Η σύμπτωση απαιτεί ίσο κανονικοποιημένο όνομα και ίση κανονικοποιημένη τιμή. Στις ονομασίες προϊόντων χρησιμοποιείται σύνολο λέξεων, επομένως η επανάληψη της ίδιας λέξης δεν αυξάνει τη βαθμολογία. Ο διαχωρισμός βασίζεται στον κανόνα $[^a-z0-9]^+$, διατηρούνται λέξεις τουλάχιστον τριών χαρακτήρων και αποκλείεται μια συγκεκριμένη λίστα συχνών αγγλικών λέξεων. Η περιγραφή αφορά αυτή την υλοποιημένη σύγκριση και όχι γλωσσική κατανόηση ή εκπαίδευση μοντέλου.

Μετά τη βαθμολόγηση, η ταξινόμηση χρησιμοποιεί κατά φθίνουσα σειρά το S. Σε ισοβαθμία προηγείται η πιο πρόσφατη ενημέρωση προϊόντος και στη συνέχεια η ονομασία χωρίς διάκριση πεζών και κεφαλαίων. Το ίδιο το προϊόν αναφοράς αποκλείεται, όπως και προϊόντα που δεν έχουν κατάσταση ACTIVE. Το ζητούμενο πλήθος περιορίζεται στο διάστημα από 1 έως 12. Εφόσον υπάρχουν υποψήφια προϊόντα, ακόμη και όσα δεν συγκεντρώνουν θετική συνιστώσα μπορούν να επιστραφούν ως εναλλακτικές του καταλόγου. Μαζί με το αποτέλεσμα

επιστρέφονται λεκτικές αιτιολογήσεις, ώστε η παρουσίαση να συνδέει τη θέση ενός προϊόντος με συγκεκριμένα γνωρίσματα.

Σε σχέση με το θεωρητικό πλαίσιο της ενότητας 1.6, η μέθοδος `scoreRelatedProduct` υλοποιεί μια σύγκριση προϊόντος προς προϊόν με χαρακτηριστικά περιεχομένου. Δεν διατηρεί προφίλ προτιμήσεων ούτε προσαρμόζει τα βάρη από ανατροφοδότηση χρηστών. Τα όρια και οι συντελεστές αποτελούν σχεδιαστικές παραμέτρους της συγκεκριμένης εφαρμογής και όχι τιμές που προκύπτουν από τα βιβλιογραφικά παραδείγματα. Έτσι, η επιστημονική πηγή εξηγεί την οικογένεια της προσέγγισης, ενώ ο κώδικας καθορίζει τον ακριβή υπολογισμό [40].

Παράλληλα, η λίστα `reasons` συνδέει την ερμηνεία με τις συνιστώσες της βαθμολογίας. Το τεκμήριο της υλοποίησης είναι ότι κάθε ενεργό κριτήριο προσθέτει τον αντίστοιχο λόγο. Το στιγμιότυπο της διεπαφής επιβεβαιώνει ότι οι λόγοι εμφανίζονται στον χρήστη. Η ποιότητα της κατάταξης και η χρησιμότητα των εξηγήσεων παραμένουν διαφορετικά ερωτήματα αξιολόγησης, σύμφωνα με τη σχετική ανασκόπηση, και δεν εξάγονται μόνο από την παρουσία των καρτών προτάσεων [42].

4.3.3 Πρότυπα κατηγοριών και έλεγχος χαρακτηριστικών

Στο επίπεδο του καταλόγου, η διαχείριση κατηγοριών λειτουργεί ως μηχανισμός ορισμού προτύπου δεδομένων για τα προϊόντα. Ο διαχειριστής δεν δημιουργεί απλώς μία ονομασία κατηγορίας, αλλά μπορεί να ορίσει ένα σύνολο χαρακτηριστικών που συνοδεύουν τα προϊόντα της. Για κάθε χαρακτηριστικό δηλώνονται το όνομά του, ο τύπος εισόδου και το αν είναι υποχρεωτικό. Οι τύποι `TEXT`, `NUMBER` και `SELECT` επιτρέπουν αντίστοιχα ελεύθερο κείμενο, αριθμητική τιμή ή επιλογή από ελεγχόμενο σύνολο επιτρεπτών τιμών. Η διεπαφή διαχείρισης κατηγοριών μετατρέπει αυτές τις ρυθμίσεις σε δυναμικό `template`, το οποίο χρησιμοποιείται κατά τη δημιουργία ή ενημέρωση προϊόντος.

Πέρα από τη διεπαφή, ο έλεγχος εφαρμόζεται εκ νέου στο `Product Service`, ώστε η εγκυρότητα να μην εξαρτάται μόνο από τη φόρμα. Η υπηρεσία απαιτεί έγκυρη υπάρχουσα κατηγορία, κανονικοποιεί τα ονόματα των χαρακτηριστικών και αποτρέπει την επανάληψη του ίδιου χαρακτηριστικού σε ένα προϊόν. Στη συνέχεια ελέγχει ότι κάθε υποχρεωτικό χαρακτηριστικό του `template` έχει δοθεί, ότι δεν έχει υποβληθεί χαρακτηριστικό εκτός του επιτρεπτού συνόλου και ότι η τιμή ακολουθεί τον δηλωμένο τύπο. Ειδικά για πεδία `SELECT`, η τιμή συγκρίνεται με τις επιτρεπτές επιλογές του `template`, ενώ για πεδία `NUMBER` επαληθεύεται ότι μπορεί να μετατραπεί σε αριθμητική τιμή. Η διαδικασία αυτή διατηρεί τον κατάλογο συνεπή και επιτρέπει στις κατηγορίες να εξελίσσονται με σαφείς κανόνες.

Η επικύρωση εκτελείται πριν από τη δημιουργία ή την ενημέρωση του προϊόντος. Αρχικά αναζητείται η κατηγορία με το όνομά της, χωρίς διάκριση πεζών και κεφαλαίων. Κατόπιν κατασκευάζεται χάρτης των υποβληθέντων χαρακτηριστικών. Κάθε εγγραφή πρέπει να περιέχει μη κενό όνομα και μη κενή τιμή, ενώ το κανονικοποιημένο όνομα λειτουργεί ως κλειδί του χάρτη. Η επανεμφάνιση του ίδιου κλειδιού απορρίπτεται, ώστε ένα χαρακτηριστικό να μην αποκτά δύο διαφορετικές τιμές μέσα στο ίδιο αίτημα.

Σε δεύτερο βήμα εξετάζονται οι ορισμοί του προτύπου. Για κάθε υποχρεωτικό χαρακτηριστικό ελέγχεται η παρουσία του στο αίτημα και για κάθε υποβληθέν χαρακτηριστικό ελέγχεται ότι υπάρχει αντίστοιχος ορισμός στην κατηγορία. Οι δύο έλεγχοι καλύπτουν διαφορετικές περιπτώσεις: ο πρώτος εντοπίζει απουσία απαιτούμενης πληροφορίας, ενώ ο δεύτερος αποτρέπει την προσθήκη πεδίου που δεν ανήκει στο επιλεγμένο πρότυπο. Έτσι, η έννοια του προτύπου περιλαμβάνει τόσο ελάχιστες απαιτήσεις όσο και επιτρεπόμενη δομή.

Ο τύπος `NUMBER` ελέγχεται μέσω κατασκευής `BigDecimal` από τη συμβολοσειρά της τιμής. Ο τύπος `SELECT` συγκρίνει την τιμή με τη λίστα επιτρεπόμενων επιλογών χωρίς διάκριση πεζών και κεφαλαίων. Για παράδειγμα, σε πρότυπο με υποχρεωτικό αριθμητικό πεδίο `RAM`, η απουσία του και η μη αριθμητική τιμή ενεργοποιούν διαφορετικούς ελέγχους. Αν το πρότυπο διαθέτει προαιρετικό πεδίο `Camera`, η παράλειψή του δεν ισοδυναμεί με παράλειψη υποχρεωτικού πεδίου. Αυτές οι περιπτώσεις αναλύονται από τις συνθήκες της μεθόδου και δεν παρουσιάζονται ως πρόσθετες εκτελεσμένες δοκιμές.

Στη διεπαφή διαχείρισης, η επιλογή κατηγορίας συνδέει τη φόρμα προϊόντος με τα διαθέσιμα χαρακτηριστικά. Στο `backend`, ο ίδιος κανόνας εφαρμόζεται ανεξάρτητα από τον τρόπο με τον

οποίο σχηματίστηκε το αίτημα. Η συνεργασία των δύο επιπέδων έχει διαφορετικούς σκοπούς: η φόρμα καθοδηγεί τον διαχειριστή, ενώ η υπηρεσία καθορίζει τη δομή που μπορεί να αποθηκευτεί. Παράλληλα, η ύπαρξη σταθερών ονομάτων χαρακτηριστικών υποστηρίζει τη συγκρίσιμη αναπαράσταση που χρησιμοποιεί ο αλγόριθμος σχετικών προϊόντων.

4.3.4 Cart Service

Για την αποθήκευση ενός καλαθιού ανά χρήστη, το Cart Service χρησιμοποιεί Redis. Το user identifier αντλείται από το subject του JWT, άρα το API δεν εμπιστεύεται user id που δηλώνεται στο σώμα αιτήματος. Κατά την προσθήκη, ενημέρωση ποσότητας ή αφαίρεση είδους, δημιουργείται ή ανανεώνεται η δέσμευση αποθέματος. Για σταθερή συσχέτιση, ο κώδικας δημιουργεί reservation UUID από το user id. Πριν από νέο συγχρονισμό, τυχόν προηγούμενη δέσμευση αποδεσμεύεται και η τρέχουσα κατάσταση του καλαθιού δεσμεύεται εκ νέου.

4.3.5 Συγχρονισμός καλαθιού και προσωρινή δέσμευση αποθέματος

Ως προσωρινή επιχειρησιακή κατάσταση, το καλάθι επηρεάζει άμεσα τη διαθέσιμη ποσότητα. Για τον λόγο αυτό, το Cart Service δεν αντιμετωπίζει την προσθήκη ή την αλλαγή ποσότητας ως απλή εγγραφή στο Redis. Σε κάθε μεταβολή δημιουργεί από το αναγνωριστικό του χρήστη ένα σταθερό UUID δέσμευσης και το χρησιμοποιεί ως κοινό κλειδί συσχέτισης με το Inventory Service. Η επιλογή deterministic UUID επιτρέπει στο ίδιο καλάθι να ανανεώνει την ίδια λογική δέσμευση χωρίς να απαιτείται πρόσθετη πληροφορία από τον client.

Κατά τον συγχρονισμό ακολουθούνται δύο βήματα. Αρχικά αποστέλλεται αίτημα αποδέσμευσης για την προηγούμενη κατάσταση του καλαθιού. Στη συνέχεια, εφόσον υπάρχουν είδη, αποστέλλεται νέο αίτημα reserve που περιλαμβάνει το SKU και την ποσότητα κάθε γραμμής. Το Inventory Service υπολογίζει για κάθε SKU τη διαθέσιμη ποσότητα αφαιρώντας τις ενεργές δεσμεύσεις από τη συνολική ποσότητα. Όταν όλες οι ζητούμενες ποσότητες είναι διαθέσιμες, δημιουργεί εγγραφές reservation με διάρκεια δεκαπέντε λεπτών. Με αυτό τον τρόπο, το σύστημα συνδέει την εμπειρία του καλαθιού με μια χρονικά περιορισμένη δέσμευση επιχειρησιακών πόρων.

Για τα μόνιμα δεδομένα προϊόντων, αποθέματος και παραγγελιών χρησιμοποιούνται σχεσιακές οντότητες και repositories, ενώ η προσωρινή κατάσταση καλαθιού διατηρείται στο Redis. Η σχεσιακή μοντελοποίηση και η επιλογή αποθηκευτικού μοντέλου σε σχέση με τις απαιτήσεις πρόσβασης παρέχουν το θεωρητικό πλαίσιο αυτού του διαχωρισμού [13, 11]. Η αναφορά στα συγκεκριμένα εργαλεία βασίζεται στην υλοποίηση των υπηρεσιών και δεν εξομοιώνει τα διαφορετικά συστήματα αποθήκευσης. Η σύνδεση του μοντέλου εφαρμογής με τις λειτουργίες αποθήκευσης τεκμηριώνεται στα Spring Data JPA και Spring Data Redis [16, 47].

Για την ερμηνεία της δέσμευσης διακρίνονται το συνολικό απόθεμα T , οι ενεργές δεσμεύσεις R και το διαθέσιμο απόθεμα A , με $A = T - R$. Αν ένα είδος έχει $T = 10$ και δεσμεύονται 2 τεμάχια, το διαθέσιμο γίνεται 8 χωρίς να μεταβληθεί ακόμη το συνολικό απόθεμα. Με την επιβεβαίωση αφαιρούνται τα 2 τεμάχια από το T και διαγράφεται η αντίστοιχη δέσμευση. Όταν δεν υπάρχουν άλλες κρατήσεις, το σύστημα εμφανίζει πλέον $T = 8$ και $A = 8$. Η ίδια αριθμητική τιμή του διαθέσιμου πριν και μετά την επιβεβαίωση δεν σημαίνει ίδια εσωτερική κατάσταση.

Η απελευθέρωση έχει διαφορετική σημασιολογία. Διαγράφει τις εγγραφές της κράτησης χωρίς να μειώνει το συνολικό απόθεμα. Στο προηγούμενο παράδειγμα, απελευθέρωση πριν από επιβεβαίωση επαναφέρει το διαθέσιμο από 8 σε 10. Η αντιπαράθεση των δύο ενεργειών εξηγεί γιατί το checkout καλεί confirmReservation, ενώ η αφαίρεση περιεχομένου καλαθιού χρησιμοποιεί release και νέο συγχρονισμό. Δεν πρόκειται για δύο ονόματα της ίδιας μεταβολής αλλά για διαφορετικά στάδια του κύκλου ζωής του αποθέματος.

Κατά την ενημέρωση καλαθιού αποστέλλεται στο Inventory Service ολόκληρη η τρέχουσα λίστα SKU και ποσοτήτων. Η υπηρεσία ελέγχει πρώτα τη διαθεσιμότητα όλων των ζητούμενων ειδών και κατόπιν δημιουργεί τις εγγραφές δέσμευσης, με χρόνο λήξης δεκαπέντε λεπτά μετά τη δημιουργία τους. Το σύνολο των γραμμών συσχετίζεται με το ίδιο αναγνωριστικό κράτησης. Η ομαδοποίηση επιτρέπει στην επιβεβαίωση και στην απελευθέρωση να αναφέρονται στη συνολική επιλογή του χρήστη αντί να απαιτούν διαφορετική εντολή για κάθε γραμμή.

Για την προσωρινή κατάσταση καλαθιού, το Spring Data Redis συνδέει το μοντέλο της εφαρμογής με αποθήκευση στο Redis [47]. Η οντότητα `Cart` δηλώνεται στον κώδικα με `@RedisHash` και χρησιμοποιεί αναγνωριστικό για την ανάκτηση του αντίστοιχου καλαθιού. Η τεκμηρίωση Redis περιγράφει τα hashes ως δομές πεδίων και τιμών, οι οποίες διαφέρουν από τη σχεσιακή οργάνωση των πινάκων PostgreSQL [48]. Η επιλογή αποθήκευσης ερμηνεύεται με βάση τις ανάγκες του καλαθιού και δεν αποτελεί εφαρμογή μηχανισμών καταμετρημένης αναπαραγωγής ή επίλυσης συγκρούσεων.

4.3.6 Inventory Service

Στην υπηρεσία Inventory διατηρούνται τα στοιχεία αποθέματος και οι κανόνες δέσμευσης. Για κάθε προϊόν η διαθέσιμη ποσότητα υπολογίζεται αφαιρώντας τις ενεργές δεσμεύσεις από τη συνολική ποσότητα. Η κράτηση διαρκεί δεκαπέντε λεπτά, αποτρέποντας μόνιμη δέσμευση από εγκαταλελειμμένο καλάθι. Η επιβεβαίωση κράτησης μειώνει οριστικά το απόθεμα και αφαιρεί τη δέσμευση. Η αποδέσμευση διαγράφει μόνο τη δέσμευση, ενώ δημιουργία inventory item και restock προστατεύονται για ADMIN.

4.3.7 Order Service

Στον πυρήνα της ροής checkout βρίσκεται το Order Service. Αρχικά εξάγει το email του χρήστη από claims με προτεραιότητα email, preferred_username και subject. Έπειτα ελέγχει αν το idempotency key έχει ήδη συνδεθεί με παραγγελία του πελάτη. Στην περίπτωση επανάλωσης επιστρέφει την υπάρχουσα παραγγελία. Διαφορετικά, ανακτά το καλάθι, επιβεβαιώνει την inventory reservation, δημιουργεί UUID order number, αποθηκεύει τις γραμμές παραγγελίας και καταγράφει την αρχική κατάσταση PENDING.

Με την επιτυχή αποθήκευση εκκαθαρίζεται το καλάθι και δημοσιεύεται το OrderPlacedEvent. Η κλήση επιβεβαίωσης αποθέματος προστατεύεται με Resilience4j circuit breaker με όνομα inventory. Το fallback μετατρέπει την προσωρινή μη διαθεσιμότητα σε ServiceUnavailableException, ώστε το API να δίνει ελεγχόμενη απόκριση αντί να παραμένει σε αβέβαιη κατάσταση. Επιπλέον, το service εφαρμόζει τους κανόνες μετάβασης κατάστασης και τις διακρίσεις δικαιωμάτων πελάτη/διαχειριστή που παρουσιάστηκαν στο Κεφάλαιο 3.

4.3.8 Idempotency και οριστικοποίηση checkout

Κατά το checkout εφαρμόζεται μία διακριτή ακολουθία οριστικοποίησης και όχι απλή μεταφορά του περιεχομένου του καλαθιού στη βάση παραγγελιών. Η React εφαρμογή παράγει μοναδικό idempotency key με UUID κατά την ενέργεια checkout και το αποστέλλει στην HTTP κεφαλίδα του αιτήματος. Το Order Service κανονικοποιεί το κλειδί και αναζητά προηγούμενη παραγγελία με τον ίδιο συνδυασμό πελατειακού email και idempotency key. Αν υπάρχει ήδη καταχώριση, επιστρέφεται η ίδια παραγγελία, χωρίς νέα επιβεβαίωση αποθέματος και χωρίς δεύτερη δημοσίευση επιχειρησιακού γεγονότος.

Για κάθε νέο αίτημα, το Order Service ανακτά το καλάθι του ταυτοποιημένου χρήστη και απορρίπτει τη διαδικασία αν δεν περιέχει γραμμές. Στη συνέχεια υπολογίζει το ίδιο reservation identifier που χρησιμοποίησε το Cart Service και ζητά από το Inventory Service την επιβεβαίωσή του. Η επιβεβαίωση μετατρέπει την προσωρινή δέσμευση σε οριστική μείωση του αποθέματος και αφαιρεί τη σχετική reservation. Μόνο μετά από αυτό το βήμα δημιουργείται η παραγγελία με νέο UUID order number, αποθηκεύονται οι γραμμές με το SKU, την ονομασία, την τιμή και την ποσότητα, και καταγράφεται η αρχική κατάσταση PENDING στο ιστορικό. Η εκκαθάριση του καλαθιού και η δημοσίευση του γεγονότος παραγγελίας ακολουθούν την επιτυχή αποθήκευση, ολοκληρώνοντας τη ροή checkout.

Στην πλευρά React, η έννοια της προσπάθειας checkout αποθηκεύεται στο attemptRef και περιλαμβάνει ένα αποτύπωμα του καλαθιού και ένα κλειδί. Το αποτύπωμα παράγεται με JSON.stringify από τις τιμές skuCode, quantity και price κάθε γραμμής. Όταν παραμένει ίδιο, επαναχρησιμοποιείται το κλειδί της προσπάθειας. Όταν διαφέρει, δημιουργείται νέο κλειδί μέσω crypto.randomUUID, με εναλλακτική παραγωγή βασισμένη στον χρόνο και σε τυχαία τιμή. Η

συμπεριφορά αυτή συνδέει την ταυτότητα του αιτήματος με το συγκεκριμένο περιεχόμενο που επιχειρεί να υποβάλει η σελίδα.

Δύο διαφορετικοί μηχανισμοί συνεργάζονται χωρίς να ταυτίζονται. Το submittingRef και η κατάσταση isSubmitting αφορούν την ενεργή υποβολή στη διεπαφή και την αποφυγή επανειλημμένου χειρισμού του ίδιου πατήματος. Το idempotency key μεταφέρεται στην υπηρεσία και χρησιμοποιείται για αναζήτηση ήδη αποθηκευμένης παραγγελίας. Η πρώτη λειτουργία αφορά τη συμπεριφορά του component, ενώ η δεύτερη αφορά την αναγνώριση αιτήματος στο επιχειρησιακό επίπεδο. Η παρουσία και των δύο αποσαφηνίζει γιατί μια ανενεργή εντολή στην οθόνη δεν αντικαθιστά τη λογική της υπηρεσίας.

Στο Order Service η αναζήτηση γίνεται με συνδυασμό email πελάτη και κανονικοποιημένου κλειδιού. Αν βρεθεί παραγγελία, επιστρέφεται η αποθηκευμένη απόκρισή της πριν ανακτηθεί το καλάθι και πριν επιβεβαιωθεί δέσμευση. Αν δεν βρεθεί, ακολουθείται η κανονική ακολουθία δημιουργίας. Επομένως, η ιδιοδυναμία που περιγράφεται εδώ αφορά αυτή τη συγκεκριμένη διαδρομή αναγνώρισης επαναλαμβανόμενης προσπάθειας. Δεν ταυτίζεται με γενική ιδιότητα κάθε HTTP POST ούτε αποτελεί μέτρηση συμπεριφοράς υπό ταυτόχρονο φορτίο.

Μετά την επιτυχή απόκριση, η διεπαφή διατηρεί αντίγραφο των ειδών που παραγγέλθηκαν και μεταβαίνει στη σελίδα επιβεβαίωσης, μεταφέροντας το αντικείμενο παραγγελίας και τα σύνολα. Η πληροφορία αυτή εξυπηρετεί την άμεση παρουσίαση του αποτελέσματος. Για τη μεταγενέστερη παρακολούθηση, το ιστορικό παραγγελιών ανακτά τις αποθηκευμένες εγγραφές της υπηρεσίας. Έτσι διακρίνεται η στιγμιαία κατάσταση πλοήγησης του browser από το μόνιμο επιχειρησιακό ιστορικό.

4.3.9 Notification Service

Με ρόλο αποκλειστικά την ενημέρωση μετά την υποβολή παραγγελίας, το Notification Service δηλώνει topic exchange orderExchange, queue notificationQueue και routing key order.placed. Ο listener λαμβάνει το JSON event που δημοσιεύει το Order Service, μετατρέπει το περιεχόμενο σε ειδοποίηση και αποστέλλει email όταν η αποστολή είναι ενεργοποιημένη στις ρυθμίσεις. Με αυτόν τον τρόπο, η δημιουργία παραγγελίας δεν περιμένει τον SMTP πάροχο και η λειτουργία ειδοποίησης μπορεί να εξελιχθεί ανεξάρτητα, για παράδειγμα με πρόσθετα κανάλια επικοινωνίας.

4.3.10 Ροή ειδοποίησης βάσει γεγονότος

Για την επικοινωνία μετά την υποβολή παραγγελίας εφαρμόζεται το event-driven πρότυπο. Μετά την επιτυχή καταχώριση, το Order Service δημιουργεί ένα OrderPlacedEvent που περιέχει τον αριθμό της παραγγελίας και το email του πελάτη. Το γεγονός μετατρέπεται σε JSON από κοινό Jackson message converter και δημοσιεύεται στο RabbitMQ topic exchange orderExchange με routing key order.placed. Η queue notificationQueue συνδέεται με το exchange μέσω του ίδιου routing key, ώστε το Notification Service να καταναλώνει μόνο τα μηνύματα που αφορούν την τοποθέτηση παραγγελίας.

Κάθε μήνυμα της notificationQueue ενεργοποιεί τον Rabbit listener της υπηρεσίας. Το θέμα και το σώμα του email σχηματίζονται από παραμετροποιησιμα templates, στα οποία τα placeholders {orderNumber} και {email} αντικαθίστανται με τα δεδομένα του γεγονότος. Υποστηρίζεται αποστολή απλού κειμένου ή HTML, καθώς και προαιρετικές ρυθμίσεις αποστολέα και διεύθυνσης απάντησης. Η ενεργοποίηση email και η συμπεριφορά σε αποτυχία ρυθμίζονται εξωτερικά από properties, ενώ τα χρονικά όρια σύνδεσης, ανάγνωσης και εγγραφής του SMTP client ορίζονται επίσης στο περιβάλλον εκτέλεσης. Η επιλογή αυτή διατηρεί τον μηχανισμό ειδοποίησης διακριτό από την επιχειρησιακή λογική δημιουργίας παραγγελίας και επιτρέπει την προσαρμογή του χωρίς μεταβολές στο Order Service.

4.4 Χειρισμός σφαλμάτων και διασύνδεση υπηρεσιών

Η υλοποίηση διαχειρίζεται αναμενόμενες επιχειρησιακές αποτυχίες κοντά στο domain που τις παράγει. Ενδεικτικά, κενό καλάθι, ανεπαρκής διαθεσιμότητα, μη έγκυρη μετάβαση κατάστασης,

μη επιτρεπτή ακύρωση και μη διαθέσιμο inventory επιστρέφονται ως κατανοητές αποκρίσεις. Το frontend καταναλώνει τα σφάλματα αυτά και ενημερώνει τον χρήστη χωρίς να υποθέτει ότι κάθε αστοχία είναι τεχνικό πρόβλημα διεπαφής.

- REST και OpenFeign χρησιμοποιούνται για τις κλήσεις που απαιτούν άμεση απάντηση, όπως ανάκτηση καλαθιού και επιβεβαίωση δέσμευσης.
- RabbitMQ χρησιμοποιείται για την ασύγχρονη συνέπεια της ειδοποίησης μετά από επιτυχή παραγγελία.
- Το idempotency key μειώνει τον κίνδυνο διπλής καταχώρισης όταν ο client επαναλαμβάνει checkout request.
- Ο circuit breaker περιορίζει την επίδραση προσωρινής αποτυχίας του Inventory Service στην εμπειρία checkout.
- Το ιστορικό κατάστασης παραγγελίας δημιουργεί audit trail για κάθε επιχειρησιακή μετάβαση.

4.5 Μοντέλο μεταβάσεων και ιστορικό παραγγελίας

Η κλάση OrderStatusWorkflow συγκεντρώνει τις επιτρεπόμενες μεταβάσεις σε χάρτη από την τρέχουσα κατάσταση προς λίστα επόμενων καταστάσεων. Από PENDING επιτρέπονται CONFIRMED, CANCELLED και FAILED. Από CONFIRMED επιτρέπονται PROCESSING και CANCELLED, ενώ από PROCESSING επιτρέπονται PACKED και CANCELLED. Τα στάδια PACKED και SHIPPED συνεχίζουν αντίστοιχα προς SHIPPED και DELIVERED. Οι καταστάσεις CANCELLED και FAILED δεν διαθέτουν εξερχόμενες μεταβάσεις. Ο χάρτης αυτός αποτελεί εκτελέσιμη αναπαράσταση του διαγράμματος καταστάσεων και όχι απλή απαρίθμηση λεκτικών ετικετών.

Για την κατάσταση DELIVERED υπάρχει δυνατότητα μετάβασης σε CANCELLED, η οποία συμπληρώνεται από τον χρονικό κανόνα της υπηρεσίας. Το χρονικό παράθυρο ορίζεται σε δεκατέσσερις ημέρες και υπολογίζεται σε σχέση με τον χρόνο παράδοσης. Στην απόκριση επιστρέφονται οι επιτρεπόμενες επόμενες καταστάσεις, η δυνατότητα ακύρωσης από τον πελάτη και το σχετικό χρονικό όριο, όταν είναι διαθέσιμος χρόνος παράδοσης. Επομένως, η διεπαφή δεν χρειάζεται να παρουσιάζει την ίδια λίστα ενεργειών για κάθε παραγγελία ανεξαρτήτως κατάστασης.

Κάθε μεταβολή συνδέεται με εγγραφή ιστορικού που περιλαμβάνει προηγούμενη και νέα κατάσταση, χρόνο, υπεύθυνο μεταβολής και σημείωση. Κατά την αρχική υποβολή καταγράφεται μετάβαση προς PENDING χωρίς προηγούμενη κατάσταση. Κατά τη διαχειριστική ενημέρωση διατηρείται η προηγούμενη τιμή πριν από την αλλαγή και χρησιμοποιείται η ταυτότητα του συνδεδεμένου χρήστη. Με αυτό τον τρόπο η τρέχουσα κατάσταση απαντά στο ερώτημα «πού βρίσκεται τώρα η παραγγελία», ενώ το ιστορικό απαντά στο ερώτημα «πώς έφθασε εκεί».

Η διαφορά ανάμεσα στις δύο αναπαραστάσεις είναι σημαντική για την τεκμηρίωση. Ένα στιγμιότυπο που εμφανίζει CANCELLED επιβεβαιώνει την τελική εμφανιζόμενη κατάσταση, αλλά δεν καταγράφει από μόνο του κάθε προηγούμενη μετάβαση. Αντίθετα, τα διαδοχικά στιγμιότυπα της ίδιας παραγγελίας πριν και μετά την ακύρωση επιτρέπουν στενότερο συμπέρασμα για τη συγκεκριμένη ενέργεια. Η αρχιτεκτονική του ιστορικού εξηγεί τη δυνατότητα διατήρησης των μεταβολών, ενώ η αξιολόγηση του Κεφαλαίου 6 περιορίζεται στα αποτελέσματα που τεκμηριώνονται από τα διαθέσιμα δεδομένα.

4.6 Διαχείριση κατάστασης και επικοινωνίας στο frontend

Η σελίδα checkout διακρίνει τη φόρτωση του καλαθιού, την τροποποίησή του και την υποβολή παραγγελίας. Οι ενδείξεις isLoading, isMutating και isSubmitting δεν περιγράφουν την ίδια εργασία. Κατά τη φόρτωση δεν είναι ακόμη διαθέσιμη επιβεβαιωμένη εικόνα του καλαθιού. Κατά την τροποποίηση βρίσκεται σε εξέλιξη ενέργεια που μπορεί να αλλάξει ποσότητες. Κατά την υποβολή αναμένεται η απόκριση δημιουργίας παραγγελίας. Ο έλεγχος των καταστάσεων πριν από την εντολή handlePlaceOrder συνδέει τη διαθεσιμότητα της ενέργειας με το πραγματικό στάδιο επεξεργασίας.

Τα σύνολα της σελίδας υπολογίζονται από τις γραμμές του καλαθιού με `useMemo`. Το `subtotal` προκύπτει ως άθροισμα τιμής επί ποσότητας και το `itemCount` ως άθροισμα ποσοτήτων. Στην παρούσα ροή το `total` ισούται με το `subtotal`, αφού ο συγκεκριμένος υπολογισμός δεν προσθέτει μεταφορικά ή φόρους. Η παρατήρηση αυτή εξηγεί τη συμφωνία των πεδίων που εμφανίζονται στις εικόνες καλαθιού και ανασκόπησης. Η περιγραφή των χρηματικών ποσών ακολουθεί τη λειτουργία που έχει υλοποιηθεί και όχι υποθετικά στάδια χρέωσης.

Σε περίπτωση αποτυχίας, η σελίδα δεν μεταβαίνει στην επιβεβαίωση. Η συνάρτηση `getCheckoutErrorMessage` εξετάζει πρώτα το περιεχόμενο της απόκρισης για πεδίο `message`, απλό κείμενο ή πεδίο `error`. Αν δεν υπάρχει διαθέσιμο μήνυμα, χρησιμοποιεί ειδικές διατυπώσεις για κωδικούς 400 και 500 και γενική διατύπωση για τις υπόλοιπες περιπτώσεις. Το `finally` επαναφέρει την ένδειξη υποβολής ανεξάρτητα από την έκβαση. Με αυτό τον τρόπο, η ερμηνεία της απόκρισης, η παρουσίαση ειδοποίησης και η κατάσταση των χειριστηρίων αποτελούν διαφορετικές, αλλά συντονισμένες ευθύνες του `component`.

Στο κοινό αντικείμενο `Axios`, ο `request interceptor` προσθέτει το διαθέσιμο `access token` στην κεφαλίδα `Authorization`. Ο `response interceptor` αναγνωρίζει απόκριση 401 και επιχειρεί ανανέωση, εφόσον το αίτημα δεν έχει ήδη σημειωθεί ως επανάληψη. Η μεταβλητή `isRefreshing` και η λίστα `failedQueue` συντονίζουν αιτήματα που αναμένουν την ίδια διαδικασία ανανέωσης. Όταν επιστραφεί νέο `token`, ενημερώνονται τα αιτήματα και επανεκτελούνται μέσω του ίδιου αντικειμένου `API`. Αν η ανανέωση αποτύχει, αφαιρούνται τα αποθηκευμένα `tokens` και γίνεται μετάβαση στη σύνδεση.

Η διαχείριση αυτή είναι διακριτή από την επιχειρησιακή ιδιοδυναμία του `checkout`. Η ανανέωση `token` αφορά τη δυνατότητα αυθεντικοποιημένης επικοινωνίας, ενώ το `idempotency key` αφορά την αναγνώριση της ίδιας προσπάθειας παραγγελίας. Παρότι και οι δύο μηχανισμοί μπορεί να οδηγήσουν σε επανάληψη αιτήματος, το κριτήριο ενεργοποίησης και το αντικείμενο ελέγχου τους διαφέρουν. Η περιγραφή τους σε διαφορετικά επίπεδα αποσαφηνίζει τη συνεργασία `React`, `Axios` και `Spring Security`.

4.7 Συγχρονισμός κοινής κατάστασης καλαθιού

Το `CartProvider` αποτελεί το κοινό σημείο διαχείρισης καλαθιού για τα `components` του `frontend`. Η συνάρτηση `mapCartItems` μετατρέπει την απόκριση της υπηρεσίας σε μορφή κατάλληλη για τη διεπαφή: το `skuCode` χρησιμοποιείται ως `id`, το `productName` ως `name` και η τιμή και ποσότητα μετατρέπονται σε αριθμητικές τιμές. Η προσαρμογή αυτή συγκεντρώνει τη μετάφραση του συμβολαίου `API` σε ένα σημείο, ώστε οι σελίδες να μη χρειάζεται να επαναλαμβάνουν διαφορετικούς μετασχηματισμούς για τα ίδια δεδομένα.

Κατά την προσθήκη, το `payload` περιλαμβάνει `skuCode`, `quantity`, `price` και `productName`. Η τοπική λίστα αντικαθίσταται από την ενημερωμένη απόκριση της υπηρεσίας, ενώ εμφανίζεται μήνυμα ολοκλήρωσης. Κατά την αφαίρεση, η τοπική μεταβολή ακολουθεί την ολοκλήρωση της απομακρυσμένης ενέργειας. Η εφαρμογή διαχωρίζει επομένως την πρόθεση του χρήστη από την ενημέρωση της κοινής κατάστασης και διατηρεί ξεχωριστές διαδρομές για επιτυχία και αποτυχία.

Πίνακας 4.2: Καταστάσεις του `CartProvider` και επίδραση στη διεπαφή.

Μεταβλητή	Πότε μεταβάλλεται	Χρήση
<code>cartItems</code>	Μετά τη φόρτωση ή την επιτυχή ενέργεια καλαθιού.	Παρέχει τα δεδομένα για γραμμές, πλήθος και χρηματικά σύνολα.
<code>isLoading</code>	Κατά την ανάκτηση του καλαθιού.	Διακρίνει την αναμονή από ένα πραγματικά κενό καλάθι.
<code>pendingChanges</code>	Αυξάνεται πριν από μεταβολή και μειώνεται στο <code>finally</code> .	Μετρά ενεργές ενέργειες και παράγει την ένδειξη <code>isMutating</code> .
<code>error</code>	Σε αποτυχία φόρτωσης ή μεταβολής, με εκκαθάριση στην έναρξη νέας προσπάθειας.	Συνδέει την κατάσταση της λειτουργίας με την παρουσίαση μηνύματος και τον έλεγχο <code>checkout</code> .

Μεταβλητή	Πότε μεταβάλλεται	Χρήση
loadCartRequest Ref	Περιέχει το εκκρεμές promise ανάκτησης και μηδενίζεται στην ολοκλήρωση.	Επιτρέπει σε επαναλαμβανόμενες κλήσεις φόρτωσης να αναφερθούν στην ήδη εκτελούμενη ανάκτηση.

Ο μετρητής `pendingChanges` επιτρέπει να εκφραστεί αν υπάρχει τουλάχιστον μία ενεργή μεταβολή. Η αύξηση πριν από την κλήση και η μείωση στο `finally` εξασφαλίζουν ότι η ολοκλήρωση της συγκεκριμένης προσπάθειας αποτυπώνεται είτε η απόκριση είναι επιτυχής είτε αποτυχημένη. Η παραγόμενη τιμή `isMutating` χρησιμοποιείται από το `checkout` ως ένδειξη ότι το περιεχόμενο δεν πρέπει ακόμη να θεωρηθεί σταθερό για υποβολή. Πρόκειται για συντονισμό κατάστασης διεπαφής και όχι για αντικατάσταση του συγχρονισμού καλαθιού με απόθεμα στο backend.

Στην αλλαγή ποσότητας, μη θετική τιμή δρομολογείται στη διαδικασία αφαίρεσης. Για θετική τιμή εκτελείται η αντίστοιχη κλήση API και ενημερώνεται η γραμμή στη `local state`. Αν προκύψει σφάλμα, καλείται εκ νέου `loadCart` ώστε η διεπαφή να ανακτήσει την εικόνα της υπηρεσίας. Η διαδρομή αυτή εξηγεί γιατί η φόρτωση καλαθιού χρησιμοποιείται τόσο κατά την αρχική είσοδο όσο και μετά από αποτυχημένη μεταβολή: στη δεύτερη περίπτωση λειτουργεί ως επανασυσχέτιση της παρούσας με την απομακρυσμένη κατάσταση.

Η `clearCart` του `context` εκκαθαρίζει μόνο τη λίστα της διεπαφής και την ένδειξη σφάλματος. Στη ροή επιτυχούς υποβολής, η εκκαθάριση του αποθηκευμένου καλαθιού έχει ήδη ζητηθεί από το `Order Service` μέσω του `CartClient`. Οι δύο λειτουργίες έχουν το ίδιο λεκτικό νόημα για τον χρήστη αλλά διαφορετικό τεχνικό πεδίο. Η πρώτη καθαρίζει αυτό που εμφανίζει το `React`, ενώ η δεύτερη εφαρμόζει την ενέργεια στην υπηρεσία καλαθιού και στην αντίστοιχη κράτηση. Η διάκριση είναι απαραίτητη για την ακριβή εξήγηση της ακολουθίας.

4.8 Χρηματικά ποσά και ανακατασκευή αποκρίσεων

Η υπηρεσία παραγγελιών υπολογίζει το σύνολο κάθε γραμμής με `BigDecimal`, πολλαπλασιάζοντας την τιμή μονάδας με την ποσότητα. Το αποτέλεσμα προσαρμόζεται σε δύο δεκαδικά ψηφία με `HALF_UP`. Το `subtotal` προκύπτει ως άθροισμα των επιμέρους συνόλων και προσαρμόζεται επίσης σε δύο δεκαδικά. Η απόκριση μεταφέρει τόσο τα δεδομένα των γραμμών όσο και τα υπολογισμένα ποσά. Στην παρούσα υλοποίηση τα πεδία `subtotal` και `total` της απόκρισης λαμβάνουν την ίδια τιμή.

Για παράδειγμα, γραμμή με τιμή 12,35 και ποσότητα 3 δίνει σύνολο 37,05. Αν υπάρχει δεύτερη γραμμή τιμής 8,10 και ποσότητας 2, προστίθενται 16,20 και προκύπτει `subtotal` 53,25. Το παράδειγμα είναι αριθμητική ερμηνεία των πράξεων της μεθόδου και δεν παρουσιάζεται ως νέα παραγγελία του περιβάλλοντος δοκιμών. Καθιστά όμως σαφές ποια ποσότητα αφορά κάθε χρηματικό πεδίο και επιτρέπει στον αναγνώστη να ελέγξει τη σχέση των τιμών χωρίς να εξετάσει ολόκληρο το `frontend`.

Η `mapToOrderResponse` δεν αντιγράφει μόνο απλά γνωρίσματα. Συνθέτει τις γραμμές, τα σύνολα, το ιστορικό, τις επιτρεπόμενες επόμενες καταστάσεις και την επιλεξιμότητα ακύρωσης. Έτσι, η απόκριση αποτελεί προβολή της παραγγελίας για τις ανάγκες της εφαρμογής και όχι άμεση δημοσίευση της οντότητας JPA. Ο διαχωρισμός επιτρέπει να εκφράζονται υπολογισμένες πληροφορίες χωρίς να απαιτείται να υπάρχουν όλες ως ανεξάρτητες στήλες της βάσης.

Για παλαιές εγγραφές χωρίς αποθηκευμένο ιστορικό, η `inferredLegacyHistory` δημιουργεί περιγραφική προβολή από τους διαθέσιμους χρόνους και την τρέχουσα κατάσταση. Οι σημειώσεις χαρακτηρίζουν ρητά τις εγγραφές ως συναγόμενες από παλαιά δεδομένα. Η λειτουργία δεν ανακατασκευάζει ανύπαρκτα ενδιάμεσα γεγονότα. Διατηρεί την παρουσίαση αρχικής και τρέχουσας κατάστασης με διαφορετική προέλευση από τις κανονικές εγγραφές `audit`, επιτρέποντας να διαβαστούν οι παλαιότερες παραγγελίες μέσα στο ίδιο σχήμα απόκρισης.

4.9 Σύνοψη υλοποίησης

Η υλοποίηση συνδέει τις απαιτήσεις του Κεφαλαίου 2 με τις αρχιτεκτονικές αποφάσεις του Κεφαλαίου 3. Η `React` εφαρμογή προσφέρει συνεπή διεπαφή για πελάτη και διαχειριστή, ενώ οι Ανάπτυξη ενός Συστήματος Διαχείρισης Παραγγελιών (`Order Management System`) βασισμένου σε `Microservices` με `Spring Boot` και `Spring Cloud`

υπηρεσίες Spring Boot διαχωρίζουν catalog, cart, inventory, order και notification. Η επόμενη ενότητα θα αναλύσει ειδικότερα τα στοιχεία Spring Cloud και την υποδομή που υποστηρίζει τη συνεργασία αυτών των υπηρεσιών.

Από πλευράς τεκμηρίωσης λογισμικού, η υλοποίηση πρέπει να διαβάζεται ως ένα σύνολο από συνεργαζόμενες ευθύνες και όχι ως ανεξάρτητες τεχνολογικές επιλογές. Το React frontend συγκεντρώνει την αλληλεπίδραση του χρήστη, αλλά δεν αποτελεί τον κάτοχο των επιχειρησιακών κανόνων. Τα REST endpoints ορίζουν το δημόσιο συμβόλαιο των υπηρεσιών, τα service classes εφαρμόζουν τους κανόνες εγκυρότητας και μεταβάσεων, ενώ η υποδομή Spring Cloud και οι εξωτερικές εξαρτήσεις παρέχουν το περιβάλλον επικοινωνίας. Η διάκριση αυτή επιτρέπει κάθε αλλαγή να αξιολογείται ως προς το επίπεδο όπου πραγματικά ανήκει: παρουσίαση, επιχειρησιακός κανόνας, αποθήκευση ή διαλειτουργικότητα.

Η ύπαρξη ανεξάρτητων εφαρμογών απαιτεί σαφή διαχείριση παραμέτρων εκτέλεσης. Στο αποθετήριο, οι θύρες, τα ονόματα υπηρεσιών, οι συνδέσεις βάσεων δεδομένων και οι ρυθμίσεις broker και ταυτότητας προσδιορίζονται σε configuration. Το Docker Compose εκκινεί τις τοπικές υποδομές. Τα credentials και τα tokens δεν περιλαμβάνονται στην τεκμηρίωση ούτε στις εικόνες της εργασίας. Η περιγραφή αυτή αφορά την οργάνωση του συγκεκριμένου περιβάλλοντος και όχι αξιολόγηση ασφάλειας ή απόδοσης.

5. Υποδομή Spring Cloud και Κατανεμημένες Υπηρεσίες

5.1 Ρόλος της υποδομής Spring Cloud

Η διάσπαση της εφαρμογής σε υπηρεσίες δημιουργεί ανάγκες που δεν εμφανίζονται με την ίδια ένταση σε μία μονολιθική εφαρμογή: δυναμική ανακάλυψη δικτυακών προορισμών, ενιαία δρομολόγηση, έλεγχος αποτυχιών απομακρυσμένων κλήσεων, παρατήρηση της διαδρομής ενός αιτήματος και ασφαλής μετάδοση ταυτότητας. Το Spring Cloud αποτελεί το υποστηρικτικό επίπεδο με το οποίο η παρούσα υλοποίηση αντιμετωπίζει αυτές τις ανάγκες. Δεν αντικαθιστά τους επιχειρησιακούς κανόνες των υπηρεσιών, αλλά παρέχει κοινές δυνατότητες ώστε οι υπηρεσίες Product, Cart, Inventory, Order και Notification να συνεργάζονται ως κατανεμημένο σύστημα.

Η επιλογή αυτή είναι συνεπής με την αρχιτεκτονική του έργου: οι υπηρεσίες παραμένουν Spring Boot εφαρμογές με διακριτό domain, ενώ το Spring Cloud εισάγεται μόνο στα σημεία όπου απαιτείται συντονισμός. Ο Discovery Server παρέχει το μητρώο υπηρεσιών, το Gateway ελέγχει το εξωτερικό όριο, το OpenFeign κάνει δηλωτικές εσωτερικές κλήσεις και το Resilience4j προστατεύει την κρίσιμη εξάρτηση από απόθεμα. Τα RabbitMQ, Keycloak και Zipkin συμπληρώνουν την υποδομή για messaging, identity και observability.

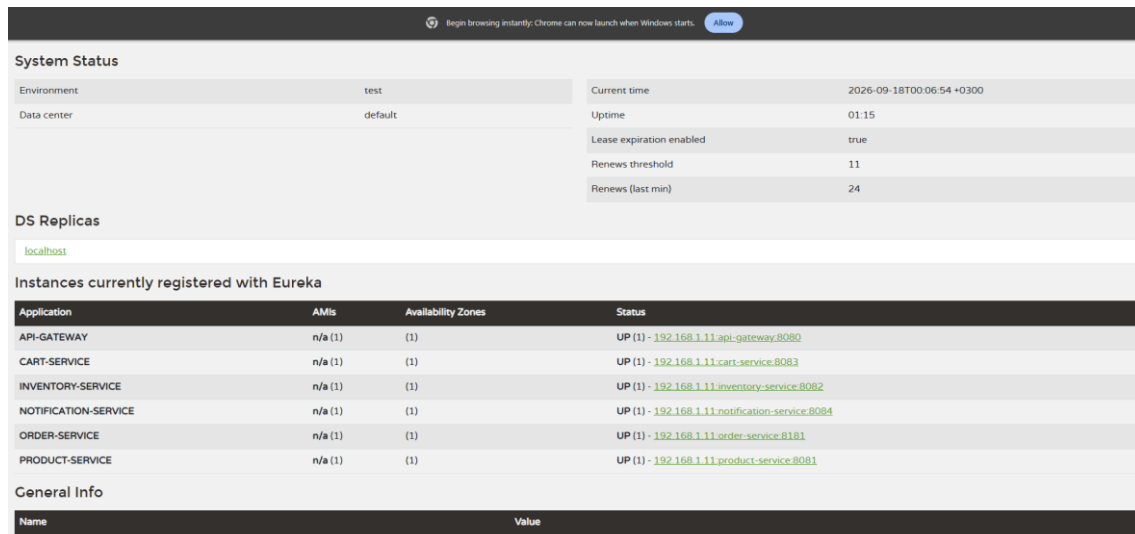
- Οι υπηρεσίες επιχειρησιακού domain παραμένουν υπεύθυνες για τα δεδομένα και τους κανόνες τους.
- Η υποδομή μειώνει την ανάγκη για hard-coded URLs και χειροκίνητη διαχείριση cross-cutting concerns.
- Οι εξαρτήσεις είναι ρητές: συγχρονισμένη επικοινωνία για αποφάσεις checkout και ασύγχρονη επικοινωνία για ειδοποιήσεις.
- Οι μηχανισμοί ασφάλειας, ανθεκτικότητας και tracing εφαρμόζονται στο όριο ή στο σημείο της πραγματικής εξάρτησης, όχι ως γενική υπόθεση του client.

5.2 Eureka Discovery Server

Ο Discovery Server εκτελείται στη θύρα 8761 και υλοποιείται ως Eureka Server. Οι backend εφαρμογές εγγράφονται σε αυτόν ως Eureka clients κατά την εκκίνησή τους. Κατά συνέπεια, μια υπηρεσία δεν χρειάζεται να γνωρίζει τη φυσική διεύθυνση ή τη θύρα μιας άλλης υπηρεσίας: χρησιμοποιεί το λογικό όνομά της, όπως order-service ή inventory-service. Η προσέγγιση αυτή είναι ιδιαίτερα χρήσιμη όταν υπάρχουν περισσότερα από ένα instances ή όταν η τοποθεσία υπηρεσιών μεταβάλλεται.

Στην παρούσα τοπική ανάπτυξη οι εφαρμογές χρησιμοποιούν σταθερές θύρες για ευκολία εκτέλεσης: 8081 για Product, 8082 για Inventory, 8083 για Cart, 8084 για Notification και 8181 για Order. Η ύπαρξη του Eureka δεν ακυρώνεται από τις σταθερές τοπικές θύρες. Αντίθετα, προετοιμάζει τη λύση για περιβάλλον όπου μία υπηρεσία μπορεί να εμφανίζεται σε περισσότερες περιπτώσεις και το Gateway ή οι Feign clients πρέπει να βρουν διαθέσιμο instance με βάση καταχώριση στο registry.

- Discovery Server: θύρα 8761 και ρόλος μητρώου διαθέσιμων υπηρεσιών.
- Κάθε υπηρεσία δηλώνει το λογικό service name της και εγγράφεται κατά την εκκίνηση.
- Το service name χρησιμοποιείται από Gateway routes και OpenFeign clients αντί για σταθερό host:port.
- Η υγεία μιας υπηρεσίας και η εγγραφή της στο registry γίνονται ορατές από το Eureka dashboard.



The screenshot shows the Eureka dashboard with the following sections:

- System Status:** A table with system information.

Environment	test	Current time	2026-09-18T00:06:54 +0300
Data center	default	Uptime	01:15
		Lease expiration enabled	true
		Renews threshold	11
		Renews (last min)	24
- DS Replicas:** A table showing replicas on localhost.
- Instances currently registered with Eureka:** A table listing registered services.

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 192.168.1.11:api-gateway:8080
CART-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:cart-service:8083
INVENTORY-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:inventory-service:8082
NOTIFICATION-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:notification-service:8084
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:order-service:8181
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:product-service:8081
- General Info:** A table with Name and Value columns.

Εικόνα 5.1: Eureka dashboard με τις εγγεγραμμένες υπηρεσίες.

5.3 Spring Cloud Gateway

Το API Gateway εκτελείται στη θύρα 8080 και είναι το δημόσιο σημείο εισόδου του backend. Η React εφαρμογή δεν καλεί απευθείας τις εσωτερικές θύρες των υπηρεσιών. Αντί αυτής της σύζευξης, στέλνει HTTP αιτήματα στο Gateway και οι routes αντιστοιχίζουν τα API prefixes σε load-balanced προορισμούς. Έτσι, η αλλαγή μιας εσωτερικής διεύθυνσης ή η προσθήκη δεύτερου instance δεν απαιτεί αλλαγή στον frontend. Το Spring Cloud Gateway προσφέρει το κατάλληλο πλαίσιο για route predicates και filters σε αντιδραστική υποδομή.

Η υλοποίηση ορίζει διαδρομές `/api/order/**` προς `lb://order-service`, `/api/inventory/**` προς `lb://inventory-service`, `/api/product/**` και `/api/categories/**` προς `lb://product-service`, `/api/cart/**` προς `lb://cart-service` και `/api/notification/**` προς `lb://notification-service`. Το πρόθεμα `lb` δηλώνει ότι ο προορισμός επιλύεται μέσω μηχανισμού load balancing και discovery. Η δρομολόγηση είναι δηλωμένη στη ρύθμιση του Gateway, άρα η συμπεριφορά της παραμένει ευανάγνωστη χωρίς να αναπαράγεται κώδικας προώθησης σε κάθε controller.

Πέρα από τη δρομολόγηση, το Gateway εφαρμόζει την πρώτη γραμμή ελέγχου ασφάλειας. Τα GET endpoints catalog επιτρέπονται χωρίς login, ώστε να μπορεί να παρουσιαστεί κατάλογος προϊόντων σε επισκέπτη. Οι υπόλοιπες προστατευμένες διαδρομές απαιτούν αυθεντικοποίηση. Η πολιτική αυτή ακολουθεί την αρχή της ελάχιστης έκθεσης: η δημόσια ανάγνωση είναι ρητά καθορισμένη, ενώ κάθε μη δηλωμένη πρόσβαση δεν θεωρείται αυτομάτως διαθέσιμη.

- Ενιαίο base URL για το frontend: `http://localhost:8080`.
- Route predicates ανά API prefix, με λογικούς προορισμούς υπηρεσιών `lb://...`
- Επαλήθευση bearer token στο επίπεδο Gateway για τις προστατευμένες διαδρομές.
- Διαχωρισμός δημόσιων catalog reads από authenticated operations cart, order και administration.
- Συγκέντρωση δρομολόγησης και security configuration σε ένα ελεγχόμενο edge component.

5.4 OpenFeign για επικοινωνία υπηρεσία-προς-υπηρεσία

Η δημιουργία παραγγελίας χρειάζεται δύο εσωτερικές πληροφορίες: το τρέχον καλάθι του πελάτη και την επιβεβαίωση της ενεργής δέσμευσης αποθέματος. Το Order Service υλοποιεί αυτές τις εξαρτήσεις με OpenFeign clients προς Cart και Inventory. Ο προγραμματιστής δηλώνει το απαιτούμενο HTTP συμβόλαιο ως Java interface με annotations, ενώ το Spring δημιουργεί την υλοποίηση του client. Με το όνομα της υπηρεσίας και το discovery, ο client δεν συνδέεται με hard-coded URL.

Η επιλογή συγχρονισμένης κλήσης είναι σκόπιμη για τη συγκεκριμένη ροή. Το Order Service δεν πρέπει να αποθηκεύσει επιτυχημένη παραγγελία πριν γνωρίζει ότι υπάρχει έγκυρη δέσμευση και ότι η ποσότητα μπορεί να οριστικοποιηθεί. Η άμεση απάντηση του Inventory Service επιτρέπει στο checkout να επιστρέψει ξεκάθαρο αποτέλεσμα. Αντίθετα, η αποστολή email δεν επηρεάζει την επιχειρησιακή εγκυρότητα της παραγγελίας και επομένως δεν υλοποιείται με OpenFeign.

- Cart client: ανάκτηση του καλαθιού του τρέχοντος χρήστη και εκκαθάριση μετά την επιτυχία.
- Inventory client: επιβεβαίωση της reservation πριν την αποθήκευση της παραγγελίας.
- Declarative interfaces: το συμβόλαιο HTTP είναι μέρος του κώδικα και ελέγχεται από τον compiler.
- Service discovery: οι clients αναφέρονται σε service name και όχι σε διαχειριζόμενη IP/θύρα.
- Η επιλογή κλήσης δικαιολογείται μόνο για βήμα που απαιτεί άμεση επιχειρησιακή απόφαση.

5.5 Resilience4j και χειρισμός μερικών αποτυχιών

Σε ένα καταναλωμένο σύστημα η αποτυχία ενός εξαρτώμενου service δεν είναι εξαιρετική περίπτωση αλλά αναμενόμενη συνθήκη λειτουργίας. Μια κλήση μπορεί να καθυστερήσει, να αποτύχει προσωρινά ή να δεχθεί υπερβολικό φορτίο. Το Order Service προστατεύει την κλήση επιβεβαίωσης inventory με circuit breaker του Resilience4j. Η πρακτική αυτή περιορίζει τις επαναλαμβανόμενες προσπάθειες προς μια μη διαθέσιμη υπηρεσία και αποτρέπει την αλυσιδωτή εξάντληση πόρων στο checkout.

Στην υλοποίηση ο circuit breaker έχει όνομα inventory. Όταν η κλήση προς Inventory αποτύχει ή ο breaker είναι ανοικτός, ενεργοποιείται fallback που καταλήγει σε ServiceUnavailableException. Η απόκριση προς τον client δηλώνει ότι η υπηρεσία αποθέματος δεν είναι προσωρινά διαθέσιμη. Το πρότυπο circuit breaker περιορίζει την επιμονή σε προβληματική απομακρυσμένη εξάρτηση, όπως αναλύεται από τον Nygard [10]. Οι συγκεκριμένες δηλώσεις Spring και Resilience4j καταγράφονται στον κώδικα του έργου· η περιγραφή τους δεν συνοδεύεται από πείραμα πρόκλησης αποτυχίας.

Η συγκεκριμένη έκδοση δεν ενεργοποιεί αυτόματο retry για την οριστικοποίηση αποθέματος, επειδή μια επανάληψη χωρίς προσεκτική idempotency θα μπορούσε να επιβαρύνει τη ροή ή να προκαλέσει μη αναμενόμενη διπλή ενέργεια. Το idempotency key του Order Service λειτουργεί συμπληρωματικά: προστατεύει από επανάληψη του checkout request από τον client, ενώ ο circuit breaker προστατεύει την εξάρτηση από απομακρυσμένη υπηρεσία.

5.6 RabbitMQ και ασύγχρονες ειδοποιήσεις

Η RabbitMQ υποδομή υλοποιεί event-driven επικοινωνία για την ειδοποίηση παραγγελίας. Μετά την επιτυχή αποθήκευση, το Order Service δημοσιεύει OrderPlacedEvent στο topic exchange orderExchange με routing key order.placed. Το Notification Service δεσμεύει την queue notificationQueue στο ίδιο routing key και καταναλώνει το JSON event. Η χρήση exchange και routing key αποσυνδέει τον παραγωγό από τον συγκεκριμένο καταναλωτή και επιτρέπει την προσθήκη νέου consumer χωρίς αλλαγή στον κώδικα δημιουργίας παραγγελίας.

Ο listener του Notification Service λαμβάνει δεδομένα όπως order number και email πελάτη και επιχειρεί αποστολή μηνύματος SMTP. Η αποστολή μπορεί να ενεργοποιείται ή να απενεργοποιείται από ρύθμιση, ενώ το fail-on-error καθορίζει πώς θα αντιμετωπίζονται σφάλματα ειδοποίησης. Η ασύγχρονη σχεδίαση μειώνει τη χρονική σύζευξη: ο πελάτης λαμβάνει αποτέλεσμα checkout όταν η παραγγελία αποθηκευθεί, όχι όταν ένας εξωτερικός email server ολοκληρώσει την εργασία του.

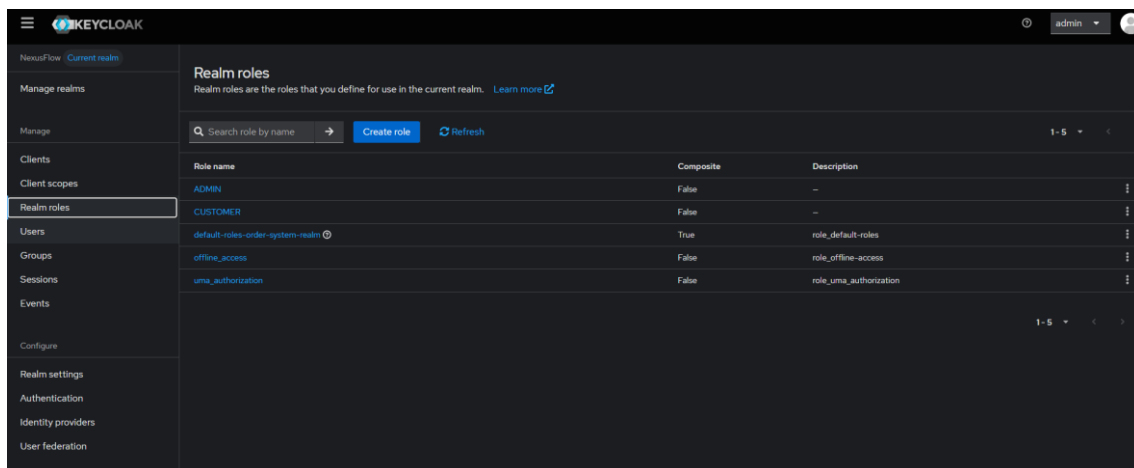
- Producer: Order Service μετά από επιτυχή δημιουργία παραγγελίας.
- Exchange: orderExchange τύπου topic.
- Routing key: order.placed.
- Consumer queue: notificationQueue, την οποία ακούει το Notification Service.
- Payload: JSON OrderPlacedEvent με στοιχεία αναγκαία για την ενημέρωση του πελάτη.

- Οφέλη: αποσύνδεση checkout από SMTP και δυνατότητα επιπλέον consumers, όπως analytics ή audit, σε επόμενη έκδοση.

5.7 Keycloak, OAuth2 και JWT

Ο Keycloak εκτελείται τοπικά στη θύρα 8180 και λειτουργεί ως κεντρικός identity provider. Το frontend χρησιμοποιεί keycloak-js για login και διατήρηση session, ενώ το Gateway και οι backend υπηρεσίες λειτουργούν ως resource servers που ελέγχουν τα bearer tokens. Η ρύθμιση issuer συνδέει την επικύρωση του JWT με το realm του Keycloak. Με αυτόν τον τρόπο, η αυθεντικοποίηση δεν υλοποιείται ξεχωριστά από κάθε microservice και η πληροφορία ρόλου μεταφέρεται ως claim του token.

Η εξουσιοδότηση είναι πολυεπίπεδη. Το Gateway περιορίζει την πρόσβαση σε προστατευμένες διαδρομές, αλλά οι επιχειρησιακές υπηρεσίες διατηρούν τους δικούς τους κανόνες. Για παράδειγμα, το Product Service απαιτεί ADMIN για μεταβολές catalog, το Inventory Service απαιτεί CUSTOMER ή ADMIN για reserve/confirm/release και ADMIN για restock, ενώ το Order Service περιορίζει τα administration endpoints και ελέγχει την ιδιοκτησία παραγγελίας για τον πελάτη. Η επανάληψη των ελέγχων είναι απαραίτητη, επειδή ένας client δεν πρέπει να μπορεί να παρακάμψει το Gateway καλώντας εσωτερική υπηρεσία σε διαφορετικό περιβάλλον ανάπτυξης.

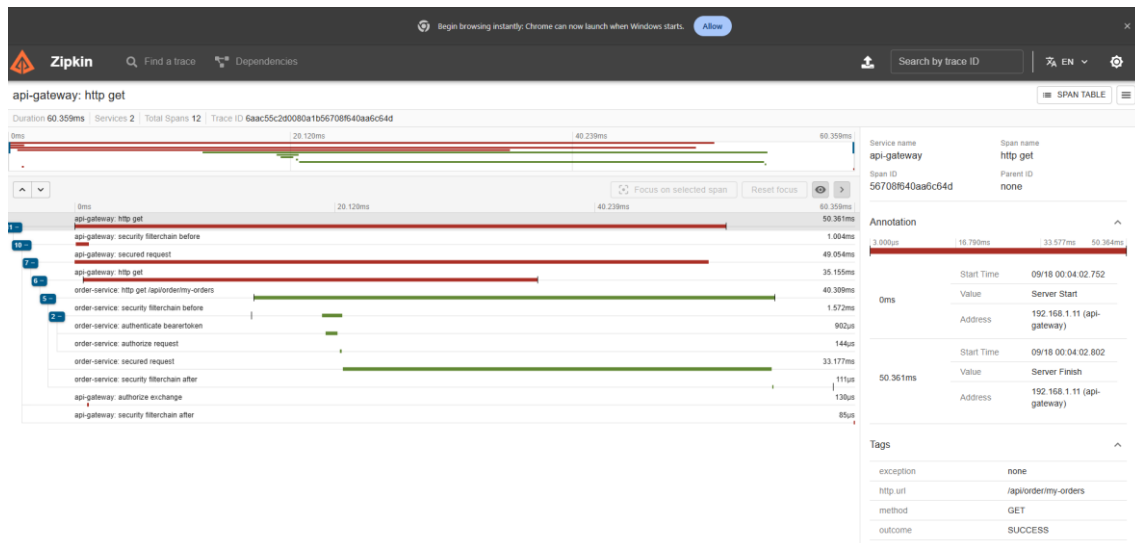


Εικόνα 5.2: Ρόλοι ADMIN και CUSTOMER στο Keycloak.

5.8 Distributed tracing με Micrometer και Zipkin

Σε μια κατανεμημένη εφαρμογή, ένα αίτημα μπορεί να προκαλέσει ενέργειες σε διαφορετικές υπηρεσίες, οπότε τα μεμονωμένα logs δεν εκφράζουν από μόνα τους τη συσχέτισή τους. Το Micrometer Tracing παρέχει την υποδομή καταγραφής trace και span, ενώ η αρχιτεκτονική Zipkin διαχωρίζει τη συλλογή, την αποθήκευση και την προβολή των δεδομένων ιχνηλάτησης [34, 35]. Στην εφαρμογή, το Micrometer παράγει τα σχετικά στοιχεία και το Zipkin τα συγκεντρώνει στη θύρα 9411. Το διαθέσιμο στιγμιότυπο τεκμηριώνει τη διαδρομή ανάκτησης παραγγελιών μέσω Gateway και Order Service, όχι κάθε πιθανή διαδρομή checkout ή μηνυμάτων.

Η ρύθμιση δειγματοληψίας είναι 1.0, συνεπώς στο αναπτυξιακό περιβάλλον συλλέγεται το πλήρες δείγμα αιτημάτων. Η επιλογή είναι κατάλληλη για τεκμηρίωση και έλεγχο, διότι επιτρέπει να παρουσιαστεί πλήρης αλυσίδα checkout σε ένα trace. Σε παραγωγικό περιβάλλον με μεγάλο φορτίο θα εξεταζόταν χαμηλότερο sampling rate ή προσαρμοσμένη πολιτική, ώστε ο όγκος τηλεμετρίας να διατηρείται ελεγχόμενος.



Εικόνα 5.3: Trace αιτήματος από API Gateway προς Order Service στο Zipkin.

Οι εξαρτήσεις tracing και οι ρυθμίσεις εξαγωγής στοιχείων στο Zipkin προσδιορίζονται στα αρχεία των εφαρμογών. Τα trace και span identifiers χρησιμοποιούνται για τη συσχέτιση του καταγεγραμμένου αιτήματος μεταξύ υπηρεσιών. Η συγκεκριμένη λειτουργική παρατήρηση διατηρείται χωριστά από μετρήσεις μέσου χρόνου απόκρισης, φορτίου ή κόστους instrumentation, οι οποίες δεν παρουσιάζονται ως αποτελέσματα της εργασίας.

Ειδικότερα, η κοινή ταυτότητα trace επιτρέπει τη συσχέτιση επιμέρους spans στην ίδια ακολουθία δραστηριότητας. Το Zipkin παρουσιάζει αυτά τα στοιχεία ως χρονική διάταξη, ώστε να είναι ορατή η συμμετοχή διαφορετικών υπηρεσιών στο αίτημα [35]. Στο διαθέσιμο trace της εφαρμογής, η κοινή ταυτότητα αιτήματος συνδέει τις εγγραφές του Gateway με την εκτέλεση του Order Service. Η παρατήρηση στηρίζει την ανάγκη κατανεμημένης ιχνηλάτησης, χωρίς να αποτελεί μέτρηση του κόστους παρακολούθησης ή συμπέρασμα για τη συνολική απόδοση του συστήματος.

5.9 Τοπική υποδομή με Docker Compose

Το Docker Compose συγκεντρώνει τις υποδομές που δεν αποτελούν επιχειρησιακές Spring Boot υπηρεσίες: RabbitMQ, Redis, Zipkin και Keycloak. Η RabbitMQ εκθέτει τις θύρες 5672 για AMQP και 15672 για το management UI. Το Redis εκτίθεται στη 6379, το Zipkin στη 9411 και ο Keycloak στη 8180. Η χρήση Compose επιτρέπει επαναλήψιμη τοπική εκτέλεση, επειδή τα εξαρτώμενα components ξεκινούν με δηλωμένες ρυθμίσεις αντί να εγκαθίστανται και να παραμετροποιούνται χειροκίνητα σε κάθε υπολογιστή.

Η υποδομή αυτή χρησιμοποιείται ως πρακτική βάση για τις δοκιμές του επόμενου κεφαλαίου. Ιδιαίτερα, το RabbitMQ management UI και το Zipkin προσφέρουν ορατά τεκμήρια ότι δημοσιεύονται events και ότι τα αιτήματα διασχίζουν τις αναμενόμενες υπηρεσίες. Το Compose δεν αποτελεί από μόνο του πλήρη στρατηγική production deployment, αλλά εξυπηρετεί αξιόπιστα την ανάπτυξη, την επίδειξη και την αναπαραγωγή του συστήματος σε ελεγχόμενο περιβάλλον.

Στο τοπικό περιβάλλον, το Docker Compose δηλώνει τις εξαρτήσεις που χρειάζονται για την εκτέλεση των εφαρμογών. Η επίσημη τεκμηρίωση περιγράφει τον ορισμό υπηρεσιών σε αρχείο Compose και τη διαχείριση του κύκλου εκτέλεσής τους μέσω κοινών εντολών [49]. Στο έργο, το αρχείο δηλώνει τις υποδομές RabbitMQ, Redis, Zipkin και Keycloak με τις αντίστοιχες θύρες και παραμέτρους. Η παρούσα χρήση περιορίζεται στην επαναλήψιμη τοπική εκκίνηση των δηλωμένων υποδομών και δεν αποτελεί διάθεση σε πλατφόρμα PaaS ή αξιολόγηση πολυνεφικού περιβάλλοντος.

5.10 Συσχέτιση παραγγελίας γεγονότος και ειδοποίησης

Η παραγωγή ειδοποίησης ξεκινά από το Order Service με γεγονός OrderPlacedEvent, το οποίο περιέχει τον αριθμό παραγγελίας και το email του πελάτη. Η δημοσίευση χρησιμοποιεί το exchange orderExchange και το routing key order.placed. Η υπηρεσία ειδοποιήσεων δηλώνει καταναλωτή στην ουρά notificationQueue μέσω της επισήμανσης RabbitListener. Τα ονόματα αυτά έχουν διακριτούς ρόλους: το exchange δέχεται τη δημοσίευση, το routing key συμμετέχει στη δρομολόγηση και η ουρά είναι το σημείο από το οποίο καταναλώνει ο listener.

Το περιεχόμενο του γεγονότος παραμένει μικρό και προσανατολισμένο στην ανάγκη ειδοποίησης. Δεν μεταφέρεται ολόκληρη η οντότητα παραγγελίας ούτε η εσωτερική δομή της βάσης. Για το μήνυμα επιβεβαίωσης αρκούν το αναγνωριστικό που θα αναφερθεί στο θέμα και στο σώμα και η διεύθυνση παραλήπτη. Η επιλογή αυτή διατηρεί σαφή τη σημασία του συμβάντος: γνωστοποιεί ότι καταχωρίστηκε παραγγελία και δίνει τα στοιχεία που χρειάζεται ο συγκεκριμένος καταναλωτής. Δεν αποτελεί εντολή νέας δημιουργίας παραγγελίας.

Στο Notification Service, οι ρυθμίσεις NotificationEmailProperties καθορίζουν αν είναι ενεργή η αποστολή, αν θα δημιουργηθεί HTML περιεχόμενο, ποια θα είναι η διεύθυνση αποστολέα και πώς θα διαμορφωθούν θέμα και σώμα. Η μέθοδος render αντικαθιστά τα πεδία {orderNumber} και {email} στα πρότυπα. Το MimeMessageHelper χρησιμοποιεί UTF-8 και το JavaMailSender εκτελεί την αποστολή. Η διαμόρφωση του μηνύματος παραμένει έτσι εντός της υπηρεσίας ειδοποιήσεων, ενώ η υπηρεσία παραγγελιών δεν εξαρτάται από τη μορφοποίηση του email.

Για το περιβάλλον επίδειξης, το Mailtrap λειτουργεί ως ελεγχόμενος προορισμός SMTP. Το παραληφθέν μήνυμα επιβεβαιώνει ότι δημιουργήθηκε email με τα συγκεκριμένα στοιχεία και έφθασε στο sandbox. Η παρατήρηση είναι διαφορετική από την παράδοση σε πραγματικό γραμματοκιβώτιο χρήστη. Αντίστοιχα, οι ενδείξεις του RabbitMQ περιγράφουν την κατάσταση του broker κατά τη λήψη του στιγμιότυπου. Η συνδυασμένη ανάγνωση κώδικα, broker και sandbox συνδέει τα διαδοχικά επίπεδα της ροής, χωρίς να μετατρέπει τις οθόνες διαχείρισης σε μέτρηση καθυστέρησης. Η χρήση ελεγχόμενου mailbox αντί πραγματικής παράδοσης αντιστοιχεί στον ρόλο του Email Sandbox που περιγράφει η Mailtrap [50].

5.11 Ταυτότητα χρήστη και κατανομή εξουσιοδότησης

Στη ροή πρόσβασης διακρίνονται η επαλήθευση token, ο προσδιορισμός χρήστη και ο έλεγχος επιτρεπόμενης ενέργειας. Η ανάλυση OAuth εξετάζει τον ρόλο της εξουσιοδότησης στην επικοινωνία [28], ενώ τα μοντέλα RBAC οργανώνουν τη σύνδεση χρηστών, ρόλων και αδειών [29]. Στο έργο, η εξαγωγή subject ή email επιλέγει τα δεδομένα του χρήστη και οι έλεγχοι CUSTOMER/ADMIN περιορίζουν τις διαθέσιμες λειτουργίες. Δεν αποδίδεται στην εφαρμογή υλοποίηση ιεραρχικού ή δυναμικά περιορισμένου RBAC πέρα από τους καταγεγραμμένους κανόνες.

Στο Gateway δηλώνονται δημόσιες εξαιρέσεις για τις αναγνώσεις προϊόντων και τις αντίστοιχες λειτουργίες ανάγνωσης ή ελέγχου διαθεσιμότητας. Τα υπόλοιπα αιτήματα απαιτούν αυθεντικοποίηση. Στις επιμέρους υπηρεσίες εφαρμόζονται οι ειδικότερες πολιτικές ρόλων. Για παράδειγμα, το Order Service απαιτεί ADMIN για τη συνολική λίστα παραγγελιών και για τις μεταβολές κατάστασης με PATCH. Το Inventory Service διακρίνει τις ενέργειες κράτησης από την αναπλήρωση και τη δημιουργία εγγραφής αποθέματος.

Πίνακας 5.1: Κατανομή ελέγχων ταυτότητας και δικαιωμάτων.

Επίπεδο	Έλεγχος στον κώδικα	Πεδίο εφαρμογής
Gateway	Δημόσιες εξαιρέσεις για GET προϊόντων και ανάγνωση ή έλεγχο διαθεσιμότητας. Τα υπόλοιπα exchanges απαιτούν αυθεντικοποίηση.	Αρχικός έλεγχος εισερχόμενης επικοινωνίας και επαλήθευση JWT.
Order Service	GET της συνολικής λίστας και PATCH απαιτούν ADMIN. Δημιουργία και ακύρωση δέχονται CUSTOMER ή ADMIN.	Διάκριση κατηγοριών λειτουργιών σε επίπεδο HTTP.

Επίπεδο	Έλεγχος στον κώδικα	Πεδίο εφαρμογής
Inventory Service	reserve, confirm και release απαιτούν CUSTOMER ή ADMIN. restock και create απαιτούν ADMIN.	Διάκριση ενεργειών αγοράς από διαχειριστική μεταβολή αποθέματος.
Cart Controller	Η ταυτότητα καλαθιού προκύπτει από <code>jwt.getSubject()</code> .	Συσχέτιση των ενεργειών καλαθιού με τον αυθεντικοποιημένο χρήστη.
Ακύρωση παραγγελίας	Ελέγχεται αν ο αιτών είναι διαχειριστής ή αν αντιστοιχεί στον πελάτη της παραγγελίας, καθώς και η επιλεξιμότητα κατάστασης.	Επιχειρησιακή εξουσιοδότηση για τη συγκεκριμένη ενέργεια ακύρωσης.

Στο καλάθι, ο controller λαμβάνει το Jwt ως `AuthenticationPrincipal` και χρησιμοποιεί το `subject` ως ταυτότητα καλαθιού. Η ταυτότητα αυτή δεν εισάγεται ως ελεύθερο πεδίο της φόρμας προσθήκης προϊόντος. Στην προσωπική προβολή παραγγελιών εξετάζονται διαδοχικά `email`, `preferred_username` και `subject` για την επιλογή της ταυτότητας πελάτη. Η ανάλυση αφορά τις συγκεκριμένες διαδρομές και βοηθά να κατανοηθεί γιατί τα αναγνωριστικά χρήστη και πελάτη δεν πρέπει να συγχέονται με το UUID προϊόντος ή παραγγελίας.

Κατά την ακύρωση, η υπηρεσία ανακτά την παραγγελία, προσδιορίζει τον αιτούντα και εξετάζει αν είναι διαχειριστής ή ο αντίστοιχος πελάτης. Έπειτα ελέγχει αν η κατάσταση επιτρέπει ακύρωση. Η απόφαση δεν εξαρτάται μόνο από το αν εμφανίστηκε ένα πλήκτρο στο React. Η διεπαφή αξιοποιεί την επιλεξιμότητα για την παρουσίαση, ενώ η υπηρεσία εφαρμόζει τον κανόνα κατά την εκτέλεση. Αυτός ο διαχωρισμός συνδέει το μοντέλο δικαιωμάτων με τον κύκλο ζωής της παραγγελίας.

Το CORS του Gateway επιτρέπει το τοπικό origin `http://localhost:5173` και τις δηλωμένες μεθόδους HTTP. Η ρύθμιση αυτή αφορά τη συνεργασία του browser με την πύλη και δεν αντικαθιστά την επαλήθευση token ή τους ρόλους. Η αποσαφήνιση είναι χρήσιμη κατά την τοπική εγκατάσταση: διαφορετική διεύθυνση frontend πρέπει να συμφωνεί με τη διαμόρφωση origin, ενώ διαφορετικό realm πρέπει να συμφωνεί με τον issuer. Η λειτουργική συνέπεια του περιβάλλοντος προκύπτει από τη συμφωνία των σχετικών ρυθμίσεων και όχι από έναν μοναδικό διακόπτη πρόσβασης.

5.12 Συμπεράσματα υποδομής

Η υποδομή Spring Cloud και τα συνοδευτικά components μετατρέπουν τις ανεξάρτητες Spring Boot εφαρμογές σε λειτουργικό σύστημα microservices. Το Eureka και το Gateway οργανώνουν discovery και edge routing, το OpenFeign υλοποιεί δηλωτικές κλήσεις για κρίσιμα βήματα, το Resilience4j περιορίζει την επίδραση μερικών αποτυχιών, το RabbitMQ αποσυνδέει την ειδοποίηση από το checkout, ο Keycloak συγκεντρώνει την ταυτότητα και το Zipkin παρέχει παρατηρησιμότητα. Η αξία των στοιχείων αυτών θα αξιολογηθεί με συγκεκριμένα σενάρια λειτουργίας και αποτυχίας στο επόμενο κεφάλαιο.

Η υποδομή δεν πρέπει να εκλαμβάνεται ως ένα σύνολο προαιρετικών εργαλείων γύρω από τον επιχειρησιακό κώδικα. Στην παρούσα εφαρμογή, καθορίζει τα όρια του εξωτερικού API, τον τρόπο με τον οποίο οι υπηρεσίες ανακαλύπτουν η μία την άλλη, τον μηχανισμό μέσω του οποίου μεταφέρεται η ταυτότητα του χρήστη και το μέσο διάγνωσης μιας κατανεμημένης ροής. Συνεπώς, η αξιολόγηση του συστήματος στο επόμενο κεφάλαιο δεν περιορίζεται στο αν εμφανίζεται σωστά μια σελίδα. Ελέγχει αν η ίδια επιχειρησιακή πράξη αφήνει ελέγχιμα τεκμήρια σε επίπεδο API, κατάσταση παραγγελίας, μηνύματος και trace.

6. Έλεγχος και Αξιολόγηση

6.1 Μεθοδολογία

Η αξιολόγηση βασίζεται σε ολοκληρωμένα σενάρια χρήσης που εκτελέστηκαν μέσω της διεπαφής της εφαρμογής στο τοπικό περιβάλλον ανάπτυξης. Για κάθε σενάριο καταγράφεται το ορατό αποτέλεσμα της ενέργειας με στιγμιότυπο οθόνης. Με αυτόν τον τρόπο, η τεκμηρίωση παρουσιάζει μόνο λειτουργίες που αποδεικνύονται από πραγματική αλληλεπίδραση με το σύστημα και δεν αποδίδει αποτελέσματα σε έλεγχο που δεν εκτελέστηκε.

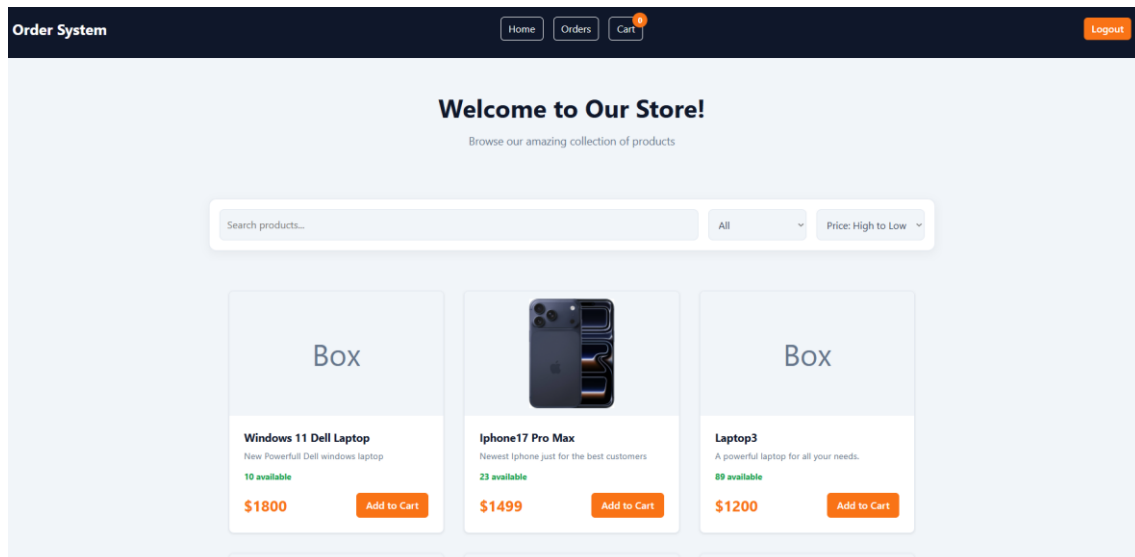
- Περιβάλλον: React frontend, Spring Boot microservices και εξαρτήσεις Docker Compose σε τοπική εκτέλεση.
- Χρήστες: CUSTOMER για τις λειτουργίες αγοράς και ADMIN για τις λειτουργίες καταλόγου και αποθέματος.
- Κριτήριο επιτυχίας: η διεπαφή εμφανίζει την αναμενόμενη κατάσταση ή επιβεβαίωση μετά από κάθε ενέργεια.

6.2 Σενάρια λειτουργικού ελέγχου

Τα ακόλουθα σενάρια καλύπτουν την κύρια ροή του πελάτη και τις διαθέσιμες διαχειριστικές ενέργειες. Τα στιγμιότυπα αποτελούν τεκμήρια της εκτέλεσης και συνδέουν την περιγραφή της λειτουργίας με το αντίστοιχο αποτέλεσμα στη διεπαφή.

6.2.1 TC-01: Περιήγηση στον κατάλογο

- Σκοπός: επιβεβαίωση εμφάνισης προϊόντων και των χειριστηρίων αναζήτησης, φιλτραρίσματος και ταξινόμησης.
- Βήματα: ανοίξτε την αρχική σελίδα και προβάλτε τον διαθέσιμο κατάλογο.
- Αποτέλεσμα: εμφανίζεται πλέγμα προϊόντων με πεδία αναζήτησης, κατηγορίας και ταξινόμησης.



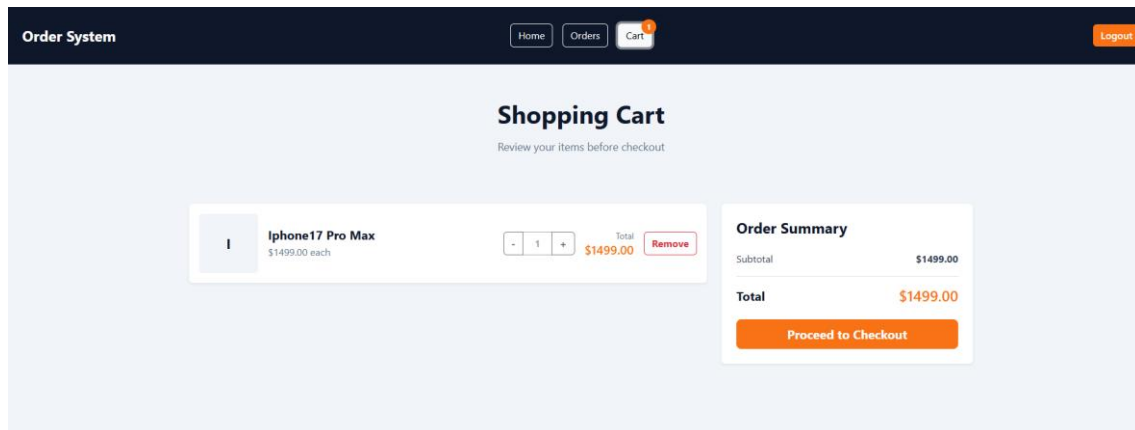
Εικόνα 6.1: Προβολή καταλόγου προϊόντων και διαθέσιμων χειριστηρίων.

6.2.2 TC-02: Διαχείριση καλαθιού

- Σκοπός: επιβεβαίωση εμφάνισης προϊόντος στο καλάθι, ποσότητας, τιμής και συνόλου.
- Βήματα: επιλέξτε προϊόν, προσθέστε το στο καλάθι και προβάλτε τη σύνοψη παραγγελίας.

Ανάπτυξη ενός Συστήματος Διαχείρισης Παραγγελιών (Order Management System) βασισμένου σε Microservices με Spring Boot και Spring Cloud

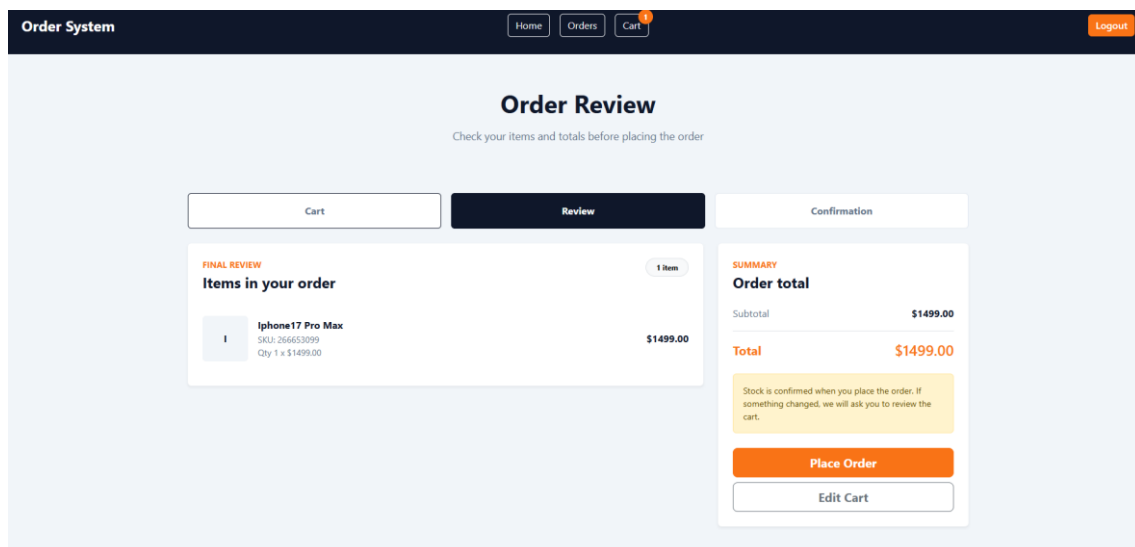
- Αποτέλεσμα: εμφανίζονται το είδος, η ποσότητα, η τιμή μονάδας και το συνολικό ποσό.



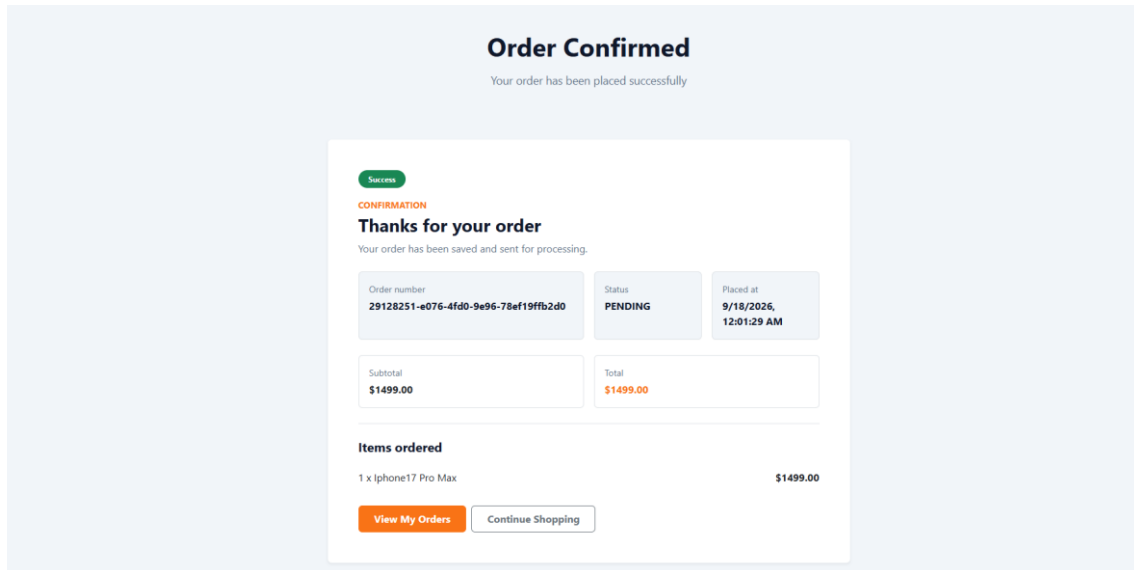
Εικόνα 6.2: Τεκμήριο διαχείρισης καλαθιού και υπολογισμού συνόλου.

6.2.3 TC-03: Υποβολή και επιβεβαίωση παραγγελίας

- Σκοπός: επιβεβαίωση της ροής από την ανασκόπηση του καλαθιού έως την επιτυχή δημιουργία παραγγελίας.
- Βήματα: προχωρήστε από το καλάθι στην ανασκόπηση και επιλέξτε Place Order.
- Αποτέλεσμα: η εφαρμογή εμφανίζει μοναδικό αριθμό παραγγελίας, αρχική κατάσταση PENDING και τα στοιχεία της παραγγελίας.



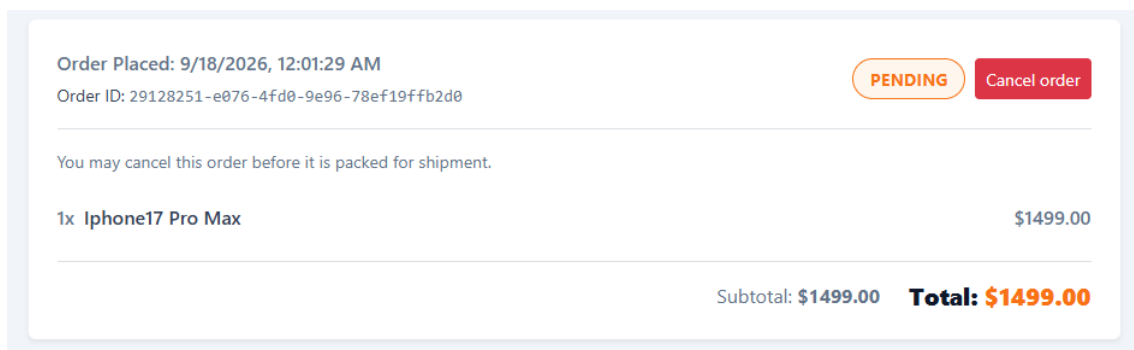
Εικόνα 6.3: Ανασκόπηση στοιχείων πριν από την υποβολή παραγγελίας.



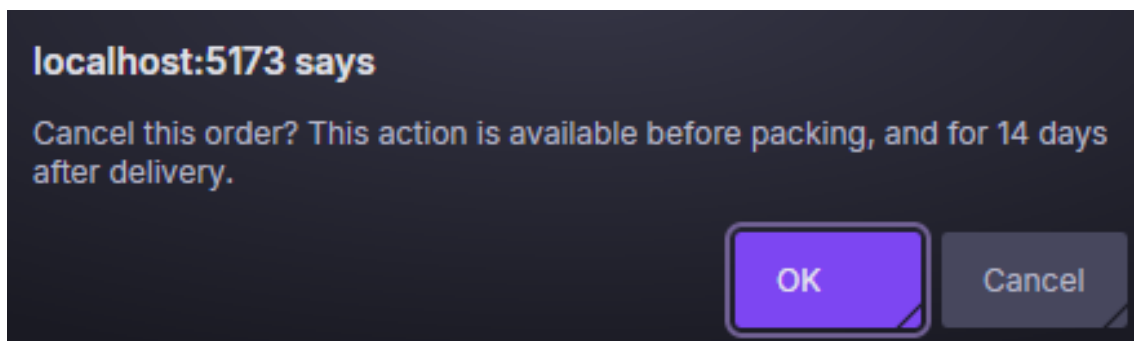
Εικόνα 6.4: Επιτυχής επιβεβαίωση παραγγελίας με κατάσταση PENDING.

6.2.4 TC-04: Ακύρωση παραγγελίας

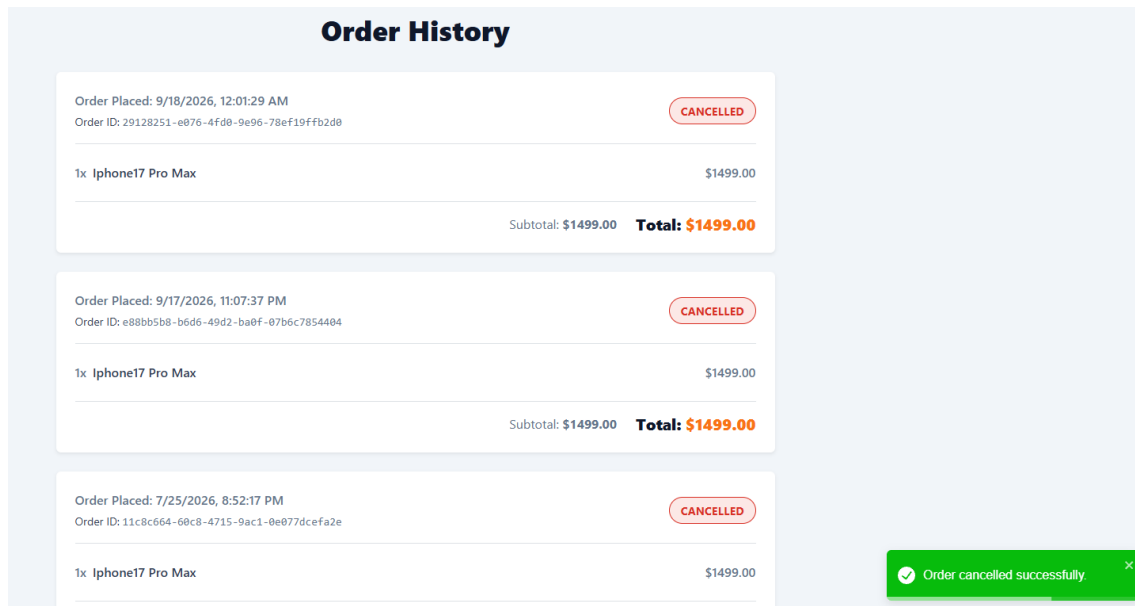
- Σκοπός: επιβεβαίωση ότι παραγγελία σε κατάσταση PENDING μπορεί να ακυρωθεί από τον πελάτη.
- Βήματα: ανοίξετε μια παραγγελία PENDING, επιλέξτε Cancel order και επιβεβαιώστε την ενέργεια.
- Αποτέλεσμα: η εφαρμογή εμφανίζει διάλογο επιβεβαίωσης, ενημερώνει την κατάσταση σε CANCELLED και προβάλλει μήνυμα επιτυχίας.



Εικόνα 6.5: Παραγγελία PENDING με διαθέσιμη ενέργεια ακύρωσης.



Εικόνα 6.6: Διάλογος επιβεβαίωσης ακύρωσης παραγγελίας.



Εικόνα 6.7: Ιστορικό παραγγελιών μετά από επιτυχή ακύρωση.

6.2.5 TC-05: Restock αποθέματος από διαχειριστή

- Σκοπός: επιβεβαίωση διαχειριστικής ενημέρωσης αποθέματος από το Catalogue and inventory panel.
- Βήματα: εισαγάγετε ποσότητα restock για το επιλεγμένο προϊόν και επιβεβαιώστε την ενέργεια.
- Αποτέλεσμα: η ποσότητα του προϊόντος αυξάνεται και η διεπαφή εμφανίζει μήνυμα επιτυχούς ενημέρωσης.

Catalogue and inventory

Product	SKU	Stock	Restock
 Laptop3 Electronics · ACTIVE	LAPTOP3-16GB-6127	89 of 89 total	Qty + Edit Delete
 Laptop4 Electronics · ACTIVE	LAPTOP4-16GB-4E0D	89 of 89 total	Qty + Edit Delete
 Laptop5 Electronics · ACTIVE	645182	15 of 15 total	Qty + Edit Delete
 Laptop6 Electronics · ACTIVE	974366532	20 of 20 total	Qty + Edit Delete
 Windows 11 Dell Laptop DEVICES · ACTIVE	239338339	10 of 10 total	9 + Edit Delete
 iPhone17 Pro Max SMARTPHONES · ACTIVE	266653099	22 of 22 total	Qty + Edit Delete

Εικόνα 6.8: Εισαγωγή ποσότητας restock από τον πίνακα διαχείρισης.

Catalogue and inventory

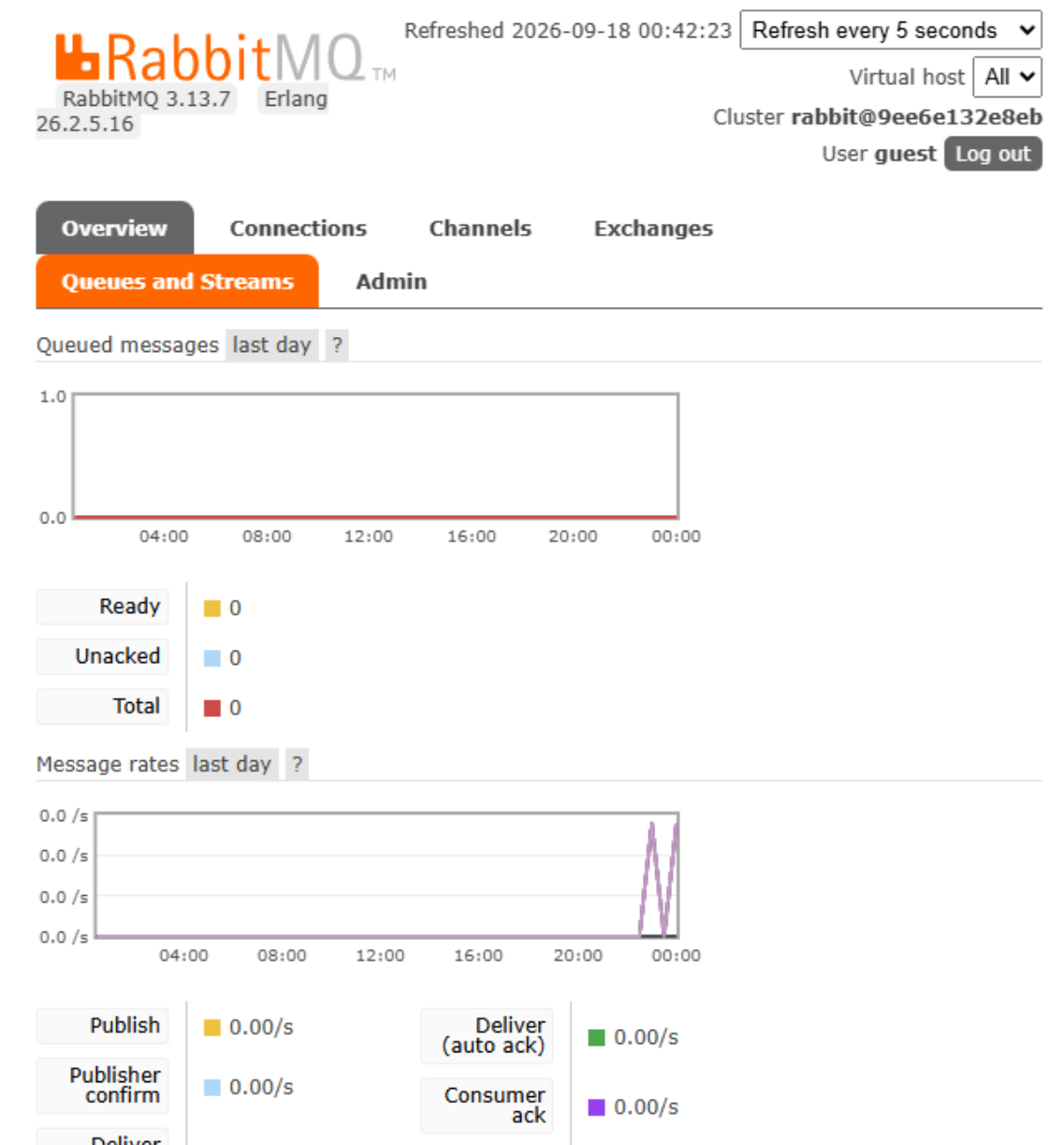
Product	SKU	Stock	Restock
 Laptop3 Electronics · ACTIVE	LAPTOP3-16GB-6127	89 of 89 total	Qty + Edit Delete
 Laptop4 Electronics · ACTIVE	LAPTOP4-16GB-4E0D	89 of 89 total	Qty + Edit Delete
 Laptop5 Electronics · ACTIVE	645182	15 of 15 total	Qty + Edit Delete
 Laptop6 Electronics · ACTIVE	974366532	20 of 20 total	Qty + Edit Delete
 Windows 11 Dell Laptop DEVICES · ACTIVE	239338339	19 of 19 total	Qty + Edit Delete
 Iphone17 Pro Max SMARTPHONES · ACTIVE	266653099	22 of 22 total	Qty + Edit Delete

✓ Windows 11 Dell Laptop restocked by 9. ✕

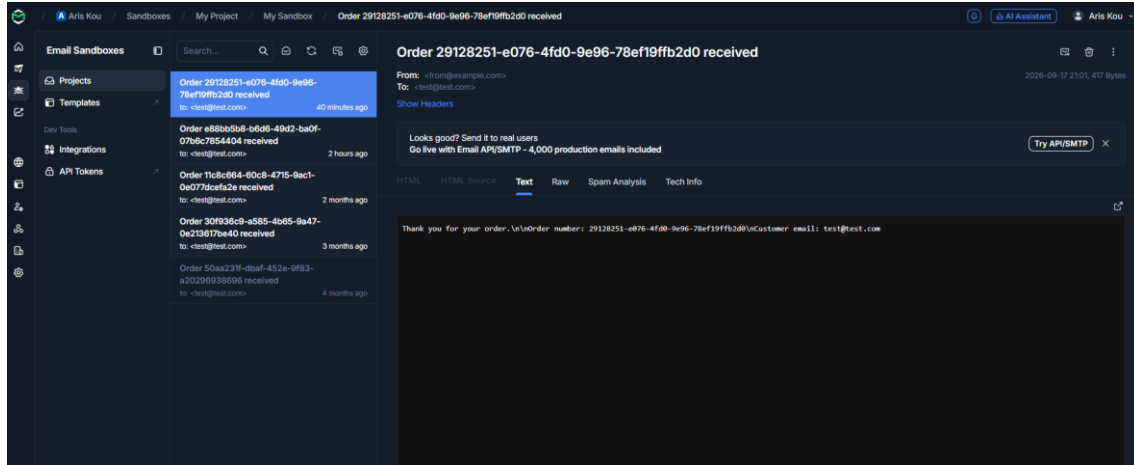
Εικόνα 6.9: Επιτυχής ενημέρωση αποθέματος μετά από restock.

6.2.6 TC-06: Ασύγχρονη ειδοποίηση παραγγελίας

- Σκοπός: συσχέτιση επιτυχούς παραγγελίας με την παραλαβή email επιβεβαίωσης στο δοκιμαστικό sandbox.
- Βήματα: ολοκληρώστε μια παραγγελία, ελέγξτε την κατάσταση του RabbitMQ broker και ανοίξτε το mailbox δοκιμών στο Mailtrap.
- Αποτέλεσμα: το RabbitMQ εμφανίζει συνολικές ενδείξεις δραστηριότητας και το Mailtrap περιέχει μήνυμα με τον αριθμό της παραγγελίας δοκιμής.



Εικόνα 6.10: Συνολικές ενδείξεις μηνυμάτων και δραστηριότητας στο RabbitMQ.



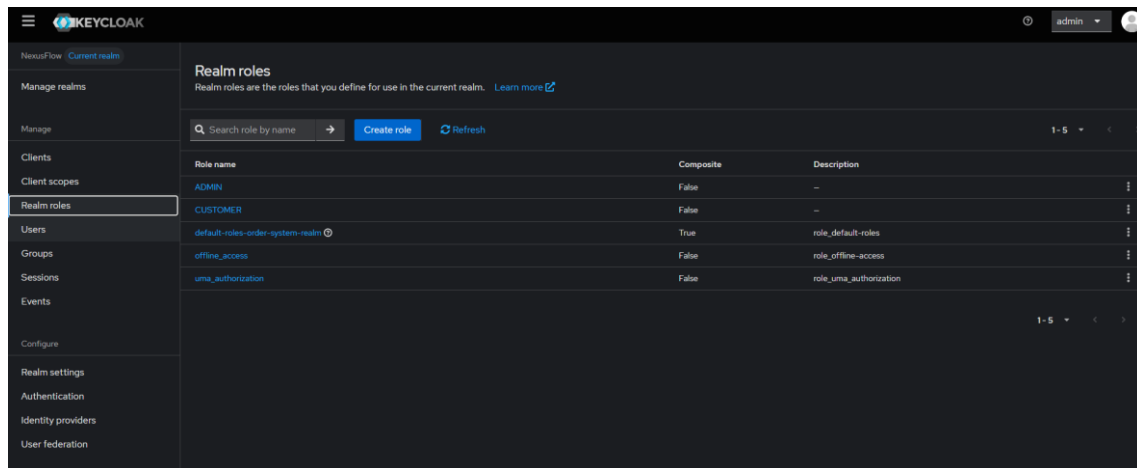
Εικόνα 6.11: Email επιβεβαίωσης παραγγελίας στο Mailtrap.

6.3 Επιβεβαίωση υποδομής και ασφάλειας

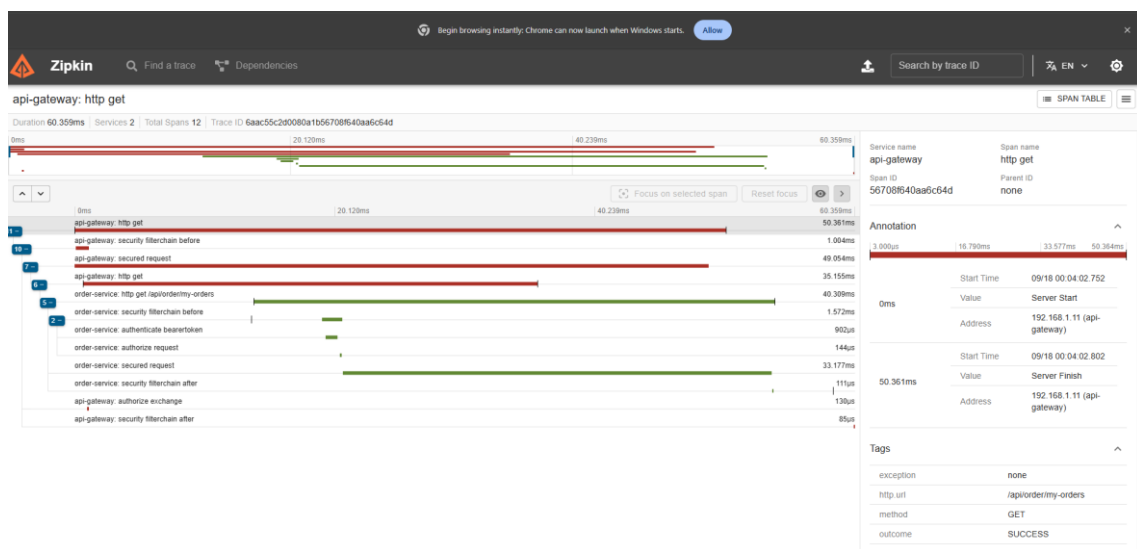
Η λειτουργία της υποδομής επιβεβαιώθηκε με οπτικά τεκμήρια για την ανακάλυψη υπηρεσιών, τους διαθέσιμους ρόλους και την κατανομημένη ιχνηλάτηση. Τα στοιχεία αυτά συμπληρώνουν τα λειτουργικά σενάρια, επειδή δείχνουν ότι η ροή της διεπαφής υποστηρίζεται από τα αντίστοιχα components Spring Cloud και Keycloak.

System Status			
Environment	test	Current time	2026-09-18T00:06:54 +0300
Data center	default	Uptime	01:15
		Lease expiration enabled	true
		Renews threshold	11
		Renews (last min)	24
DS Replicas			
localhost			
Instances currently registered with Eureka			
Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 192.168.1.11:api-gateway:8080
CART-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:cart-service:8083
INVENTORY-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:inventory-service:8082
NOTIFICATION-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:notification-service:8084
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:order-service:8181
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.1.11:product-service:8081
General Info			
Name	Value		

Εικόνα 6.12: Καταχωρισμένες υπηρεσίες στο Eureka Discovery Server.



Εικόνα 6.13: Ρόλοι ADMIN και CUSTOMER στο Keycloak realm.



Εικόνα 6.14: Trace αιτήματος από API Gateway προς Order Service.

6.4 Ανάλυση τεκμηρίων και εύρος συμπερασμάτων

Η αξιολόγηση χρησιμοποιεί τρεις κατηγορίες τεκμηρίων: διαδοχικές καταστάσεις της διεπαφής, στοιχεία από τις υποδομές και επιθεώρηση της υλοποίησης. Οι κατηγορίες αυτές απαντούν σε διαφορετικά ερωτήματα. Η διεπαφή καταγράφει τι εμφανίστηκε στον χρήστη σε μια συγκεκριμένη εκτέλεση. Οι οθόνες υποδομής δείχνουν καταχωρίσεις υπηρεσιών, ίχνη ή παραλαβές μηνυμάτων. Ο κώδικας εξηγεί τους κανόνες με τους οποίους παράγονται οι ενέργειες. Η συμφωνία τους ενισχύει την τεκμηρίωση της λειτουργικής διαδρομής, αλλά δεν καταργεί τη διάκριση μεταξύ παρατήρησης και ανάλυσης.

Στην ακύρωση, το ισχυρότερο στοιχείο συσχέτισης είναι η διατήρηση του ίδιου αριθμού παραγγελίας. Πριν από την ενέργεια εμφανίζεται PENDING και υπάρχει διαθέσιμο πλήκτρο ακύρωσης. Μεσολαβεί διάλογος επιβεβαίωσης και έπειτα εμφανίζονται CANCELLED και μήνυμα επιτυχίας. Η συγκεκριμένη ακολουθία τεκμηριώνει τη διαδρομή ακύρωσης για την παραγγελία του στιγμιότυπου. Δεν επεκτείνεται αυτόματα σε κάθε κατάσταση παραγγελίας ή σε κάθε χρονικό όριο, επειδή τέτοιες πρόσθετες εκτελέσεις δεν απεικονίζονται.

Στην αναπλήρωση αποθέματος, η σύγκριση είναι αριθμητική. Το προϊόν Windows 11 Dell Laptop εμφανίζεται αρχικά με δέκα τεμάχια, η φόρμα περιέχει αύξηση εννέα και το τελικό στιγμιότυπο εμφανίζει δεκαεννέα μαζί με επιβεβαίωση της αύξησης. Ο έλεγχος αντιπαραβάλλει είδος, αρχική ποσότητα, υποβληθείσα μεταβολή και τελική ποσότητα. Η συμφωνία $10 + 9 = 19$

είναι το ουσιαστικό αποτέλεσμα, ενώ το μήνυμα της διεπαφής λειτουργεί ως συμπληρωματική επιβεβαίωση της ενέργειας.

Στη ροή ειδοποίησης, ο αριθμός 29128251-e076-4fd0-9e96-78ef19ffb2d0 εμφανίζεται στη σελίδα επιβεβαίωσης και στο επιλεγμένο email του Mailtrap. Η ταύτιση συνδέει το μήνυμα με τη συγκεκριμένη παραγγελία και όχι απλώς με μια γενική δοκιμή SMTP. Στο RabbitMQ εμφανίζονται μηδενικά Ready και Unacked κατά τη λήψη της εικόνας και δραστηριότητα στο γράφημα. Οι τιμές αυτές δεν απομονώνουν το συγκεκριμένο γεγονός ούτε τεκμηριώνουν από μόνες τους εγγύηση ακριβώς μίας παράδοσης. Το συμπέρασμα παραμένει η παρατηρούμενη παραλαβή email στο sandbox σε συνδυασμό με την υλοποιημένη διαδρομή μηνύματος.

Για την παρατηρησιμότητα, το στιγμιότυπο Zipkin αφορά GET προς /api/order/my-orders. Αναφέρει δύο υπηρεσίες, δώδεκα spans και συνολική διάρκεια 60,359 ms. Πρόκειται για ένα συγκεκριμένο ίχνος ανάκτησης ιστορικού μέσω Gateway και Order Service. Η διάρκειά του δεν χρησιμοποιείται ως μέση απόδοση του συστήματος, ως εκατοστημόριο ή ως μέτρηση του checkout. Το τεκμήριο επιβεβαιώνει τη δυνατότητα συσχέτισης των επιμέρους βημάτων αυτής της κλήσης, με ορατή διάκριση ανάμεσα στο εξωτερικό αίτημα και στην επεξεργασία της υπηρεσίας.

Τα στιγμιότυπα Eureka και Keycloak ερμηνεύονται επίσης στο κατάλληλο επίπεδο. Οι εγγραφές UP στο Eureka δείχνουν την καταχώριση των αντίστοιχων instances κατά τον χρόνο λήψης. Οι ρόλοι ADMIN και CUSTOMER στο Keycloak δείχνουν τη διαμόρφωση του realm. Η παρουσία ρόλων δεν αποτελεί από μόνη της δοκιμή απόρριψης μη εξουσιοδοτημένου αιτήματος. Αντίστοιχα, η εμφάνιση πεδίων αναζήτησης και φίλτρων στον κατάλογο αποδεικνύει την παρουσία των χειριστηρίων, όχι κάθε πιθανό συνδυασμό χρήσης τους.

6.5 Αναπαραγωγικότητα της λειτουργικής επίδειξης

Για να επαναληφθεί μια λειτουργική διαδρομή χρειάζεται να διατηρούνται οι συνθήκες που καθορίζουν το αποτέλεσμα: ταυτότητα και ρόλος λογαριασμού, επιλεγμένο προϊόν, ποσότητα, αρχικό απόθεμα και αρχική κατάσταση παραγγελίας. Η επανάληψη δεν απαιτεί τον ίδιο τυχαία παραγόμενο αριθμό παραγγελίας. Απαιτεί όμως νέο αναγνωριστικό που παραμένει συνεπές ανάμεσα στη σελίδα επιβεβαίωσης, στο ιστορικό και στο μήνυμα ειδοποίησης. Με αυτή τη λογική, τα αναγνωριστικά χρησιμοποιούνται ως σύνδεσμοι μεταξύ τεκμηρίων και όχι ως σταθερές τιμές που πρέπει να αντιγραφούν.

Η πρακτική σειρά της επίδειξης αρχίζει με την επιβεβαίωση των υπηρεσιών στο Eureka, συνεχίζεται με σύνδεση πελάτη, επιλογή διαθέσιμου προϊόντος, προσθήκη στο καλάθι και υποβολή. Στη συνέχεια εξετάζεται το ιστορικό και το σχετικό email στο sandbox. Σε χωριστή συνεδρία διαχειριστή μπορεί να καταγραφεί αναπλήρωση αποθέματος, σημειώνοντας την αρχική ποσότητα πριν από την ενέργεια. Οι οδηγίες του Παραρτήματος Β περιγράφουν την προετοιμασία του περιβάλλοντος, ώστε η επίδειξη να μην εξαρτάται μόνο από την ύπαρξη ήδη ανοικτών εφαρμογών.

Η παρούσα αξιολόγηση είναι λειτουργική και τεκμηριωτική. Τα συμπεράσματα προκύπτουν από τις καταγεγραμμένες ενέργειες και τις εικόνες της εφαρμογής και της υποδομής. Δεν αποδίδονται αριθμητικές εγγυήσεις αντοχής σε φορτίο, ποσοστά διαθεσιμότητας ή αποτελέσματα αυτοματοποιημένων ελέγχων που δεν έχουν παραχθεί στο πλαίσιο της εργασίας.

6.6 Πίνακας ιχνηλασιμότητας των αποτελεσμάτων

Η ιχνηλασιμότητα της αξιολόγησης απαιτεί να μπορεί ο αναγνώστης να εντοπίσει το τεκμήριο στο οποίο βασίζεται κάθε συμπέρασμα. Στον Πίνακα 6.1 συγκεντρώνονται τα σενάρια και οι εικόνες που τα συνοδεύουν. Η συγκέντρωση δεν εισάγει νέες δοκιμές. Αποσαφηνίζει ποια πλευρά κάθε λειτουργίας παρατηρήθηκε και ποια πληροφορία προκύπτει από επιθεώρηση κώδικα ή διαμόρφωσης.

Πίνακας 6.1: Αντιστοίχιση σεναρίων με τα διαθέσιμα τεκμήρια.

Σενάριο	Εικόνες	Συγκεκριμένο παρατηρούμενο αποτέλεσμα
TC-01 Κατάλογος	6.1	Πλέγμα προϊόντων, τιμές και ορατά χειριστήρια. Δεν παρουσιάζεται πλήρης ακολουθία συνδυασμένων φίλτρων.
TC-02 Καλάθι	6.2	Μία γραμμή προϊόντος, ποσότητα και σύνολο. Η εικόνα δεν καταγράφει επαναφόρτωση ή διαδοχικές αλλαγές ποσότητας.
TC-03 Υποβολή	6.3–6.4	Ανασκόπηση και επιβεβαίωση με αριθμό παραγγελίας και PENDING.
TC-04 Ακύρωση	6.5–6.7	Ίδια παραγγελία πριν και μετά την ενέργεια, διάλογος επιβεβαίωσης και τελική ένδειξη CANCELLED.
TC-05 Αναπλήρωση	6.8–6.9	Ποσότητα δέκα πριν, εισαγωγή εννέα και τελικό σύνολο δεκαεννέα για το ίδιο προϊόν.
TC-06 Ειδοποίηση	6.10–6.11	Ενδείξεις broker και παραλαβή email με τον αριθμό της παραγγελίας στο Mailtrap.
Ανακάλυψη υπηρεσιών	6.12	Καταχωρισμένα instances με ένδειξη UP στο συγκεκριμένο στιγμιότυπο Eureka.
Διαμόρφωση ρόλων	6.13	Παρουσία ADMIN και CUSTOMER στο realm. Τεκμηριώνεται η διαμόρφωση, όχι ανεξάρτητη δοκιμή απαγόρευσης πρόσβασης.
Ιχνηλάτηση	6.14	Ίχνος GET ιστορικού μεταξύ Gateway και Order Service. Η διάρκεια αφορά μόνο τη συγκεκριμένη εκτέλεση.

Η αντιστοίχιση επιτρέπει να διαβαστεί η διαδρομή αγοράς ως ακολουθία σχετικών παρατηρήσεων. Η εικόνα καλαθιού περιγράφει την επιλογή ειδών, η ανασκόπηση το στάδιο πριν από την υποβολή και η επιβεβαίωση την επιστροφή αριθμού παραγγελίας. Το email προσθέτει τεκμήριο άλλου υποσυστήματος με το ίδιο αναγνωριστικό. Αντίθετα, η εικόνα Zipkin αφορά ανάκτηση ιστορικού και δεν τοποθετείται τεχνητά ως μέτρηση της ίδιας ακολουθίας checkout. Η διαφορετική προέλευση των εικόνων διατηρείται ρητή.

Για την αξιολόγηση των διαχειριστικών ενεργειών, η αναγνώριση του ίδιου προϊόντος είναι εξίσου σημαντική με τη σύγκριση ποσοτήτων. Η αύξηση αποθέματος δεν αξιολογείται από δύο αυθαίρετες γραμμές του πίνακα αλλά από τη γραμμή του επιλεγμένου SKU. Στην ακύρωση εφαρμόζεται ανάλογη λογική με τον αριθμό παραγγελίας. Αυτή η πρακτική καθιστά τα στιγμιότυπα συγκρίσιμα και αποτρέπει την εξαγωγή συμπεράσματος μόνο από ομοιότητα ονομασίας ή ποσού.

Η επανάληψη της επίδειξης μπορεί να χρησιμοποιήσει διαφορετικά προϊόντα και νέες παραγγελίες, αρκεί να διατηρείται η δομή του ελέγχου. Καταγράφεται η αρχική κατάσταση, εκτελείται μία σαφής ενέργεια και συγκρίνεται το αποτέλεσμα με το αναμενόμενο. Σε κάθε νέο κύκλο τα αναγνωριστικά συσχετίζονται εκ νέου. Οι εικόνες της εργασίας αποτελούν τεκμήρια της συγκεκριμένης επίδειξης και δεν μετατρέπονται σε αναμενόμενα σταθερά δεδομένα για κάθε μελλοντική εκτέλεση.

Συνολικά, η τεκμηρίωση συνδέει την εμπειρία χρήστη με τον σχεδιασμό των υπηρεσιών χωρίς να εξισώνει διαφορετικά επίπεδα απόδειξης. Η παρατήρηση επιτυχούς ενέργειας, η παρουσία διαμόρφωσης και η ανάλυση αλγορίθμου είναι συμπληρωματικά στοιχεία. Η λειτουργική αξιολόγηση καλύπτει τις καταγεγραμμένες διαδρομές, ενώ οι προτάσεις αυτοματοποίησης και ποσοτικών μετρήσεων παραμένουν διακριτές στη μελλοντική εργασία.

6.7 Συνοπτική αποτίμηση

Τα τεκμήρια των σεναρίων δείχνουν ότι η εφαρμογή καλύπτει την ορατή ροή catalog, cart, checkout, order history, cancellation και διαχειριστικής ενημέρωσης αποθέματος. Παράλληλα, το Eureka επιβεβαιώνει την καταχώριση των υπηρεσιών κατά τη λήψη του στιγμιότυπου, το Keycloak την οργάνωση των ρόλων και το Zipkin τη συσχέτιση ενός αιτήματος μεταξύ Gateway και Order Service.

7. Συμπεράσματα και Μελλοντική Εργασία

7.1 Συμπεράσματα

Η εργασία σχεδίασε και υλοποίησε ένα Order Management System στο οποίο οι κρίσιμες ευθύνες έχουν διαχωριστεί σε υπηρεσίες προϊόντων, καλαθιού, αποθέματος, παραγγελιών και ειδοποιήσεων. Η υλοποίηση δεν περιορίζεται στη διάσπαση του κώδικα σε διαφορετικά projects. Συνδυάζει service discovery, API Gateway, εξουσιοδότηση με JWT, δέσμευση αποθέματος, idempotency, event-driven ειδοποιήσεις, circuit breaker και distributed tracing, ώστε η κατανεμημένη αρχιτεκτονική να υποστηρίζει συγκεκριμένους επιχειρησιακούς κανόνες.

Η επιλογή Spring Boot και Spring Cloud αποδείχθηκε κατάλληλη για το αντικείμενο της εργασίας, καθώς επιτρέπει την κατασκευή υπηρεσιών με κοινό προγραμματιστικό μοντέλο Java και ταυτόχρονα προσφέρει έτοιμα στοιχεία για επαναλαμβανόμενα προβλήματα microservices. Το React frontend ενσωματώνεται στο μοντέλο αυτό μέσω ενός ενιαίου Gateway και Keycloak login flow, χωρίς να χρειάζεται να γνωρίζει τις εσωτερικές τοποθεσίες των υπηρεσιών.

- Η διαίρεση domain σε Product, Cart, Inventory, Order και Notification περιορίζει τη σύζευξη και καθιστά τις ευθύνες ευανάγνωστες.
- Η επιβεβαίωση reservation πριν από αποθήκευση order και το idempotency key περιορίζουν σημαντικούς κινδύνους της ροής checkout.
- Το RabbitMQ αποσυνδέει την ειδοποίηση πελάτη από την κρίσιμη συναλλαγή δημιουργίας παραγγελίας.
- Το Keycloak και οι έλεγχοι ρόλων στο Gateway/υπηρεσίες εφαρμόζουν προστασία σε περισσότερα από ένα επίπεδα.
- Το Zipkin δημιουργεί τεκμήριο για τη διαδρομή ενός αιτήματος σε πολλές υπηρεσίες και υποστηρίζει τη διάγνωση προβλημάτων.

7.2 Απάντηση στα ερευνητικά ερωτήματα

- EE1 — Πώς μπορεί να διασπαστεί ένα Order Management System σε microservices; Με τη διάκριση επιχειρησιακών ορίων catalog, cart, inventory, order και notification, με κάθε υπηρεσία να ελέγχει το δικό της μοντέλο δεδομένων.
- EE2 — Πώς διασφαλίζεται η ροή checkout; Με προσωρινή δέσμευση αποθέματος, επιβεβαίωση πριν από αποθήκευση παραγγελίας, idempotency key και ιστορικό κατάστασης.
- EE3 — Πώς υποστηρίζει το Spring Cloud τη συνεργασία υπηρεσιών; Με Eureka discovery, Gateway routes, OpenFeign clients και circuit breaker γύρω από κρίσιμες απομακρυσμένες κλήσεις.
- EE4 — Πώς αντιμετωπίζονται ασφάλεια και παρατηρησιμότητα; Με Keycloak/OAuth2/JWT, role-based authorization, Micrometer tracing και Zipkin.

7.3 Μελλοντική εργασία

- Προσθήκη πραγματικού payment service και σαφών αντισταθμιστικών ενεργειών για αποτυχημένη χρέωση ή ακύρωση.
- Ενσωμάτωση provider αποστολών και tracking shipment ώστε οι καταστάσεις PACKED, SHIPPED και DELIVERED να προέρχονται από πραγματικό workflow.
- Υλοποίηση transactional outbox και/ή πλήρους saga για ισχυρότερη εγγύηση δημοσίευσης event σε περίπτωση αποτυχίας μετά την τοπική αποθήκευση.
- Εκτέλεση load, stress και chaos tests με μετρικές p50/p95 latency, error rate και resource consumption ανά υπηρεσία.
- Ανάπτυξη σε container orchestration περιβάλλον με πολλαπλά replicas, εξωτερικό secrets management και προσαρμοσμένο tracing sampling.

- Επέκταση ειδοποιήσεων σε web push, SMS και προσθήκη analytics consumer στο order event.

7.4 Αποτίμηση της σχεδιαστικής συνεισφοράς

Η συνεισφορά της εφαρμογής προκύπτει από τη σύνδεση συγκεκριμένων επιχειρησιακών λειτουργιών με σαφείς μηχανισμούς συνεργασίας υπηρεσιών. Το καλάθι δεν αποτελεί μόνο προσωρινή λίστα στην οθόνη, αλλά συνδέεται με κρατήσεις αποθέματος. Η παραγγελία δεν περιορίζεται σε μια εγγραφή με ποσό, αλλά αποκτά κατάσταση, ιστορικό μεταβάσεων και αναγνωριστικό που χρησιμοποιείται στην ειδοποίηση. Ο κατάλογος δεν περιλαμβάνει μόνο περιγραφές προϊόντων, αλλά διαθέτει πρότυπα χαρακτηριστικών και ερμηνεύσιμη κατάταξη σχετικών ειδών. Αυτές οι συνδέσεις δίνουν στην τεκμηρίωση ενιαίο επιχειρησιακό αντικείμενο.

Στο επίπεδο της αρχιτεκτονικής, οι αποφάσεις δεν αξιολογούνται μόνο από τον αριθμό υπηρεσιών. Η ουσιαστική διάκριση είναι ποια υπηρεσία γνωρίζει έναν κανόνα, ποια διατηρεί τα δεδομένα του και ποια ζητά την εφαρμογή του. Η διαθεσιμότητα ανήκει στο Inventory Service, η δομή προϊόντος στο Product Service και η εξέλιξη παραγγελίας στο Order Service. Το Gateway προσφέρει σημείο εισόδου, ενώ το Eureka επιτρέπει τη συσχέτιση λογικών ονομάτων με instances. Η κατανομή αυτή εξηγεί τον ρόλο του Spring Cloud μέσα στο έργο και όχι απλώς την παρουσία του στις εξαρτήσεις Maven.

Στο επίπεδο της παρουσίασης, οι αιτιολογήσεις των σχετικών προϊόντων και οι επιτρεπόμενες ενέργειες παραγγελίας φέρνουν αποτελέσματα του backend στη διεπαφή με κατανοητό τρόπο. Ο χρήστης βλέπει γιατί προτείνεται ένα προϊόν και ποιες ενέργειες μπορεί να εκτελέσει στην τρέχουσα κατάσταση. Ο διαχειριστής ορίζει χαρακτηριστικά κατηγορίας και μεταβάλλει το απόθεμα μέσω ειδικών φορμών. Η διεπαφή καθιστά προσβάσιμους τους κανόνες χωρίς να υποκαθιστά την εφαρμογή τους στις αντίστοιχες υπηρεσίες.

Ως προς την αξιολόγηση, το αποτέλεσμα είναι μια τεκμηριωμένη λειτουργική επίδειξη και μια αναλυτική αντιστοίχιση ανάμεσα στον κώδικα και στις παρατηρούμενες οθόνες. Το checkout, η ακύρωση, η αναπλήρωση και η παραλαβή ειδοποίησης συνδέονται με συγκεκριμένα τεκμήρια. Ο αλγόριθμος και οι έλεγχοι προτύπων εξηγούνται από τις συνθήκες και τις πράξεις της υλοποίησης. Η διάκριση αυτή επιτρέπει στον αναγνώστη να γνωρίζει ποιο συμπέρασμα προκύπτει από εκτέλεση και ποιο από ανάλυση, ενώ τα παραρτήματα παρέχουν τα μέσα για ανεξάρτητη εξέταση και επανάληψη.

7.5 Σύνδεση της συνεισφοράς με τη βιβλιογραφία

Σε σύγκριση με τις γενικές αρχιτεκτονικές ανασκοπήσεις, η συνεισφορά της εργασίας είναι εφαρμοσμένη και οριοθετημένη σε ένα συγκεκριμένο επιχειρησιακό πεδίο. Παρουσιάζει πώς οι ευθύνες καταλόγου, καταθέσεων, αποθέματος, παραγγελίας και ειδοποίησης συνεργάζονται μέσω Spring Boot και Spring Cloud. Δεν επιχειρεί να αποδείξει ότι κάθε σύστημα παραγγελιών πρέπει να έχει την ίδια διάσπαση ή ότι η συγκεκριμένη επιλογή υπερέρχει ποσοτικά μιας μονολιθικής υλοποίησης. Η σύνδεση με τις ανασκοπήσεις αφορά την αιτιολόγηση των ορίων και των συνεργασιών και όχι την αναπαραγωγή των ερευνητικών τους αποτελεσμάτων [1, 3].

Αντίστοιχα, η κατάταξη σχετικών προϊόντων αποτελεί συγκεκριμένη αξιοποίηση χαρακτηριστικών περιεχομένου. Η συνεισφορά της μέσα στο έργο είναι ο συνδυασμός ελεγχόμενων γνωρισμάτων καταλόγου, σαφούς προσθετικής βαθμολογίας και λεκτικών αιτιολογήσεων που προέρχονται από τον ίδιο υπολογισμό. Δεν προτείνεται ως νέο εκπαιδευμένο μοντέλο ούτε συγκρίνεται πειραματικά με συνεργατικό φιλτράρισμα. Η τοποθέτησή της στη σχετική βιβλιογραφία διευκρινίζει γιατί ο μηχανισμός είναι ερμηνεύσιμος και ποια πληροφορία χρησιμοποιεί, χωρίς να αποδίδει στις προκαθορισμένες βαρύτητες καθολική εγκυρότητα.

Συνολικά, το τελικό τεκμήριο συνδυάζει την εφαρμογή, την εξήγηση των επιχειρησιακών κανόνων και την επιστημονική τοποθέτηση των σχεδιαστικών επιλογών. Οι βιβλιογραφικές πηγές δεν αντικαθιστούν την παρουσίαση του πραγματικού κώδικα ούτε αποδίδουν στην εφαρμογή αποτελέσματα άλλων συστημάτων. Επιτρέπουν να εξηγηθεί γιατί οι μικροϋπηρεσίες οργανώνονται γύρω από επιχειρησιακές ευθύνες και γιατί ο μηχανισμός σχετικών προϊόντων

χαρακτηρίζεται ως ερμηνεύσιμη κατάταξη βάσει περιεχομένου. Η συνεισφορά παραμένει εκείνη ενός υλοποιημένου συστήματος διαχείρισης παραγγελιών με σαφώς περιγεγραμμένες λειτουργίες και διαθέσιμα τεκμήρια.

Πίνακας ορολογίας

English term	Ελληνική απόδοση
Microservice	Μικροϋπηρεσία
Service discovery	Ανακάλυψη υπηρεσιών
API gateway	Πύλη διεπαφών εφαρμογής
Reservation	Προσωρινή δέσμευση αποθέματος
Idempotency	Ιδιοδυναμία
Circuit breaker	Διακόπτης προστασίας κλήσεων
Message broker	Διαμεσολαβητής μηνυμάτων
Distributed tracing	Κατανεμημένη ιχνηλάτηση
Span	Επιμέρους τμήμα ίχνους
Content-based recommendation	Σύσταση βάσει περιεχομένου
Authentication	Αυθεντικοποίηση
Authorization	Εξουσιοδότηση

Πίνακας ακρωνυμίων

Ακρωνύμιο	Ανάπτυξη
API	Application Programming Interface
DTO	Data Transfer Object
FR	Functional Requirement
NFR	Non-Functional Requirement
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
OMS	Order Management System
REST	Representational State Transfer
SKU	Stock Keeping Unit
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
UUID	Universally Unique Identifier

Βιβλιογραφία

- [1] Di Francesco, P., Lago, P., & Malavolta, I. (2019). Architecting with microservices: A systematic mapping study. *Journal of Systems and Software*, 150, 77-97. <https://doi.org/10.1016/j.jss.2019.01.001>
- [2] Dragoni, N., Giallorenzo, S., Lafuente, A. L., Mazzara, M., Montesi, F., Mustafin, R., & Safina, L. (2017). Microservices: Yesterday, today, and tomorrow. In M. Mazzara & B. Meyer (Eds.), *Present and ulterior software engineering* (pp. 195-216). Springer, Cham. https://doi.org/10.1007/978-3-319-67425-4_12
- [3] Soldani, J., Tamburri, D. A., & van den Heuvel, W.-J. (2018). The pains and gains of microservices: A systematic grey literature review. *Journal of Systems and Software*, 146, 215-232. <https://doi.org/10.1016/j.jss.2018.09.082>
- [4] Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45-77. <https://doi.org/10.2753/MIS0742-1222240302>
- [5] Alshuqayran, N., Ali, N., & Evans, R. (2016). A systematic mapping study in microservice architecture. In 2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA) (pp. 44-51). IEEE. <https://doi.org/10.1109/SOCA.2016.15>
- [6] Jamshidi, P., Pahl, C., Mendonça, N. C., Lewis, J., & Tilkov, S. (2018). Microservices: The journey so far and challenges ahead. *IEEE Software*, 35(3), 24-35. <https://doi.org/10.1109/MS.2018.2141039>
- [7] Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.
- [8] Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning.
- [9] Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12), 1053–1058. <https://doi.org/10.1145/361598.361623>
- [10] Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2nd ed.). Pragmatic Bookshelf.
- [11] Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.
- [12] Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine). <https://roy.gbiv.com/pubs/dissertation/top.htm>
- [13] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387. <https://doi.org/10.1145/362384.362685>
- [14] Garcia-Molina, H., & Salem, K. (1987). Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data* (pp. 249-259). ACM. <https://doi.org/10.1145/38713.38742>
- [15] Helland, P. (2007). Life beyond distributed transactions: An apostate's opinion. In *Proceedings of CIDR 2007*. <http://cidrdb.org/cidr2007/papers/cidr07p15.pdf>
- [16] Spring. (χ.χ.). *Spring Data JPA Transactionality*. <https://docs.spring.io/spring-data/jpa/reference/jpa/transactions.html>
- [17] PostgreSQL Global Development Group. (χ.χ.). *PostgreSQL Documentation Transaction Isolation*. <https://www.postgresql.org/docs/current/transaction-iso.html>
- [18] Eugster, P. Th., Felber, P. A., Guerraoui, R., & Kermarrec, A.-M. (2003). The many faces of publish/subscribe. *ACM Computing Surveys*, 35(2), 114–131. <https://doi.org/10.1145/857076.857078>
- [19] Gilbert, S., & Lynch, N. (2002). Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59. <https://doi.org/10.1145/564585.564601>

- [20] Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., & Stoica, I. (2014). Coordination avoidance in database systems. *Proceedings of the VLDB Endowment*, 8(3), 185–196. <https://doi.org/10.14778/2735508.2735509>
- [21] Spring. (χ.χ.). Spring Boot 3.5 Reference Documentation. <https://docs.spring.io/spring-boot/3.5/reference/index.html>
- [22] Spring. (χ.χ.). Spring Cloud. <https://spring.io/projects/spring-cloud/>
- [23] Spring. (χ.χ.). Spring Cloud Netflix 4.3 Reference Documentation. <https://docs.spring.io/spring-cloud-netflix/reference/4.3/index.html>
- [24] Spring. (χ.χ.). Spring Cloud Gateway 4.3 Server WebFlux Reference Documentation. <https://docs.spring.io/spring-cloud-gateway/reference/4.3/spring-cloud-gateway-server-webflux.html>
- [25] Spring. (χ.χ.). Spring Cloud OpenFeign 4.3 Reference Documentation. <https://docs.spring.io/spring-cloud-openfeign/reference/4.3/index.html>
- [26] RabbitMQ. (χ.χ.). Publishers. <https://www.rabbitmq.com/docs/publishers>
- [27] Spring. (χ.χ.). Spring AMQP Reference Documentation. <https://docs.spring.io/spring-amqp/reference/index.html>
- [28] Fett, D., Küsters, R., & Schmitz, G. (2016). A comprehensive formal security analysis of OAuth 2.0. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS 2016)* (pp. 1204–1215). ACM. <https://doi.org/10.1145/2976749.2978385>
- [29] Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *Computer*, 29(2), 38–47. <https://doi.org/10.1109/2.485845>
- [30] Keycloak. (χ.χ.). Server Administration Guide. https://www.keycloak.org/docs/latest/server_admin/index.html
- [31] Spring. (χ.χ.). Spring Security OAuth 2.0 Resource Server JWT. <https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html>
- [32] Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9), 1278–1308. <https://doi.org/10.1109/PROC.1975.9939>
- [33] Resilience4j. (χ.χ.). CircuitBreaker. <https://resilience4j.readme.io/docs/circuitbreaker>
- [34] Micrometer. (χ.χ.). Micrometer Tracing Reference Documentation. <https://docs.micrometer.io/tracing/reference/>
- [35] OpenZipkin. (χ.χ.). Zipkin Architecture. <https://zipkin.io/pages/architecture.html>
- [36] Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6), 734–749. <https://doi.org/10.1109/TKDE.2005.99>
- [37] Jannach, D., Zanker, M., Felfernig, A., & Friedrich, G. (2010). *Recommender systems: An introduction*. Cambridge University Press. <https://doi.org/10.1017/CBO9780511763113>
- [38] Schafer, J. B., Konstan, J. A., & Riedl, J. (2001). E-commerce recommendation applications. *Data Mining and Knowledge Discovery*, 5(1–2), 115–153. <https://doi.org/10.1023/A:1009804230409>
- [39] Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). *Recommender systems handbook* (3rd ed.). Springer, New York. <https://doi.org/10.1007/978-1-0716-2197-4>
- [40] Lops, P., de Gemmis, M., & Semeraro, G. (2011). Content-based recommender systems: State of the art and trends. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 73–105). Springer, New York. https://doi.org/10.1007/978-0-387-85820-3_3
- [41] Pazzani, M. J., & Billsus, D. (2007). Content-based recommendation systems. In P. Brusilovsky, A. Kobsa, & W. Nejdl (Eds.), *The adaptive web: Methods and strategies of web personalization* (pp. 325–341). Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-540-72079-9_10

- [42] Zhang, Y., & Chen, X. (2020). Explainable recommendation: A survey and new perspectives. *Foundations and Trends in Information Retrieval*, 14(1), 1-101. <https://doi.org/10.1561/15000000066>
- [43] Tintarev, N., & Masthoff, J. (2011). Designing and evaluating explanations for recommender systems. In F. Ricci, L. Rokach, B. Shapira, & P. B. Kantor (Eds.), *Recommender systems handbook* (pp. 479–510). Springer, New York. https://doi.org/10.1007/978-0-387-85820-3_15
- [44] Gysel, M., Kölbener, L., Giersche, W., & Zimmermann, O. (2016). Service Cutter: A systematic approach to service decomposition. In M. Aiello, E. Broch Johnsen, S. Dustdar, & I. Georgievski (Eds.), *Service-Oriented and Cloud Computing (ESOCC 2016)* (pp. 185–200). Springer, Cham. https://doi.org/10.1007/978-3-319-44482-6_12
- [45] React. (χ.χ.). Quick Start. <https://react.dev/learn>
- [46] Vite. (χ.χ.). Getting Started. <https://vite.dev/guide/>
- [47] Spring. (χ.χ.). Spring Data Redis Reference Documentation. <https://docs.spring.io/spring-data/redis/reference/>
- [48] Redis. (χ.χ.). Redis Hashes. <https://redis.io/docs/latest/develop/data-types/hashtes/>
- [49] Docker. (χ.χ.). Docker Compose. <https://docs.docker.com/compose/>
- [50] Mailtrap. (χ.χ.). Email Sandbox. <https://docs.mailtrap.io/email-sandbox>

Παράρτημα Α Αντιπροσωπευτικός πηγαίος κώδικας

Τα ακόλουθα αποσπάσματα προέρχονται από την έκδοση του αποθετηρίου που εξετάζεται στην εργασία. Παρουσιάζονται οι μέθοδοι που συνδέουν τις επιχειρησιακές απαιτήσεις με την υλοποίηση. Οι εισαγωγές κλάσεων και οι μη σχετικές μέθοδοι παραλείπονται, ενώ ο πλήρης κώδικας διατίθεται στο αποθετήριο του Παραρτήματος Γ.

A.1 Βαθμολόγηση σχετικών προϊόντων

Αρχείο: backend/product-service/src/main/java/
com/example/thesis/product_service/service/ProductService.java

```
private RelatedCandidate scoreRelatedProduct(Product source, Product
candidate) {
    int score = 0;
    List<String> reasons = new ArrayList<>();

    if (sameText(source.getCategory(), candidate.getCategory())) {
        score += 40;
        reasons.add("Same category");
    }

    int matchingSpecs = countMatchingSpecifications(source, candidate);
    if (matchingSpecs > 0) {
        score += Math.min(matchingSpecs * 12, 25);
        reasons.add(matchingSpecs == 1 ? "1 matching specification" :
matchingSpecs + " matching specifications");
    }

    int matchingWords = countMatchingWords(source.getName(),
candidate.getName());
    if (matchingWords > 0) {
        score += Math.min(matchingWords * 7, 15);
        reasons.add("Similar product name");
    }

    int priceScore = calculatePriceScore(source.getPrice(),
candidate.getPrice());
    if (priceScore > 0) {
        score += priceScore;
        reasons.add("Similar price range");
    }

    if (reasons.isEmpty()) {
        reasons.add("Alternative from the catalogue");
    }

    return new RelatedCandidate(candidate, score, reasons);
}
```

A.2 Έλεγχος προτύπου κατηγορίας

Αρχείο: backend/product-service/src/main/java/
com/example/thesis/product_service/service/ProductService.java

```
private Category validateCharacteristics(ProductRequest request) {
```

```

        if (request.category() == null || request.category().isBlank()) {
            throw new IllegalArgumentException("A category is required.");
        }
        Category category =
categoryRepository.findByNameIgnoreCase(request.category().trim())
        .orElseThrow(() -> new IllegalArgumentException("Select a category created in
the category manager."));

        Map<String, String> submitted = new HashMap<>();
        if (request.attributes() != null) {
            for (ProductAttributeRequest attribute : request.attributes()) {
                if (attribute.key() == null || attribute.value() == null ||
attribute.key().isBlank() || attribute.value().isBlank()) {
                    throw new IllegalArgumentException("Characteristics must
have both a name and value.");
                }
                String key = normalize(attribute.key());
                if (submitted.put(key, attribute.value().trim()) != null) {
                    throw new IllegalArgumentException("A characteristic can
only be included once.");
                }
            }
        }

        Map<String, CategoryAttributeDefinition> definitions = new
HashMap<>();
        for (CategoryAttributeDefinition definition :
category.getAttributeDefinitions()) {
            definitions.put(normalize(definition.getAttributeName()),
definition);
            if (definition.isRequired() &&
!submitted.containsKey(normalize(definition.getAttributeName()))) {
                throw new IllegalArgumentException("Missing required
characteristic: " + definition.getAttributeName());
            }
        }

        for (Map.Entry<String, String> characteristic : submitted.entrySet())
        {
            CategoryAttributeDefinition definition =
definitions.get(characteristic.getKey());
            if (definition == null) {
                throw new IllegalArgumentException("Characteristic is not
allowed for this category: " + characteristic.getKey());
            }
            validateCharacteristicValue(definition,
characteristic.getValue());
        }
        return category;
    }
}

```

A.3 Επιβεβαίωση δέσμευσης

Αρχείο: backend/inventory-service/src/main/java/
com/example/thesis/inventory_service/service/InventoryService.java

```

    public void confirmReservation(UUID orderNumber) {

```

Ανάπτυξη ενός Συστήματος Διαχείρισης Παραγγελιών (Order Management System)
Βασισμένου σε Microservices με Spring Boot και Spring Cloud

```
        List<InventoryReservation> reservations =  
reservationRepository.findByOrderNumber(orderNumber);  
        for (InventoryReservation r : reservations) {  
            InventoryItem item = itemRepository.findBySkuCode(r.getSkuCode())  
.orElseThrow(() -> new RuntimeException("SKU not found"));  
            item.setTotalQuantity(item.getTotalQuantity() -  
r.getReservedQuantity());  
            itemRepository.save(item);  
        }  
        reservationRepository.deleteByOrderNumber(orderNumber);  
    }  
}
```

A.4 Κατανάλωση γεγονότος ειδοποίησης

Αρχείο: backend/notification-service/src/main/java/
com/example/thesis/notification_service/NotificationService.java

```
    public void handleNotification(OrderPlacedEvent orderPlacedEvent) {  
        if (!emailProperties.isEnabled()) {  
            log.info("Email notification disabled for order {}",  
orderPlacedEvent.getOrderNumber());  
            return;  
        }  
  
        try {  
            sendOrderPlacedEmail(orderPlacedEvent);  
            log.info("Email notification sent for order {} to {}",  
orderPlacedEvent.getOrderNumber(), orderPlacedEvent.getEmail());  
        } catch (MessagingException ex) {  
            log.error("Failed to build email notification for order {}",  
orderPlacedEvent.getOrderNumber(), ex);  
            if (emailProperties.isFailOnError()) {  
                throw new IllegalStateException("Failed to build email  
notification", ex);  
            }  
        } catch (RuntimeException ex) {  
            log.error("Failed to send email notification for order {}",  
orderPlacedEvent.getOrderNumber(), ex);  
            if (emailProperties.isFailOnError()) {  
                throw ex;  
            }  
        }  
    }  
}
```

Παράρτημα Β Οδηγίες τοπικής εκτέλεσης

Β.1 Προϋποθέσεις και δομή έργου

Απαιτούνται JDK 21, Docker με Docker Compose, Node.js συμβατό με το Vite 7 του έργου και πρόσβαση σε PostgreSQL στη θύρα 5433. Τα έργα backend διαθέτουν Maven Wrapper, επομένως χρησιμοποιείται το mvnw.cmd στα Windows ή το ./mvnw σε περιβάλλον Unix. Η εγκατάσταση των εξαρτήσεων frontend ακολουθεί το package-lock.json μέσω npm ci. Οι αριθμοί θυρών που ακολουθούν αντιστοιχούν στην τοπική διαμόρφωση του αποθετηρίου.

Β.2 Υποδομές και βάσεις δεδομένων

Από τον ριζικό κατάλογο εκτελείται docker compose up -d και ελέγχεται η κατάσταση με docker compose ps. Το συγκεκριμένο αρχείο Compose εκκινεί RabbitMQ, Zipkin, Keycloak και Redis. Δεν εκκινεί τις εφαρμογές Spring Boot, το React ή PostgreSQL. Η βάση PostgreSQL προετοιμάζεται χωριστά με τις βάσεις product_db, inventory_db και order_db, σύμφωνα με τα spring.datasource.url των αντίστοιχων υπηρεσιών. Τα διαπιστευτήρια ορίζονται τοπικά και δεν αντιγράφονται στο κείμενο της εργασίας.

Οι βασικές διευθύνσεις διαχείρισης είναι http://localhost:8180 για Keycloak, http://localhost:15672 για RabbitMQ και http://localhost:9411 για Zipkin. Το RabbitMQ χρησιμοποιεί επιπλέον τη θύρα 5672 για AMQP και το Redis τη θύρα 6379. Επιβεβαιώνεται ότι οι θύρες δεν χρησιμοποιούνται από άλλη εφαρμογή πριν από την εκκίνηση.

Β.3 Διαμόρφωση ταυτότητας

Στο Keycloak προετοιμάζεται realm με αναγνωριστικό order-system-realm και client με αναγνωριστικό order-system-client, σε συμφωνία με τη διαμόρφωση της εφαρμογής. Δημιουργούνται οι ρόλοι ADMIN και CUSTOMER και αποδίδονται στους αντίστοιχους δοκιμαστικούς λογαριασμούς. Τα επιτρεπόμενα origins και redirect URLs του client αντιστοιχίζονται στη διεύθυνση τοπικής εκτέλεσης του frontend. Η διεύθυνση issuer των resource servers πρέπει να συμφωνεί με το http://localhost:8180/realms/order-system-realm.

Στο frontend ορίζονται οι μεταβλητές VITE_API_BASE_URL=http://localhost:8080/api, VITE_KEYCLOAK_URL=http://localhost:8180/, VITE_KEYCLOAK_REALM=order-system-realm, VITE_KEYCLOAK_CLIENT_ID=order-system-client και VITE_KEYCLOAK_TOKEN_URL=http://localhost:8180/realms/order-system-realm/protocol/openid-connect/token. Η συμφωνία των ονομάτων και των διευθύνσεων είναι απαραίτητη για τη σύνδεση και την ανανέωση token.

Β.4 Εκκίνηση εφαρμογών

Σε χωριστό τερματικό ανοίγεται ο κατάλογος backend/discovery-server και εκτελείται mvnw.cmd spring-boot:run. Ακολουθούν οι κατάλογοι backend/product-service, backend/inventory-service, backend/cart-service, backend/order-service και backend/notification-service με την ίδια εντολή σε ξεχωριστά τερματικά. Έπειτα εκκινείται το backend/api-gateway. Οι εφαρμογές Java δεν εκκινούν όλες από ένα κοινό root pom.xml αλλά από τα επιμέρους έργα τους.

Οι θύρες είναι 8761 για discovery-server, 8081 για product-service, 8082 για inventory-service, 8083 για cart-service, 8181 για order-service, 8084 για notification-service και 8080 για api-gateway. Στο http://localhost:8761 ελέγχεται η εμφάνιση των καταχωρισμένων υπηρεσιών. Η αναμονή ολοκλήρωσης της αρχικοποίησης προηγείται της λειτουργικής επίδειξης.

Από τον κατάλογο frontend εκτελούνται npm ci και npm run dev. Χρησιμοποιείται η διεύθυνση που εμφανίζει το Vite στο τερματικό, συνήθως http://localhost:5173. Για παραγωγή

των στατικών αρχείων εκτελείται `npm run build` και για τοπική προεπισκόπηση του `build` `npm run preview`. Η ανάπτυξη του frontend δεν υποκαθιστά την εκκίνηση των backend υπηρεσιών.

B.5 Ειδοποιήσεις και επανάληψη επίδειξης

Για δοκιμή με Mailtrap ορίζονται στο περιβάλλον του `notification-service` οι `MAIL_HOST`, `MAIL_PORT`, `MAIL_USERNAME` και `MAIL_PASSWORD` από το `sandbox` του χρήστη, μαζί με `MAIL_SMTP_AUTH` και τις ρυθμίσεις `STARTTLS` που αντιστοιχούν στον λογαριασμό. Η `NOTIFICATION_EMAIL_ENABLED` ενεργοποιεί την αποστολή και η `MAIL_FROM` ορίζει αποστολέα. Τα πραγματικά μυστικά παραμένουν εκτός εγγράφου και δημόσιου αποθετηρίου.

Η επίδειξη εκτελείται με προϊόν που έχει αρχικό απόθεμα. Καταγράφονται ο αριθμός της νέας παραγγελίας, η κατάσταση της και η εμφάνιση του ίδιου αριθμού στο Mailtrap. Για αναπλήρωση καταγράφονται αρχική ποσότητα, ποσότητα προσθήκης και τελικό σύνολο. Για ακύρωση επιλέγεται παραγγελία με διαθέσιμη σχετική ενέργεια και διατηρούνται στιγμιότυπα πριν και μετά. Οι ενέργειες αυτές μεταβάλλουν τα τοπικά δοκιμαστικά δεδομένα.

Η διακοπή των εφαρμογών γίνεται από τα τερματικά τους και η διακοπή των υποδομών με `docker compose stop`. Η επανεκκίνηση δεν απαιτεί διαγραφή `volumes`. Τα δεδομένα και οι ρυθμίσεις του τοπικού περιβάλλοντος διατηρούνται για την επανάληψη της επίδειξης.

Παράρτημα Γ Αποθετήριο και ψηφιακό υλικό

Αποθετήριο πηγαίου κώδικα:

<https://github.com/Arankou7/OrderManagmentSystemFinal>

Αναγνωριστικό της καταγεγραμμένης έκδοσης Git:

becb6e32a0b9c75a8da0cfa71606c161d16adf65. Τα αποσπάσματα του Παραρτήματος Α προέρχονται από τα τοπικά αρχεία της εφαρμογής που χρησιμοποιήθηκαν κατά την τεκμηρίωση.

Ο κατάλογος backend περιλαμβάνει τα ανεξάρτητα έργα Maven των υπηρεσιών. Ο κατάλογος frontend περιέχει την εφαρμογή React και το αρχείο docker-compose.yml ορίζει τις τοπικές υποδομές. Η πρόσβαση στο πλήρες έργο επιτρέπει την αντιπαράβολή των αποσπασμάτων του Παραρτήματος Α με τις αντίστοιχες κλάσεις και την αναπαραγωγή του περιβάλλοντος του Παραρτήματος Β. Κωδικοί πρόσβασης και διαπιστευτήρια υπηρεσιών δεν αποτελούν τμήμα της παρούσας τεκμηρίωσης.